



中国科学院大学
University of Chinese Academy of Sciences

博士学位论文

访存友好的通用图形处理器设计研究

作者姓名：赵皓宇

指导教师：胡伟武 研究员 中国科学院计算技术研究所

学位类别：工学博士

学科专业：计算机系统结构

培养单位：中国科学院计算技术研究所

2024 年 6 月

**A Study of Memory-Access-Friendly General-Purpose Graphics
Processor**

**A dissertation submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Doctor of Engineering
in Computer System Architecture**

By

Zhao Haoyu

Supervisor: Professor Hu Weiwu

Institute of Computing Technology, Chinese Academy of Sciences

June, 2024

中国科学院大学
学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：

日 期：

中国科学院大学
学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以按照学术研究公开原则和保护知识产权的原则公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘要

通用图形处理器 (General-Purpose Graphics Processing Unit, 简称 GPGPU) 被广泛应用于图形处理、科学计算和深度学习等领域。通用图形处理器的内存带宽远低于其计算性能, 导致 GPGPU 在访存密集型应用上性能受限。GPGPU 需要设计高效的访存架构来弥合内存带宽和计算性能间的差距。高效的 GPGPU 访存架构设计的主要挑战有两点: 一是相对较小的缓存容量, 导致缓存争用和缓存抖动问题; 二是相对较长的访存延迟, 不能被并发访存请求所充分掩盖, 导致访存带宽利用率低。因此本文从缓存命中率、平均并发访存请求数和平均访存延迟三个层面来优化 GPGPU 的访存架构, 设计了一个访存友好的通用图形处理器。本文的贡献如下:

1. 设计了统一缓存架构的通用图形处理器 (Unified-Cache GPGPU), 通过增大缓存容量来提高缓存命中率。在传统的图形渲染流水线中, 着色器之间通过固定功能流水线的专用片上存储进行数据传输。在 GPGPU 执行通用计算时, 这些片上存储往往处于闲置状态。本文将该片上存储与缓存相统一, 使得在 GPGPU 执行通用计算时, 该片上存储可以作为缓存被充分利用, 从而增大了 GPGPU 的缓存容量。实验结果表明, 统一缓存架构的 GPGPU 与传统 GPGPU 相比在缓存高效型应用上的性能平均提高了 35.4%。

2. 设计了基于重命名的标记解耦合一级缓存 (Decoupled-Tag Cache), 以增加 GPGPU 的平均并发访存请求数。GPGPU 的平均访存延迟可以达到数百周期, 传统一级缓存由于等待指令缓冲区、缺失状态保存寄存器 (Missing Status Holding Register, MSHR) 以及组相联结构, 会限制 GPGPU 并发访存请求的数量, 导致 GPGPU 无法有效地掩盖长延迟访存带来的性能损失。本文通过重命名的方法将标记 (Tag) 和缓存行的映射解耦合, 并消除了 MSHR 和传统容量有限的等待指令缓冲区结构的限制, 使得 GPGPU 能够发出更多的缺失访存请求, 从而提高了其掩盖访存延迟的能力和访存带宽利用率。实验结果显示, 基于重命名的标记解耦合一级缓存与传统一级缓存相比在图形渲染应用上提高了超过 50.0% 的性能, 在所有应用上平均提高了 30.0% 的性能。

3. 设计了流量感知的可重配置热点缓存 (Re-Hot Cache), 通过重配置 GPGPU 的图形固定功能流水线中的重排序缓冲区为中间层级的热点缓存, 以减小访存延迟。图形固定功能流水线上的像素着色的重排序缓冲区的大小随着流多处理

器数量的增加而增大，引入了可观的片上存储开销，但在 GPGPU 执行通用计算时却处于闲置状态。本文提出了将图形固定功能流水线的缓冲区重配置为通用计算的高速缓存，并通过监控二级缓存行的访问频率，仅存储高访问频率的缓存行，优化了缓存管理策略，从而提高了缓存效率。实验结果表明，Re-Hot Cache 与传统 GPGPU 架构相比以较低的硬件开销平均提高了二级缓存高效型应用 8.61% 的性能。

上述部分工作已经在龙芯处理器中得到具体应用。

关键词：通用图形处理器，访存，缓存，图形固定功能流水线

Abstract

General-Purpose Graphics Processing Units (GPGPUs) are widely used in fields such as graphics processing, scientific computing, and deep learning. The memory bandwidth of GPGPUs is significantly lower than their computational performance, leading to performance limitations in memory-intensive applications. An efficient memory architecture design is crucial for GPGPUs to bridge the gap between memory bandwidth and computational performance. The main challenges in designing efficient GPGPU memory architecture are twofold: first, the relatively small cache capacity leads to cache contention and cache thrashing issues; second, the relatively long memory access latency cannot be sufficiently covered by concurrent memory access requests, resulting in low memory bandwidth utilization. Consequently, this thesis proposes optimizing the memory architecture of GPGPUs from three aspects: cache hit rate, average concurrent memory access requests, and average memory access latency, culminating in a memory-access-friendly GPGPU design. The contributions of this paper are as follows:

1. A Unified-Cache General-Purpose Graphics Processing Unit (Unified-Cache GPGPU) is designed to enhance cache hit rate by increasing cache capacity. In traditional graphics rendering pipelines, data transfer between shaders is facilitated by dedicated on-chip storage within fixed-function pipelines. During GPGPU execution of general computations, these on-chip storages often remain idle. This thesis unifies these on-chip storages with caches, allowing them to be fully utilized as caches during GPGPU execution of general computations, thereby increasing the cache capacity of GPGPUs. Experimental results demonstrate that Unified-Cache GPGPUs exhibit an average performance improvement of 35.4% on cache-efficient applications compared to traditional GPGPUs.

2. A Decoupled-Tag L1 Cache based on renaming (Decoupled-Tag Cache) is designed to increase the average concurrent memory access requests of GPGPU. The average memory access latency of GPGPU can reach several hundred cycles. Traditional L1 caches, due to the presence of pending instruction buffers, Missing Status Holding Registers (MSHR), and set-associative structures, limit the number of concur-

rent memory access requests of GPGPU, resulting in the inability to effectively cover the performance loss caused by long-latency memory accesses. This paper decouples the mapping of tags and cache lines by renaming, eliminating the limitations of MSHR and the traditional, capacity-limited pending instruction buffer structure. This allows GPGPU to issue more missing memory access requests, thereby enhancing its ability to cover memory access latency and improve memory bandwidth utilization. Experimental results show that the Decoupled-Tag L1 Cache based on renaming improves performance by over 50.0% in graphics rendering applications and achieves an average performance improvement of 30.0% across all applications compared to the traditional L1 cache.

3. A traffic-aware Reconfigurable Hot Cache (Re-Hot Cache) is designed to reduce memory access latency by reconfiguring the reorder buffer in the fixed-function graphics pipeline of GPGPU into a mid-level hot cache. The size of the reorder buffer for pixel shading on the fixed-function graphics pipeline increases with the number of streaming processors, introducing a significant on-chip storage overhead. However, during GPGPU execution of general-purpose computations, the buffer remains idle. This thesis proposes reconfiguring the buffer of the fixed-function graphics pipeline into a cache for general-purpose computation. Furthermore, by monitoring the access frequency of cache lines and selectively storing cache lines with high access frequency, cache management policies are optimized, and the efficiency of the cache is enhanced. Experimental results demonstrate that Re-Hot Cache achieves an average performance improvement of 8.61% on L2-cache-efficient applications with relatively low hardware overhead compared to the traditional GPGPU architecture.

Some of the aforementioned work has been applied in the Loongson processors.

Keywords: General-purpose Graphics Processor, Memory Access, Cache, Graphics Fixed-Function Pipeline

目 录

第 1 章 引言	1
1.1 通用图形处理器概述	1
1.1.1 通用图形处理器架构	1
1.1.2 通用图形处理器内存层次	2
1.1.3 传统通用图形处理器架构面临的挑战	3
1.2 通用图形处理器访存优化研究现状	4
1.2.1 调度策略	4
1.2.2 缓存设计	5
1.2.3 片上互连网络	6
1.2.4 内存控制器	6
1.3 主要研究内容及贡献	7
1.3.1 主要研究路线	7
1.3.2 主要贡献	7
1.4 本文的内容组织	10
第 2 章 通用图形处理器访存优化的研究背景与现状	11
2.1 图形渲染应用	11
2.1.1 图形渲染流水线	11
2.1.2 图形渲染应用的数据存储格式	12
2.1.3 图形渲染数据的存储格式总结	14
2.2 通用计算应用	15
2.2.1 计算密集型应用	15
2.2.2 访存密集型应用	15
2.2.3 图形渲染应用和通用计算应用的比较	16
2.3 通用图形处理器架构的演进	17
2.3.1 固定功能图形流水线	17
2.3.2 可编程着色器	18
2.3.3 统一着色器架构	18
2.3.4 通用计算支持	18
2.3.5 高性能计算	20
2.4 通用图形处理器访存优化的研究现状	21
2.4.1 调度策略	21
2.4.2 缓存设计	22

2.4.3 片上互连网络	23
2.4.4 内存控制器	24
2.4.5 不同访存优化策略的比较	25
2.4.6 实验平台	26
第 3 章 统一缓存架构的通用图形处理器设计	29
3.1 相关背景：图形处理器的着色器输入输出缓冲区	29
3.2 设计动机：通用图形处理器的缓存低效问题	30
3.3 统一缓存架构的通用图形处理器架构设计	32
3.3.1 设计理念：统一缓存、便笺式存储和着色器输入输出缓冲区	32
3.3.2 固定功能流水线	34
3.3.3 流多处理器	35
3.3.4 片上互连网络	36
3.4 统一缓存架构的软硬件协同实现	36
3.4.1 一级数据缓存和便笺式存储的统一	37
3.4.2 便笺式存储和 GPC 内 Shader IOB 的统一	38
3.4.3 二级缓存和 GPC 间 Shader IOB 的统一	40
3.4.4 可扩展性分析	43
3.5 实验结果分析	43
3.5.1 实验评估方法	43
3.5.2 二级缓存大小与性能的关系	43
3.5.3 一级缓存大小与性能的关系	44
3.5.4 片外数据访问量	45
3.5.5 流多处理器组数量与性能的关系	46
3.6 本章总结	47
第 4 章 基于重命名的标记解耦合的一级缓存设计	49
4.1 设计动机：传统一级缓存架构限制了并发访存请求数	49
4.1.1 传统一级缓存架构	49
4.1.2 传统一级缓存架构访存瓶颈的量化分析	51
4.2 设计理念：消除传统一级缓存架构对并发请求数的限制	53
4.2.1 消除紧耦合组相联映射导致的访存阻塞	53
4.2.2 消除 MSHR 导致的访存阻塞	53
4.2.3 消除等待指令缓冲区导致的访存阻塞	54
4.3 基于重命名的标记解耦合的一级缓存实现	54
4.3.1 顺序的基于重命名的标记解耦合的一级缓存实现	55
4.3.2 乱序的基于重命名的标记解耦合的一级缓存的实现	57

4.3.3 基于重命名的标记解耦合的一级缓存的开销	61
4.4 基于重命名的标记解耦合的一级缓存与缓存旁路的兼容性	62
4.5 实验结果分析	63
4.5.1 实验评估方法	63
4.5.2 性能分析	63
4.5.3 面积分析	64
4.5.4 平均并发缺失请求数	64
4.5.5 逻辑缓存空间大小与性能的关系	66
4.5.6 物理缓存空间大小与性能的关系	67
4.5.7 等待指令缓冲区大小与性能的关系	67
4.6 本章总结	69
第 5 章 流量感知的可重配置热点缓存设计	71
5.1 设计动机	71
5.1.1 闲置的固定功能流水线的片上存储	71
5.1.2 二级缓存的缓存争用	72
5.2 流量感知的可重配置热点缓存的实现	74
5.2.1 流量感知的可重配置热点缓存架构	74
5.2.2 流量感知的可重配置热点缓存的执行流程	76
5.2.3 流量感知的可重配置热点缓存的硬件开销	78
5.3 实验结果分析	78
5.3.1 实验评估方法	78
5.3.2 加速比	79
5.3.3 缓存命中率	79
5.3.4 平均访存延迟	80
5.3.5 一级缓存大小对流量感知的可重配置热点缓存性能的影响	81
5.3.6 访存友好的通用图形处理器的性能分析	81
5.4 本章总结	82
第 6 章 总结与展望	83
6.1 本文总结	83
6.2 展望	84
参考文献	87
致谢	93
作者简历及攻读学位期间发表的学术论文与研究成果	95

图形列表

1.1 通用图形处理器架构·····	2
1.2 通用图形处理器的内存层次架构及研究现状·····	5
1.3 访存友好的通用图形处理器架构研究路线·····	8
2.1 图形渲染流水线·····	11
2.2 图形渲染应用的数据存储格式 (a) 顶点数据的存储格式, (b) 纹理数据的存储格式, (c) 帧缓冲区数据的存储格式·····	13
2.3 LGPU 图形处理器架构·····	26
3.1 一级数据缓存的冷缺失率和容量缺失率·····	32
3.2 统一缓存架构的通用图形处理器架构·····	33
3.3 统一缓存架构的通用图形处理器的流多处理器架构·····	35
3.4 统一的一级数据缓存和便笺式存储的架构·····	37
3.5 基于便笺式存储的 Shader IO 的编译器实现 (a) 顶点和外壳着色器间的 Shader IO, (b) 外壳和域着色器间的 Shader IO, (c) 顶点、域和几何着色器间的 Shader IO·····	39
3.6 基于便笺式存储的 GPC 内的 Shader IO 的执行流程·····	40
3.7 基于二级缓存的 GPC 间的 Shader IO 的架构·····	42
3.8 不同二级缓存大小的性能比较·····	44
3.9 不同一级缓存大小的性能比较·····	45
3.10 Unified-Cache GPGPU 和传统 GPGPU 的片外数据访问量的比较·····	46
3.11 不同流多处理器数目的性能比较·····	46
4.1 传统一级缓存架构·····	50
4.2 一级缓存访存瓶颈的量化分析·····	51
4.3 IO-Decoupled-Tag Cache 的实现 (a) IO-Decoupled-Tag Cache 的架构, (b) IO-Decoupled-Tag Cache 的执行流程·····	55
4.4 OO-Decoupled-Tag Cache 的实现 (a) OO-Decoupled-Tag Cache 的架构, (b) OO-Decoupled-Tag Cache 的执行流程·····	58
4.5 Decoupled-Tag Cache 和传统一级缓存的性能比较·····	64
4.6 Decoupled-Tag Cache 的各模块的面积占比 (a) IO-Decoupled-Tag Cache 的各模块的面积占比, (b) OO-Decoupled-Tag Cache 的各模块的面积占比·····	65
4.7 Decoupled-Tag Cache 和传统一级缓存的平均并发缺失请求数的比较·····	65

4.8 不同缓存大小的 Decoupled-Tag Cache 的性能比较 (a) 不同缓存大小的 IO-Decoupled-Tag Cache 的性能比较, (b) 不同缓存大小的 OO-Decoupled-Tag Cache 的性能比较	66
4.9 通用计算应用上不同物理缓存大小的 Decoupled-Tag Cache 的性能比较	68
4.10 不同等指令缓冲区大小的基于重命名的标记解耦合的一级缓存的性能比较	69
5.1 图元引擎架构	72
5.2 不同二级缓存大小的性能比较	72
5.3 二级缓存行访问频率的分布	74
5.4 流量感知的可重配置热点缓存架构	74
5.5 流量感知的可重配置热点缓存的执行流程	77
5.6 L1.5 Cache 和流量感知的可重配置热点缓存相对于传统 GPGPU 架构的加速比	79
5.7 流量感知的可重配置热点缓存架构和传统 GPGPU 架构的缓存命中率的比较	80
5.8 流量感知的可重配置热点缓存架构和传统 GPGPU 架构的平均访存延迟的比较	80
5.9 不同一级缓存大小的流量感知的可重配置热点缓存架构的性能比较 ..	81
5.10 访存友好的通用图形处理器架构和传统 GPGPU 架构的性能比较	82

表格列表

2.1 NVIDIA 不同图形处理器的配置	19
2.2 不同图形处理器访存优化策略的对比	25
3.1 评估图形处理器的基准测试应用	31
4.1 评估图形处理器的基准测试应用	52
5.1 评估图形处理器的基准测试应用	73

第1章 引言

1.1 通用图形处理器概述

图形处理器 (Graphics Processing Units, 简称 GPU) 最初是用来加速图形处理的处理器, 后来研究者发现了其图形渲染加速部分高并发和高吞吐量的计算特性, 于是对 GPU 加以完善形成通用图形处理器 (General-Purpose Graphics Processing Unit, 简称 GPGPU), 用来完成一些高并发多线程场景的通用计算任务并取得了巨大成功。现在 GPGPU 已经成为了通用计算领域中的重要算力, 被广泛用于科学计算、图计算和深度学习等领域。

1.1.1 通用图形处理器架构

图1.1展示了当前通用图形处理器普遍采用的硬件结构, 该结构基于 NVIDIA 和 AMD 向公众披露的通用图形处理器架构设计^[1-6]。本文采用 NVIDIA 的术语来描述 GPU 硬件, 但图1.1所展示的结构广泛适用于其他 GPU 架构, 如 AMD 的 GPU 架构^[5,6]。

图形处理器通过并发地执行数以千计的线程来实现极高的吞吐率。图形处理器所执行的通用计算任务被称作内核, 每个内核都由称为协作线程阵列 (Cooperative Thread Array, CTA) 的线程组组成, 即线程块或工作组。在每个协作线程阵列中, 被称为 warp 的线程子块以锁步的方式执行。通常每 32 个线程被打包成一个 warp, warp 以单指令多线程 (Single-instruction Multi-Thread, SIMT) 的模式执行相同的指令流。

图形处理器包含多个图形处理簇 (Graphics Processing Cluster, GPC), 每个图形处理簇理论上都可以作为一个独立的 GPU 工作。每个图形处理簇包含几何引擎 (Geometry Engine, GE)、图元引擎 (Primitive Engine, PE)、输出混合器 (Output Merger, OM) 和数个流多处理器 (Streaming Multiprocessor, SM)。几何引擎负责管理顶点、曲面细分和几何着色任务, 以及将图元 (三角形、线和点) 坐标转换到屏幕坐标系。图元引擎的功能包括光栅化和像素着色任务的管理。输出混合器对像素着色后的像素进行深度测试和颜色混合等操作。流多处理器本质上是单指令多线程处理器核, 可以执行图形渲染的可编程流水线的着色器, 也可以执行通用计算的协作线程阵列。每个协作线程阵列只能由一个流多处理器完成, 每个流多处理器内部有一个计算引擎 (Compute Engine, CE), 计算引擎

管理协作线程阵列的调度、发射和提交。每个流多处理器可以并发地执行数十个 warp，使数以千计的线程同时执行。图形处理器的所有流多处理器共享二级缓存，二级缓存有多个分区，每个分区与一个内存控制器相连。图形处理器通过片上互连网络将流多处理器和二级缓存相连。每个流多处理器包含一个私有的一级缓存和便笺式存储（Scratchpad Memory），两者的大小可以由软件配置^[1-4]。

图1.1还展示了图形处理器的着色器间数据传输的机制，称为着色器输入输出（Shader Input Output, Shader IO）。Shader IO 利用片上的着色器输入输出缓冲区（Shader Input Output Buffer, Shader IOB）来为着色器间进行数据传输，顶点着色器将结果写入到顶点几何着色输入输出缓冲区（Vertex Geometry Shader IOB, VG Shader IOB）中，然后几何引擎读取后进行处理再发起几何着色任务。几何着色任务最终将处理结果通过片上互连网络发送到不同 GPC 的几何像素输入输出缓冲区（Geometry Pixel Shader IOB, GP Shader IOB）中，图元引擎读取并进行光栅化，随后发起像素着色任务。

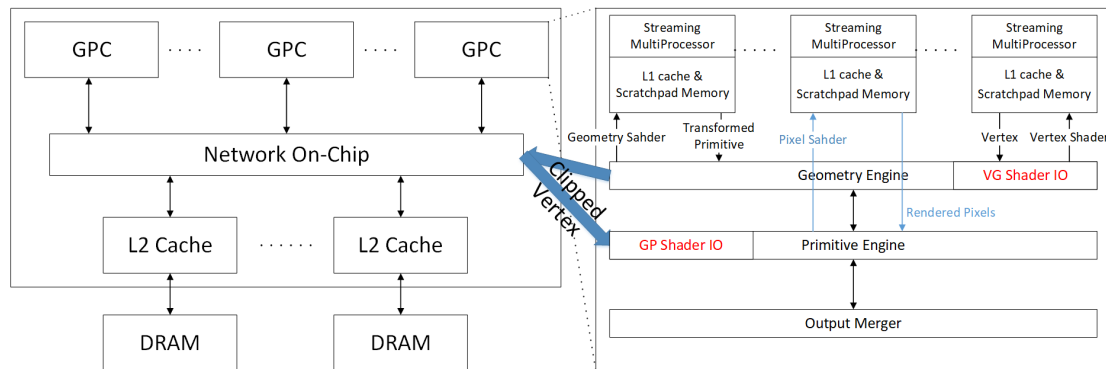


图 1.1 通用图形处理器架构

Figure 1.1 GPGPU architecture

1.1.2 通用图形处理器内存层次

图形处理器的内存层次结构包括寄存器文件、一级便笺式存储和缓存、二级缓存以及片外内存。一级缓存包括数据、指令和常量缓存^[1-4]，是每个流多处理器的私有缓存，由同一流多处理器上的 warp 共享；便笺式存储是程序员可见的，用于协作线程阵列的内部通信。二级缓存由整个图形处理器的所有 warp 共享，因此图形处理器可以在二级缓存中处理缓存一致性。二级缓存通过片上互连网络与流多处理器相连，被划分为多个分区，每个分区与一个内存控制器相连。内存控制器将在二级缓存中未命中的内存请求发送到内存。在一级和二级缓存中，未命中的请求被记录在缺失状态保持寄存器（Missing Status Holding Register,

MSHR) 中, 然后被发送到内存层次结构的下一级。原子操作由每个二级缓存分区中的原子操作单元执行。

图形处理器的缓存设计在访存合并、写策略、缓存一致性和包含属性等方面都旨在最大化吞吐量。通过将同一 warp 同时发出的请求合并, 图形处理器可以节省内存带宽。在理想情况下, 一个 warp 的所有请求在发送到一级缓存之前都会被硬件合并单元合并为一个请求。如果程序员对程序中的内存访问模式设计非常严谨, 数据缓存所提供的空间局部性优势在很大程度上可以被合并单元所捕获。图形处理器的一级缓存通常采用写穿策略, 可采用^[5,6]或不采用^[1-4]写分配策略。与写回策略相比, 写穿策略可以节省带宽^[7,8], 因为图形处理器应用程序很少重复使用已写入的数据。二级缓存采用写分配和写回策略, 这与传统 CPU 最后一级缓存的设计选择相同。现代图形处理器通常不提供对一级缓存一致性的硬件支持, 以避免一致性信息增加片上互连网络流量和内存访问的延迟^[7,8]。此外, 目前的图形处理器的二级缓存不强制包含。例如, NVIDIA 的图形处理器缓存设计是非包容性非排他性缓存^[1-4], 这意味着被带入一级缓存的缓存行也会被带入二级缓存, 但是当替换发生时, 二级缓存行会被无声地换出, 无需通知一级缓存。这种设计减少了冗余数据副本的数量, 但仍允许一级缓存在局部提供共享的输入数据。根据实验, 硬件预取和受害者缓存 (Victim Cache) 对图形处理器的访存性能的提升有限^[9], 因此图形处理器没有采用硬件预取和受害者缓存设计。

1.1.3 传统通用图形处理器架构面临的挑战

传统图形处理器被广泛应用于图形渲染^[10]、深度学习^[11-13]和图计算^[14]等领域。然而, 这些领域的应用对高吞吐量的访存需求极大, 这也使得现有的通用图形处理器架构在访存方面面临着巨大挑战。首先, 通用图形处理器的缓存容量远小于图形渲染和通用计算的工作负载, 导致严重的缓存争用和缓存抖动问题。一级缓存需要同时为数千个线程服务, 而平均每个线程拥有的缓存空间只有数十字节, 有用的缓存行经常会在被再次访问前被替换出一级缓存^[15-17]。其次, 图形渲染和通用计算中存在一些不规则的细粒度访存, 例如光线跟踪和图遍历, 这导致了缓存的命中率低^[18,19], 缓存行利用效率低^[19,20]和访存分歧^[19,21]等问题。最后, 即使图形处理器的访存系统设计可以提供足够的带宽, 但由于平均访存延迟的增加, 流多处理器可能无法掩盖如此长的访存延迟, 从而导致吞吐量下降, 进而导致性能下降^[21-23]。

1.2 通用图形处理器访存优化研究现状

针对通用图形处理器在访存方面的挑战，研究者致力于设计访存友好的通用图形处理器架构，以应对这些挑战并为应用带来显著的性能和能效提升。通用图形处理器的访存性能主要取决于访存带宽和带宽利用率，而带宽利用率则取决于平均访存延迟和平均并发访存请求数。访存带宽决定了通用图形处理器访存性能的上限。一方面，通过提高访存带宽，可以增加数据的传输速率，从而加快访存操作的执行速度。另一方面，降低访存延迟可以减少程序等待数据的时间，从而提高通用图形处理器的执行效率。增加平均并发访存请求数可以有效掩盖访存延迟，使处理器能够更充分地利用带宽，从而提高带宽的利用率。因此，针对这些挑战，设计访存友好的通用图形处理器架构需要综合考虑提高访存带宽、降低访存延迟和增加并发访存请求数等方面的策略。

通用图形处理器的访存系统设计旨在实现高带宽多线程感知的内存层次架构。这种架构以吞吐量为导向，能够同时运行数千甚至数万个线程。由于片外带宽有限且流多处理器的计算性能与内存性能之间的差距不断增大，内存带宽成为现代通用图形处理器的关键性能瓶颈。即使具有高带宽内存，也需要具备高吞吐量的内存层次结构，以便以与内存带宽相匹配的速度为流多处理器提供数据，否则昂贵的高带宽内存将变得无用。因此，设计一个高带宽的、支持多线程的内存层次结构至关重要，以满足运行线程对内存带宽的需求。在通用图形处理器的内存层次架构中，涉及到调度策略、缓存、片上互连网络和内存控制器等方面的设计和研究。所有的访存优化策略都着重于两个方面：一是增大线程级并行，增强图形处理器掩盖访存延迟的能力；二是降低资源争用，避免过多的并发请求造成拥塞，导致访存资源（缓存、带宽等）的浪费。图1.2展示了通用图形处理器的高带宽多线程感知的内存层次架构设计和研究现状，突出了调度策略、缓存、片上互连网络和内存控制器等关键部分。这些方面的研究和优化将有助于提升通用图形处理器的内存性能，从而实现更高的吞吐量和效率。

1.2.1 调度策略

图形处理器的流多处理器可以同时执行数十个 warp，因此不同的调度策略决定了访存系统所收到的访存请求的序列。合适的访存请求序列可以增加平均并发访存请求数，同时减轻缓存争用和缓存抖动问题。常见的轮询调度策略会导致所有 warp 在相同时间到达相同的长延迟的访存指令，从而减少可执行的 warp 数量，阻碍了图形处理器掩盖长延迟的能力^[24,25]。然而，尽可能多地执行 warp

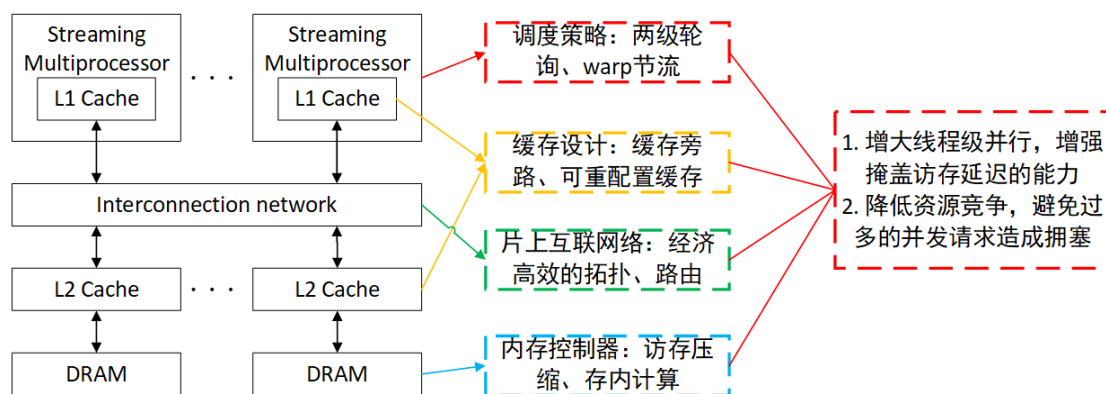


图 1.2 通用图形处理器的内存层次架构及研究现状

Figure 1.2 Memory hierarchy architecture and research status of GPGPUs

虽然可以提高掩盖延迟的能力，但会导致大量的缺失的访存请求，引起缓存、片上互连网络和内存的争用，导致访存延迟增加和性能下降^[26]。因此，研究者试图设计和优化调度策略，使图形处理器在增加并发访存请求数和减轻缓存争用之间找到最优的平衡，致力于在减少资源争用的同时，最大化图形处理器的平均并发访存请求数。

1.2.2 缓存设计

尽管图形处理器采用了多级缓存，并且在访存合并、写策略、缓存一致性和包含属性方面都已经为最大化吞吐量而设计，但面对图形渲染和通用计算应用频繁且高吞吐量的访存请求，图形处理器的缓存仍然面临严重的缓存争用和缓存抖动问题^[27,28]。于是，研究者优化了缓存管理策略，通过对局部性低的访存请求采用绕过一级缓存的缓存旁路策略，来减少一级缓存所处理的工作集^[15]。然而，如何判断访存请求的局部性是缓存旁路策略的关键。一部分方法通过硬件监测访存请求的行为（如命中率、重用距离等）来预测缓存行的局部性^[27,28]。然而，预测错误不仅不会减轻缓存争用，反而会由于丧失了对局部性的利用而导致访存性能的降低。另一部分方法则采用软件方式，由程序员来声明内存访问的局部性。然而，这种方法会增加程序员的负担^[15]。

此外，研究者试图采用可配置缓存的方式，通过动态地配置流处理器内部的片上存储资源，来达到对片上存储资源的最优利用，实现更高效的缓存设计，减轻缓存抖动和缓存争用问题^[1-4,29]。NVIDIA 的图形处理器实现了可配置的便笺式存储和数据缓存，它们物理上共享同一片上存储空间，其容量的分配由软件决定^[1-4]。此外，一些研究者也发现图形处理器的调度策略可能导致寄存器的存储空间没有被完全利用，通过将该闲置的存储空间配置为受害者缓存，可以提高图

形处理器的访存性能^[29]。

1.2.3 片上互连网络

现代图形处理器可以同时运行数万个线程。为了迅速为这大量的线程提供所需数据，通用图形处理器都配备了高带宽的内存（例如 GDDR 或 HBM）^[30,31]。为了以与内存带宽相匹配的速度为计算单元提供数据，还需要高吞吐量的片上互连网络，否则，昂贵的高带宽内存将无法发挥作用。常用的片上互连网络有网格、蝶网和交叉开关等，然而为了达到高吞吐量，片上互连网络在面积和功耗方面开销巨大。研究者需要设计高效的片上互连网络，在提供足够的吞吐量的情况下，尽可能地降低片上互连网络的开销^[32,33]。

1.2.4 内存控制器

现代图形处理器的主要性能瓶颈来自于内存带宽，例如 NVIDIA GA102 可以达到 19.5 TFLOPS 峰值单精度浮点性能，但是却只有着 769GB/s 访存带宽^[4]，如果应用是访存密集型的，那么其性能会受限于内存带宽。对图形处理器的内存控制器的研究，主要在于对内存带宽的节省。图形处理器常用的节省内存带宽的技术有访存粒度的压缩，访存数据的压缩和存内计算等。

访存粒度的压缩 虽然访存规则的图形处理器应用受益于粗粒度的内存访问，通过利用空间局部性可以达到峰值内存带宽，但是对于访存不规则的图形处理器应用，细粒度内存访问通过避免不必要的数据传输可以实现对片外内存带宽的高效利用和节能。为了同时实现对访存规则和不规则应用的最佳效果，研究者对图形处理器采用了一种适应性调整运行内核内存访问粒度的局部性感知内存层次结构^[34,35]。对具有高空间局部性的访存规则的内核采用粗粒度访问，同时对访存不规则的内核采用细粒度访问，以减少内存的无效读取。然而，细粒度的内存访问虽然可以提高带宽的利用率，但是可能降低缓存行的空间局部性。

访存数据的压缩 在图形渲染应用中，图形处理器可以通过对帧缓冲区数据和纹理数据进行压缩，来减小内存带宽压力^[36,37]。这是因为图形渲染应用的显示可以容忍一定的误差，对于用户来说无法发现区别，并且性能可以得到显著提升。此外，研究者发现对于访存密集型应用，由于访存系统的供数能力不足，导致运算单元处于利用率低的状态，因此可以投机地利用这些运算单元来进行数据压缩从而降低访存系统的带宽压力^[38]。然而，对于大多数通用计算，尤其是科学

计算应用来说,误差是难以容忍的错误。而且,通用计算应用的数据通常具有无规律性,很难被高效地无损压缩,因此访存数据的压缩本身具有一定的局限性,只能适用于部分应用^[22,39]。

存内计算 存内计算采用了三维堆叠内存技术以高带宽紧密连接逻辑层和DRAM层,来缓解内存带宽瓶颈。在该体系结构中,通过内存逻辑层来处理访存密集型应用的计算部分,来实现存内计算。虽然存内计算架构有望缓解内存带宽瓶颈,但是存内计算需要相应的编程模型和软件支持,以便有效利用存储器中的计算资源,这需要新的编程范式或工具,使开发人员能够充分利用存内计算的优势^[40,41]。如何在不增加开发人员负担的情况下透明地启用计算卸载和数据映射是一个具有挑战性的问题。而且,现有的存内计算只支持加法和乘法操作,无法支持基于比较操作的算法,能够高效支持的应用有限^[42]。

1.3 主要研究内容及贡献

1.3.1 主要研究路线

正如图1.3所示,本文的研究目的是设计访存友好的通用图形处理器架构,包含了图形渲染和通用计算访存优化架构的研究。研究路线主要包含三个步骤: 1) 首先从图形处理器的图形渲染架构和通用计算架构设计入手,主要研究如何设计高效的通用图形处理器架构,将图形渲染流水线和通用计算的访存系统相统一,以实现高效的硬件资源利用率,在相同硬件开销下实现更高的访存性能。2) 在步骤1的基础上研究适用于图形处理器应用的高吞吐量的缓存架构,研究如何消除传统一级缓存架构对最大并发访存请求数的限制,以达到高吞吐量来匹配图形处理器高性能计算对内存数据的需求。3) 在步骤2的基础上,本文发现高吞吐量的访存会对二级缓存造成严重的缓存争用,而且图形处理器的固定功能流水线的片上存储资源在执行通用计算期间处于闲置的状态,因此本文研究如何设计可配置的缓存架构,利用相同的片上存储资源在执行图形渲染和通用计算时分别完成不同的功能,以减轻二级缓存争用问题,并实现对两者来说都高效的架构。

1.3.2 主要贡献

本文中的主要研究内容和主要贡献为以下三个方面:

- 统一缓存架构的通用图形处理器设计 (Unified-Cache GPGPU): 1) 本文对图形渲染流水线和通用计算的访存系统进行了分析。图形渲染流水线各个着

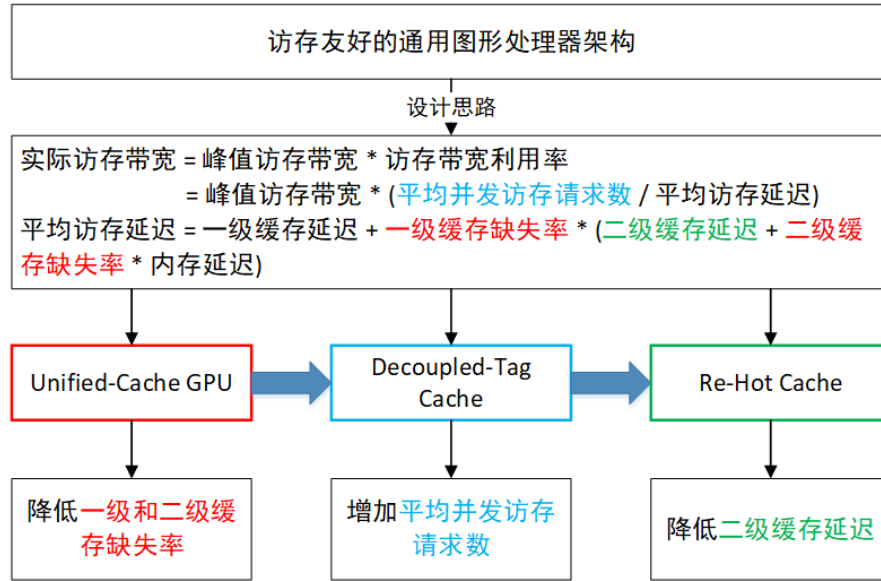


图 1.3 访存友好的通用图形处理器架构研究路线

Figure 1.3 Research path of memory-access-friendly general-purpose graphics processor architecture

色器之间的数据传输使用内存和片上存储作为输入输出缓冲区，其片上存储开销约占通用图形处理器内部总缓存容量的 20%。然而，该专用的片上存储空间只被用于图形渲染，无法被通用计算的执行所利用。2) 图形处理器内部的便笺式存储和缓存可以同时被图形渲染和通用计算所利用。而上述输入输出缓冲区的片上存储空间只被用于图形渲染。因此，为了提高硬件资源利用率，本文将该缓冲区与便笺式存储和缓存相统一，提出了一种软硬件协同的利用便笺式存储和缓存来进行着色器数据传输的图形渲染方案，并设计了相应的统一缓存架构的通用图形处理器。3) 本文提出了利用便笺式存储和缓存进行着色器数据传输的图形渲染流水线，以此统一了图形渲染的 Shader IOB 和通用计算的便笺式存储和缓存，因此在片上存储总开销不变的情况下获得了更大的缓存容量，有效减轻了缓存争用和缓存抖动对性能的影响。实验显示，Unified-Cache GPGPU 在缓存高效型应用上性能平均提高了 35.4%。

- 基于重命名的标记解耦合一级缓存 (Decoupled-Tag Cache)：1) 本文首先在传统图形处理器的一级缓存上对图形渲染和通用计算应用的访存行为特征进行了定性和定量分析。分析结果表明传统一级缓存的组相联，等待指令缓冲区和缺失状态保持寄存器会限制最大并发访存请求数。这说明传统的以降低延迟为目标设计的缓存架构，不适用于以增大吞吐量目标的图形处理器的访存系统。而且，图形应用和通用计算应用常常会访问流数据，导致一级缓存的命中

率较低。2) 为了消除传统一级缓存的组相联、缺失状态保持寄存器和等待缓冲区对最大并发访存请求数的限制,并且利用图形处理器应用的流数据访问模式,本文提出了基于重命名的标记解耦合一级缓存。该架构通过采用重命名的方法将标记和缓存行的映射解耦合,消除了组相联所导致的访存阻塞。然后,采用了双端口的 SRAM 设计了大容量的等待指令缓冲区,消除了等待指令缓冲区对最大并发访存请求数的限制。最后,没有使用缺失状态保持寄存器,而是通过访存命中的行为来合并缺失请求,消除了缺失状态保持寄存器对最大并发请求数的限制。3) 本文分别设计了顺序和乱序提交访存指令的 Decoupled-Tag Cache。图形渲染应用在两者上有着接近的性能,但在顺序的 Decoupled-Tag Cache 上有更低的硬件开销。通用计算应用在乱序的 Decoupled-Tag Cache 上有着更高的性能。4) 为了进一步提升性能,本文为 Decoupled-Tag Cache 设计了缓存旁路机制,通过识别并绕过流数据对缓存的访问,来避免低时间局部性的流数据对一级缓存的污染,从而减轻无用缓存行所引起的缓存争用问题。5) 与传统的缓存相比,在图形渲染应用上,顺序的 Decoupled-Tag Cache 可以提高超过 50% 的性能,在所有应用上,顺序的 Decoupled-Tag Cache 平均提高了 22.0% 的性能,而乱序的 Decoupled-Tag Cache 平均提高了 30.0% 的性能。

• 流量感知的可重配置热点缓存架构 (Re-Hot Cache): 1) 本文发现图形处理器的固定功能流水线的像素着色重排序缓冲区的大小会随着流多处理器的数量增加而增大,该片上存储空间容量可观,却在图形处理器执行通用计算应用时被闲置。而且由于二级缓存无法容纳应用的整个工作集,导致片外内存带宽成为性能的主要瓶颈。2) Re-Hot Cache 利用该固定功能流水线的重排序缓冲区的片上存储,在执行图形渲染应用时配置为像素着色重排序缓冲区,在执行通用计算应用时重配置为一个二级缓存的热点缓存。该热点缓存位于流多处理器的一级缓存和共享二级缓存之间,通过只存储在二级缓存中被频繁访问的热点缓存行,减小了热点缓存行的访问延迟和二级缓存的工作负载,降低了片外内存带宽的压力。而且,为了提高效率,二级缓存通过监测流量来调整热点缓存行的数量,只将适量的数据存储在热点缓存中,以避免热点缓存发生缓存争用问题。3) 与基准图形处理器相比,Re-Hot Cache 以少量的硬件开销平均提高了二级缓存高效型应用 8.61% 的性能。

1.4 本文的内容组织

本文一共六个章节，组织结构如下：在第二章中介绍了图形渲染应用和通用计算应用的相关内容，以及通用图形处理器架构的演进，同时探讨了与图形处理器的访存优化相关的研究背景和相关工作。第三章介绍了统一缓存架构的通用图形处理器（Unified-Cache GPGPU）设计的相关研究，包括图形渲染流水线的上 Shader IO 的开销分析、设计理念、结构设计和性能评估。第四章介绍了基于重命名的标记解耦合一级缓存（Decoupled-Tag Cache）设计的相关研究，包括图形渲染和通用计算应用的访存瓶颈分析、设计理念、架构设计和性能评估。第五章介绍了流量感知的可重配置热点缓存（Re-Hot Cache）设计的相关研究，包括二级缓存争用问题和可重配置的片上存储资源的分析、设计理念、架构设计和性能评估。第六章总结了本文访存友好的通用图形处理器架构设计的研究工作，并提出了未来可能的研究方向和技术突破点。

第2章 通用图形处理器访存优化的研究背景与现状

本章节主要介绍图形渲染应用和通用计算应用、通用图形处理器架构的演进以及通用图形处理器的访存系统的研究背景和现状。

2.1 图形渲染应用

2.1.1 图形渲染流水线

图形处理器的图形渲染流水线用于处理图形数据并将其转换成最终在屏幕上显示的图像。图2.1展示了 DirectX 12 中所规范的图形渲染流水线^[43]，其中红色的部分是可编程流水线，也被称为着色器，包括顶点着色器，外壳着色器，域着色器，几何着色器和像素着色器，由图形处理器的流多处理器执行；黑色的部分是固定功能流水线，包括输入装配，曲面细分，光栅化和输出混合，由图形处理器的专用的硬件部分完成。

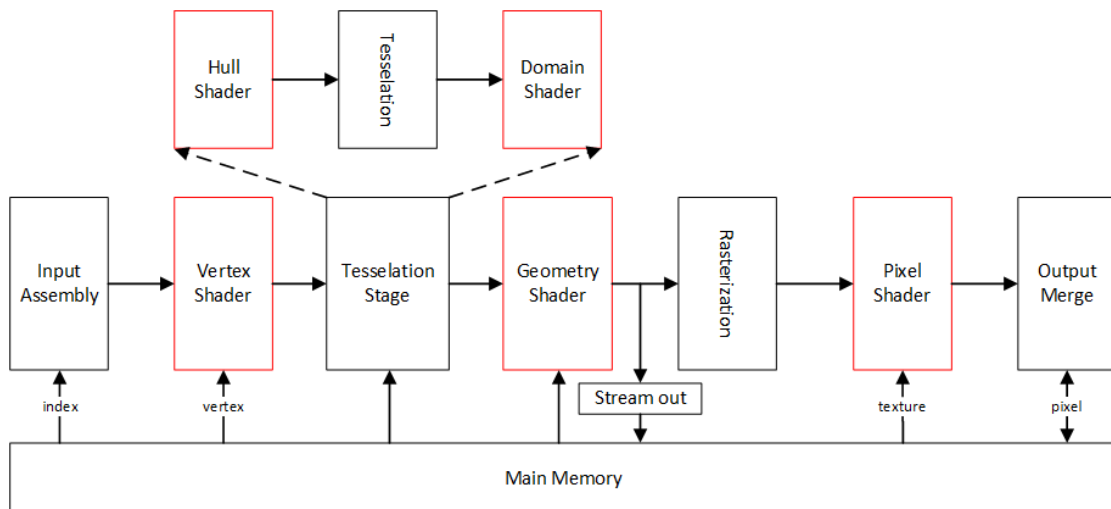


图 2.1 图形渲染流水线

Figure 2.1 Graphics rendering pipeline

1) **输入装配器 (Input Assembly, IA)**: 图形处理器从内存中读取顶点索引, 并依据指定的图元拓扑将他们装配为图元 (三角形, 线和点等)。2) **顶点着色器 (Vertex Shader, VS)**: 可以把顶点着色器看作是一种输入与输出均为单个顶点的函数, 顶点着色器从内存中读取顶点坐标和属性数据, 然而每个顶点都须经过顶点着色器的处理后再送往后续的阶段, 一般而言, 顶点着色器会将顶点坐标从世界坐标系变换为裁剪坐标系。3) **外壳着色器 (Hull Shader, HS)**, 曲面细

分 (Tessellation) 和域着色器 (Domain Shader, DS): 曲面细分功能由这三者联合实现, 是一个可选的图形渲染阶段。曲面细分利用镶嵌化处理技术对网格中的三角形进行细分, 以此来增加物体表面上的三角形数量, 再将这些新增的三角形偏移到合适的位置, 使得细分产生的图元的网格可以表现出更加细腻的细节。

4) 几何着色器 (Geometry Shader, GS): 几何着色器是一个可选的渲染阶段, 几何着色器的输入是完整的图元, 可以创建或销毁图元, 比如可以将输入的图元拓展为一个或多个其他图元, 或者依据某些条件不输出任何图元。

5) 光栅化 (Rasterization): 光栅化的作用是将图元转换为其所覆盖的屏幕像素。光栅化会首先会裁剪图元在可视范围外的部分, 然后将图元坐标从裁剪坐标系转换到屏幕坐标系, 接着剔除不可见的图元, 然后根据顶点的属性计算图元的插值系数。最后, 光栅化会遍历图元所覆盖的 2×2 的像素块, 称为 Quad, 并计算每个像素的重心坐标和深度。插值系数、重心坐标和深度等数据会作为像素着色器的输入进行后续处理。

6) 像素着色器 (Pixel Shader, PS): 针对每个像素进行处理, 可以对每个像素进行颜色计算和纹理映射等操作。纹理映射从内存的纹理图中读取每个像素所采样的纹素, 然后对采样的纹素进行滤波, 从而计算出每个像素的纹理。

7) 输出混合 (Output Merge, OM): 在此阶段中, 未通过深度缓冲区测试或者模板缓冲区测试的像素会被丢弃, 通过测试的像素会被更新进内存的深度缓冲区和颜色缓冲区中。混合操作也是在此阶段实现的, 此技术可以让当前处理的像素与颜色缓冲区对应像素相融合, 而不仅是对后者进行完全的覆写。

2.1.2 图形渲染应用的数据存储格式

从图形渲染流水线可以看出, 图形应用的访存开销主要是顶点、纹理图和帧缓冲区。

2.1.2.1 顶点数据的存储格式

图2.2a以左侧图形为例, 展示了顶点数据在内存中的存储格式。左侧的图形包含 3 个三角形图元, 分别是由顶点 $(0, 1, 2)$, $(1, 3, 2)$ 和 $(2, 3, 4)$ 组成。在内存中, 三角形的顶点的索引按序存储在索引数组 (Index Array) 中, 而 5 个顶点的数据存储在顶点属性数组 (Vertex Property Array) 中。流多处理器在访问图元的顶点数据时, 通过将顶点属性数组的基地址加上索引偏移的方式计算各个顶点在内存中的地址, 然后读取顶点属性。如果没有使用索引数组, 顶点着色器的每个线程默认使用递增的索引访问不同的顶点数据。由于相邻的顶点通常会被相邻的线程所访问, 导致顶点数据的访问几乎没有时间局部性, 而且空

间局部性基本上会被访存合并所捕获。

2.1.2.2 纹理图的存储格式

图2.2b展示了一个 8×8 的纹理的存储格式，纹理在内存中通常以 Mipmap 的方式进行存储^[44]。Mipmap 是一种用于提高纹理映射性能和图像质量的技术，它通过预先生成并存储一系列不同分辨率的纹理图像，从原始纹理图像生成一系列缩小版本，这些缩小版本的长宽都是原始纹理的二分之一、四分之一、八分之一等。

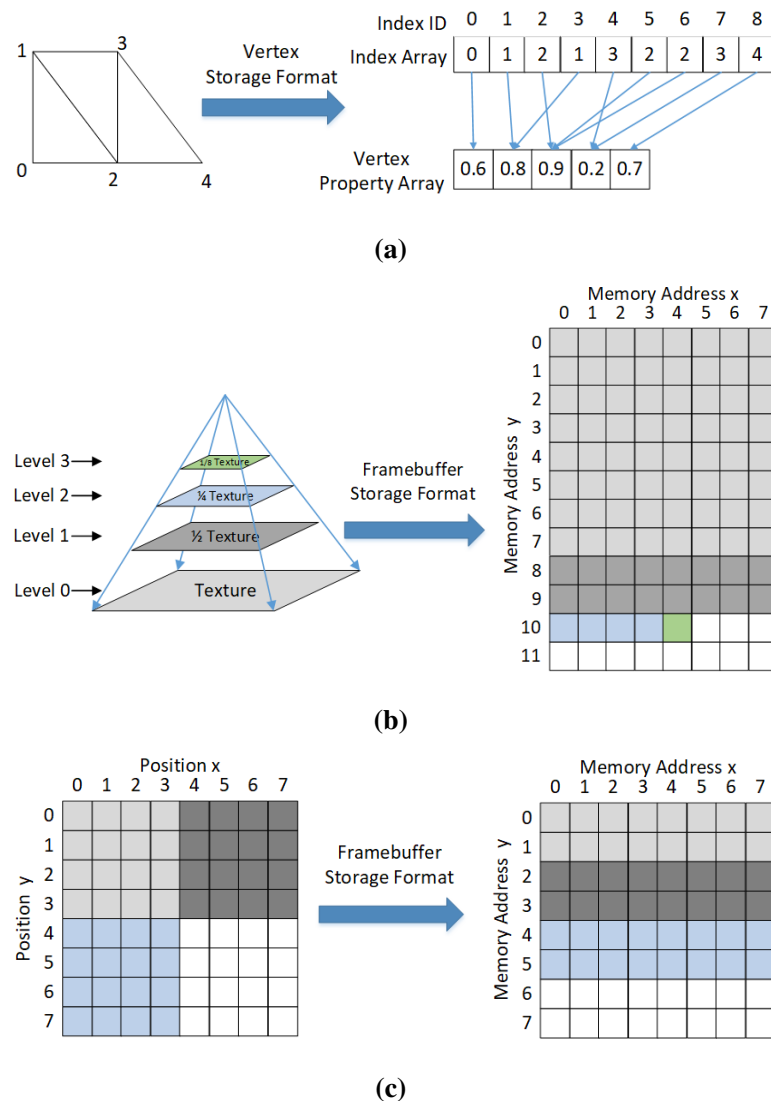


图 2.2 图形渲染应用的数据存储格式 (a) 顶点数据的存储格式, (b) 纹理数据的存储格式, (c) 帧缓冲区数据的存储格式

Figure 2.2 Data storage formats in graphics rendering applications (a) Storage format of vertex data, (b) Storage format of texture data, (c) Storage format of frame buffer data

图2.2b在原本的 Level 0 的 8×8 的纹理的基础上，通过插值或者其他滤波的方式分别生成了 Level 1 的 4×4 的纹理、Level 2 的 2×2 的纹理和 Level 3 的 1×1 的纹理，各个层级的纹理在内存中以连续或者对齐的方式存储。Mipmap 的作用是减少计算量和节省内存带宽。当纹理被映射到远处的物体时，只需使用较低分辨率的 Mipmap 级别，而不需要加载完整的高分辨率纹理，从而节省图形处理器的计算量和内存带宽，提高渲染性能。而对于各个层级的纹理，可以采用压缩^[45]的格式来进行存储，从而进一步节省内存带宽。

图形处理器的纹理映射通常采用双线性插值或者三线性插值，此时为了获得一个像素的纹理需要访问 4 个或者 8 个纹素。相邻的像素经常会访问相邻的纹素，因此像素着色器的 warp 的 32 个线程所对应的像素在进行纹理映射时所访问的 128 或者 256 个纹素通常会被访存合并为 4 或者 8 个缓存行，导致纹理数据的访问具有很强的空间局部性和较低的时间局部性。大多数空间局部性会被访存合并所捕获，即使当前 warp 的纹理映射所访问的缓存行会被后续 warp 所访问，其重用距离也会很长。

2.1.2.3 帧缓冲区的存储格式

图2.2c展示了一个 8×8 的帧缓冲区在内存中的存储格式。该 8×8 的帧缓冲区被划分为 4 个 4×4 的像素块，每个像素块在内存中连续存储。除了 4×4 的划分，帧缓冲区还可以以 8×8 的像素块为基本单位进行存储，或者直接以普通的矩阵的方式进行存储。在图形渲染流水线中，图元通常以边长为 4 或 8 的块 (Tile) 为单位进行光栅化。这样，对于以 Tile 为单位进行存储的帧缓冲区，当执行输出混合操作时，Tile 所访问的帧缓冲区数据位于同一个缓存行内，从而可以充分利用帧缓冲区数据的空间局部性。此外，在一次绘制任务中重复绘制相同屏幕区域的情况很少出现，即使存在，其重复访问的时间间隔也较长，故几乎没有时间局部性。

2.1.3 图形渲染数据的存储格式总结

从顶点、纹理和帧缓冲区的存储格式可以看出，图形渲染应用的数据普遍以规则的方式存储，其访问具有较强的空间局部性和较弱的时间局部性，即相邻的线程或者像素通常会访问同一缓存行的数据，而且其数据对于每个线程和像素通常只访问一次，即使存在重复访问，重用距离也较长，缓存行很难被保留以重复利用。因此，图形渲染应用的大部分空间局部性可以被访存合并单元捕获，剩余的小部分空间局部性只需要较小的缓存即可利用，所以图形渲染应用的性能

瓶颈通常不是缓存容量，而是较少的并发访存请求数导致访存带宽利用率不足。

2.2 通用计算应用

根据通用计算应用程序的计算访存比，可以将其分为计算密集型应用和访存密集型应用。

2.2.1 计算密集型应用

典型的计算密集型应用有深度学习推理^[11-13]和密码学应用。密码学应用包括加密算法，例如哈希函数、加密解密算法等^[46]。这些应用代码中的计算指令的比例高，相对计算的吞吐量需求而言其对内存访问的吞吐量的需求不高，它们的性能瓶颈不是图形处理器的内存访问的带宽和效率，而是运算单元的数量和利用率。对于计算密集型应用而言，无法通过访存系统的优化来提升性能。本文所设计的访存友好的通用图形处理器虽然不能提升计算密集型应用的性能，但是也不会对该类应用造成性能损失。

2.2.2 访存密集型应用

图形渲染^[10]，科学计算应用和大规模数据分析应用^[14]都属于访存密集型应用，典型的科学计算应用包括天气模拟^[47]、流体动力学^[48]、分子动力学模拟^[49]等领域。这些应用需要大量和频繁的内存读取和写入。根据性能瓶颈，可以将访存密集型应用进一步分类，分为缓存高效型应用和缓存低效型应用。

缓存高效型应用 如果一个应用是访存密集型应用，且性能与缓存容量强相关，那么该应用属于缓存高效型应用。算法1展示了简单的矩阵乘法的通用计算应用。从算法可以看出，对于数组 a 的访问，存在 warp 内的局部性，同一 warp 在不同的循环内会访问相同的数组 a 的缓存行，也存在 warp 间的局部性，相同 $global_thread_id.y$ 的 warp 会访问数组 a 的同一行；对于数组 b 的访问，只存在 warp 间的局部性，每次发射的 warp 的 $global_thread_id.y$ 增加后，都会重新访问一遍整个数组 b 。由此可知，如果想要完全利用该应用的局部性，不发生有用的缓存行被提前驱逐的情况，那么一级缓存至少要能存储数组 a 的一行以及整个数组 b ，即一级缓存的容量至少为 $S * K * (N + 1)$ 字节（ S 为数组元素的大小）。对于大型矩阵来说，一级缓存和二级缓存的容量通常无法存储这些数据。如果增加缓存容量，增加捕获的局部性，就可以提高缓存高效型应用的性能。

算法 1 Matrix Multiple

```

1: procedure MATRIX_MULTIPLE( $a[0..M * K - 1]$ ,  $b[0..K * N - 1]$ ,  $c[0..M * N - 1]$ )
2:    $j \leftarrow global\_thread\_id.x$ 
3:    $i \leftarrow global\_thread\_id.y$ 
4:    $c[i * N + j] \leftarrow 0$ 
5:   for  $k \leftarrow 0$  to  $K$  do
6:      $c[i * N + j] += a[i * K + k] * b[k * N + j];$ 
7:   end for
8: end procedure

```

缓存低效型应用 如果一个应用是访存密集型应用，且性能与缓存容量无关或弱相关，那么该应用属于缓存低效型应用。算法2展示了向量加法的通用计算应用，其功能为从内存中分别读取两个数组的值并求和，然后存储在另一个数组中。图形处理器会将线程打包为 warp，warp 以锁步的方式执行，而且 warp 的访存指令在执行访存合并时，访问相同缓存行的请求会被合并为一个请求。此时对于算法2而言，局部性已经被访存合并完全捕获，缓存的容量不会影响该应用的性能，只有访存带宽是其性能瓶颈。对于缓存低效型应用来说，其局部性已经可以被现有的访存系统（包括访存合并和缓存）所利用，增大缓存容量不会提高性能。想要提高缓存低效型应用的性能，一方面是提高内存带宽，一方面是提高内存带宽的利用率。图形处理器使用 GDDR DRAM 和 HBM^[30,31] 来提高内存带宽。如果想要使内存带宽的利用率达到最大，图形处理器需要发出足够多的访存请求以占满内存端口。

算法 2 Vector Add

```

1: procedure VECTOR_ADD( $a[0..n - 1]$ ,  $b[0..n - 1]$ ,  $c[0..n - 1]$ )
2:    $i \leftarrow global\_thread\_id$ 
3:    $c[i] \leftarrow a[i] + b[i]$ 
4: end procedure

```

2.2.3 图形渲染应用和通用计算应用的比较

图形渲染应用和通用计算应用的相同点在于两者都可能是访存密集型的应用，需要高带宽和低延迟的内存访问来支持高性能运算。

图形渲染应用和通用计算应用的不同点在于如下三点：1) **数据存储格式**：

图形渲染应用的数据是以数组或者类似矩阵的形式组织，拥有规则的格式。通用计算应用的数据存储格式更加灵活，取决于特定应用的数据结构和计算需求，可以以数组或矩阵的规则的方式存储，也可以以链表或者图等不规则的方式进行存储。规则的存储方式可以很好地适应大块的缓存行，而不规则的存储格式通常导致较低的空间局部性，导致大块缓存行的利用率较低。

2) 访存指令的类型：图形渲染应用的绝大多数访存为读操作，写操作只在少数着色器和更新帧缓冲区时使用，而且几乎不会使用原子指令。通用计算的访存更加灵活，读写和原子操作的使用取决于应用，这也导致了通用计算应用会有更复杂的数据依赖性，需要对内存访问模式进行优化以处理不同的算法。

3) 访存的行为特征：图形渲染应用的访存的空间局部性较高，时间局部性较低，而且大多数的空间局部性可以被访存合并所捕获，具有时间布局性的缓存行的重用距离也通常较长。而通用计算应用的局部性更加多样化。这使得开发人员在图形渲染应用编程时，很少会关注数据布局和内存访问模式，而对于通用计算应用可能更加关注数据的局部性，尝试使数据在内存中的布局更具连续性，以最大程度地利用缓存和减少内存访问延迟。

2.3 通用图形处理器架构的演进

通用图形处理器架构的演进是一个持续不断的过程，从最初专用于图形渲染的处理器，到现在支持高性能计算的通用图形处理器。总体来说，通用图形处理器架构的演进的趋势是从专用到通用、从分离到统一。

2.3.1 固定功能图形流水线

早期的图形处理器主要用于图形渲染，采用固定功能的图形流水线来执行渲染任务。固定功能的图形流水线由一系列功能单元组成，如顶点处理单元、像素处理单元和几何处理单元等。NVIDIA 于 1999 年发布了首个硬件支持顶点变换和光照处理的图形处理器，GeForce 256，采用了固定功能的图形流水线^[50]。GeForce 256 没有使用类似于现代图形处理器的高速缓存（Cache）架构，而是主要依赖于其主存储器，即显存，来存储和管理图形数据。尽管 GeForce 256 没有高速缓存，但它使用了一些优化技术来提高性能，例如使用双通道内存架构（TwinBank）以增加内存带宽，并且采用了高速 SDRAM（Synchronous Dynamic Random Access Memory）作为显存，以降低内存访问延迟。

2.3.2 可编程着色器

由于固定功能流水线不能灵活控制图形渲染过程，因此图形处理器引入了可编程着色器的概念，允许开发人员编写自定义的着色器程序来控制渲染过程。NVIDIA 于 2001 年发布了使用可编程着色器的图形处理器 GeForce 3^[51]。GeForce 3 引入了顶点着色器和像素着色器，允许开发人员编写自定义的顶点和像素处理程序，顶点着色器用于对顶点数据进行变换、光照计算等操作，像素着色器用于对像素数据进行颜色计算、纹理采样等操作。顶点着色器和像素着色器分别由独立的硬件功能单元执行。为了提高内存访问效率，像素着色器单元使用了纹理缓存^[52]。

2.3.3 统一着色器架构

独立的着色器单元会由于负载不均衡问题而导致执行效率低，因此提出了统一着色器架构，由单个处理核心执行多种着色器任务。该架构使得图形处理器能够更高效地处理图形渲染任务，并为后续的通用计算任务奠定了基础。NVIDIA 于 2006 年推出首个采用统一着色器架构的图形处理器架构，Tesla^[53]。Tesla 架构的着色器单元（顶点着色器、像素着色器等）集成到了同一个处理单元（即流多处理器）中，实现了更高的灵活性和效率。Tesla 的内存架构也有着显著发展，Tesla 架构的每个流多处理器都有一个私有的一级指令缓存、一级常量缓存和便笺式存储，每两个流多处理器共享一个一级纹理缓存，所有流多处理器共享二级缓存。二级缓存由多个分区组成，每个分区连接一个内存控制器。流多处理器和二级缓存之间通过片上互连网络连接。Tesla 的内存架构基本奠定了当代图形处理器内存架构的基础。

2.3.4 通用计算支持

随着统一着色器架构的引入，图形处理器开始被用于执行通用计算任务，而不仅仅局限于图形渲染。图形处理器厂商开始提供支持通用计算的软件工具，如 CUDA^[54] 和 OpenCL^[55] 等，使开发人员能够利用图形处理器的并行计算能力来加速各种类型的计算任务。

2.3.4.1 硬件架构

NVIDIA 继 Tesla 架构后，又相继推出了 Fermi^[56]、Kepler^[1]、Maxwell^[57] 和 Pascal^[58] 等架构。表 2.1 展示了 NVIDIA 的不同架构的图形处理器的配置，可以看出虽然架构的演进提供了越来越强的计算能力、越来越大的缓存和越来越高

的内存带宽,但是内存带宽与峰值计算性能的比值在不断降低,说明通用图形处理器面临着越来越严重的存储墙问题。

随着架构的演进,内存架构也发生着变化。Fermi 采用了统一的一级数据缓存和便笺式存储,以及独立的一级纹理缓存^[56]。与 Fermi 架构类似,Kepler 架构也采用了统一的一级数据缓存和便笺式存储,但是却更进一步地使用了独立的只读数据缓存,该只读数据缓存除了服务于图形渲染的纹理数据以外,也可用于通用计算的只读数据^[1]。Fermi 和 Kepler 架构的一级数据缓存和便笺式存储使用统一的片上存储,两者的大小由驱动根据应用的特性进行分配,从而提高片上存储的利用率。Maxwell 和 Pascal 架构则采用了统一的一级数据缓存和一级纹理缓存,以及独立的便笺式存储^[1,57]。此时,图形渲染的纹理缓存和通用计算的数据缓存以及便笺式存储正在逐步融合。

NVIDIA 在 Pascal 架构中首次引入了 NVLink 技术^[59,60]。NVLink 是一种高速互连技术,用于连接多个图形处理器或连接图形处理器与其他系统组件(如 CPU、内存等)。NVLink 的引入使得多个图形处理器之间能够以更高的带宽和更低的延迟进行通信,从而提高了多图形处理器系统的性能和扩展性。而且,NVLink 还提供了一种高效的图形处理器内存共享方式,使得多个图形处理器之间可以直接共享内存,而不必通过主机内存进行数据传输,进一步提高了系统的性能和效率。

表 2.1 NVIDIA 不同图形处理器的配置

Architecture	Product	Year	Memory bandwidth	FP32 Peak Performance	Cache size(L1+L2)
Tesla ^[53]	GTX280	2008	141.7 GB/s	622.1 GFLOPS	256KB
Fermi ^[56]	GTX480	2010	177.4 GB/s	1.345 TFLOPS	1488KB
Kepler ^[1]	GTX780	2013	288.4 GB/s	4.156 TFLOPS	2112KB
Maxwell ^[57]	GTX980	2014	224.4 GB/s	4.981 TFLOPS	2816KB
Pascal ^[58]	GTX1080	2016	320.3 GB/s	8.873 TFLOPS	3008KB
Turing ^[3]	RTX2080	2018	448.0 GB/s	10.07 TFLOPS	7040KB

2.3.4.2 系统软件

CPU 和 GPU 之间的数据传输一直是通用计算的性能瓶颈和程序开发的难点。为此,NVIDIA 在 CUDA 4 和 CUDA 6 中分别引入了统一虚拟寻址和统一内存的概念^[54],并在 Pascal 架构上实现了页错误的处理^[58]。

统一虚拟寻址为 CPU 和 GPU 内存提供单一的虚拟内存地址空间,并使指针能够从 GPU 代码中访问它们所在的系统中的任何位置,无论是在 GPU 内存(在

同一或不同的 GPU 上)、CPU 内存还是芯片上的共享内存中。统一虚拟寻址可以实现内存零拷贝, GPU 可以直接通过 PCIe 访问 CPU 内存, 无需进行内存拷贝。虽然零拷贝可以便利程序开发, 但性能提升有限, 因为 GPU 需要通过低带宽和高延迟的 PCIe 进行内存访问。而统一内存解决了这一问题。

统一内存创建了一个由 CPU 和 GPU 共享的内存池, CPU 和 GPU 使用相同指针即可访问该共享内存池的同一数据。CUDA 系统软件会自动在 GPU 和 CPU 之间迁移和分配在统一内存中的数据, 使得统一内存存在 CPU 上运行的程序看来是 CPU 内存, 在 GPU 上运行的程序看来是 GPU 内存。但是, CUDA 6 的统一内存受到 Kepler 和 Maxwell 架构特性的限制, CPU 的所有共享内存的分配都必须在内核启动之前与 GPU 进行同步, 导致统一内存地址空间的大小受限于 GPU 物理内存的大小。而页错误的处理可以解决这一问题。

页错误的处理将虚拟地址空间大小拓展到 49 位, 足以覆盖现代 CPU 48 位的虚拟地址空间和 GPU 自身的内存。页错误处理提供了更无缝的统一内存功能: 首先, 页错误处理使得 CUDA 系统软件不需要在内核启动之前将所有共享内存的分配与 GPU 同步。如果在 GPU 上运行的内核访问不在其内存中的页面, 它将产生页错误, 从而允许页面按需自动迁移至 GPU 内存, 或者将页面映射到 GPU 地址空间以通过 PCIe 或 NVLink 进行访问。其次, 页错误处理使得统一内存可以保证全局数据一致性, CPU 和 GPU 可以访问统一内存, 而无需程序员进行同步。最后, 大虚拟地址空间和页错误处理使应用程序能够访问整个系统的虚拟内存, 即它们可以分配、访问和共享大于系统总物理容量的数据, 从而实现非常大数据集的处理。

2.3.5 高性能计算

随着计算性能的不断提升, 通用图形处理器逐渐成为高性能计算领域的重要组成部分。通用图形处理器开始被用于加速科学计算和深度学习等领域的应用, 为解决复杂的计算问题提供了强大的计算能力。NVIDIA 在 2017 年发布了首次引入 Tensor core 的 Volta 架构的图形处理器^[2]。Tensor Core 是一种专门设计用于加速深度学习工作负载的硬件单元, 能够高效地执行张量计算。NVIDIA 的 V100 的每个流多处理器包含 8 个 Tensor core, 每个 Tensor Core 可以在单个时钟周期内执行 4x4 的半精度浮点矩阵乘法和累加运算。在 Volta 架构上, 便笺式存储、一级数据缓存和一级纹理缓存也得到完全统一, 三者共享同一 128KB 的片上存储。如果便笺式存储被配置为 64KB 大小, 那么数据和纹理将会共享剩余的 64KB 的一级缓存。

2.4 通用图形处理器访存优化的研究现状

为了适应通用图形处理器应用的高吞吐量访存需求，通用图形处理器设计采用了高带宽多线程感知的内存层次结构设计。高带宽多线程感知的内存层次结构设计主要包含四个方面：调度策略，缓存，片上互连网络和内存控制器。

2.4.1 调度策略

图形处理器的流多处理器的调度策略影响了访存指令的发射的数量和顺序，发射更多的访存指令可以提高掩盖内存延迟的能力，而合适的顺序可以更好地捕获缓存局部性^[24,61-63]。Narasiman 发现在常用的轮询调度策略中，所有 warp 在相同时间内到达相同的长内存延迟指令，会减少可执行的 warp 数量，阻碍了掩盖长内存延迟的能力，并且轮询调度策略以相同步调运行所有 warp 会破坏 warp 内的数据局部性^[62]。因此，Narasiman 提出了两级轮询调度策略，warp 被分成多组，每次调度的时候首先以最老优先的方式选择一组，然后再以轮询调度的方式选择组内的一个 warp 进行调度，直到该组的所有 warp 都运行到长延迟指令后，才会选择下一组的 warp 进行调度^[62]。两级轮询调度策略使 warp 可以在不同的时间到达长延迟指令，当运行中的 warp 达到长延迟指令时，就会优先处理下一组 warp，，从而提高了掩盖访存延迟的能力。并且，同一时间只有一组的 warp 在占用一级缓存，可以更好地利用 warp 内的局部性，从而减轻缓存争用。

Jog 在两级轮询调度的基础上，提出了协作线程阵列 (Cooperative Thread Array, CTA) 感知的 warp 调度策略 (cOoperative thread array aWare warp scheduLing policy, OWL)^[24]。OWL 会根据 CTA 来将 warp 分成不同的组，以提高 CTA 内的局部性。严格的两轮询调度会优先处理一组 CTA，直到其完全停滞为止。而对于 OWL，当一组 CTA 的数据存在于一级缓存中时，会给予其更高的优先级，从而提供更多缓存行重用的机会。

两级轮询调度策略总是执行尽可能多的 warp，以期望更好的掩盖访存延迟，然而 Kayran 发现在流多处理器上执行尽可能多的 warp 并不总是导致更好的性能^[25]。这是因为同时运行较多的 warp 可能会造成大量的缺失访存请求，在缓存、片上互连网络和内存中引起争用，导致了更长的访存延迟。因此，研究者提出 warp 节流技术，其目的是根据应用程序的特性为每个流多处理器分配最佳数量的 warp，以减轻资源争用^[25,26,64,65]。

Rogers 等人^[26]注意到，由于大规模多线程和相对较小的一级缓存容量，GPGPU 应用会面临严重的缓存争用问题。因此，他们提出了缓存感知的 warp

调度 (Cache Conscious Wavefront Scheduling, CCWS)。CCWS 与之前的 warp 节流方法不同, CCWS 在细粒度 (warp 级别) 而不是粗粒度 (CTA 级别) 进行限制。CCWS 会检测由于缓存争用而失去局部性的 warp, 这些 warp 会被优先调度直到它们利用了其局部性, 而其他 warp 则被取消调度 (即不允许发出任何访存指令)。

之后, Rogers 等人^[65]提出了后续工作, 提出了分歧感知的 warp 调度 (Divergence-Aware Warp Scheduling, DAWS)。DAWS 是一种基于分歧的缓存足迹预测器, 用于计算每个 warp 在循环中所需的局部性数据量。DAWS 利用这些预测来优先考虑一组 warp, 以确保这些 warp 的缓存足迹不超过一级缓存的容量。与 CCWS 相比, DAWS 是一种主动机制, 它在失去局部性之前预防了局部性的丧失, 并根据当前 warp 的分歧状态调整限制机制。然而, warp 节流技术为了监测缓存的历史信息, 需要大量的额外硬件开销, 并且对于缓存低效型应用性能提升有限。

2.4.2 缓存设计

2.4.2.1 缓存管理策略

Li 等人发现虽然 warp 节流技术可以减轻缓存抖动和资源争用, 但是也会导致内存带宽和其他资源 (二级缓存、片上互连网络和运算单元) 未被充分利用^[27]。因此研究者在 warp 节流技术的基础上提出了缓存旁路策略^[15,27,28,66,67]。Li 提出了一种建立在 CCWS 之上的缓存旁路方案, 称为基于优先级的缓存分配 (Priority-based Cache Allocation, PCAL)。在内核开始执行时, PCAL 执行一个最佳数量的活跃 warp, 来缓解缓存争用, 然后允许其他的非活跃 warp 绕过一级缓存进行访存, 从而利用处理器上的其他资源。因此, PCAL 减少了缓存争用, 并有效利用了未使用的处理器资源。

Zheng 等人^[28]观察到图形处理器应用中存在一些流式的访存指令, 这些访存指令所取回的缓存行不会被其他访存指令所访问, 却会将其他有用的缓存行驱逐出缓存, 造成缓存污染, 因此提出了一种类似 PCAL 的方法, 称为自适应缓存和并发分配 (adaptive Cache and Concurrency Allocation, CCA)。CCA 改进了 DAWS, 除了非活跃 warp 的访存指令会绕过一级缓存以外, 活跃 warp 的流式访存指令也会绕过一级缓存。

此外, Li 等人^[15]提出了一种基于软件的 CTA 聚类技术, 将具有潜在重用的 CTA 分组放在同一流多处理器上执行, 以改进 CTA 调度策略并利用 CTA 之间的局部性。而且, 他们提出了一种基于软件的 CTA 节流和缓存旁路机制, 通

过限制流多处理器上并发 CTA 的数量,来缓解缓存争用问题。然而,缓存旁路策略可能会使图形处理器的片上互连网络、二级缓存和内存带宽过载,导致访存延迟增大并降低图形处理器的性能。

2.4.2.2 可重配置缓存

因为不同应用对图形处理器上资源(寄存器,便笺式存储和数据缓存)的需求不同,所以图形处理器的某些资源发生争用时,其他资源可能没有被完全利用。图形处理器可重配置缓存就是利用图形处理器应用对资源利用的不均衡性,通过重配置资源来实现资源的最优化利用。NVIDIA 统一了便笺式存储和数据缓存,即便笺式存储和数据缓存共享同一片上存储空间,它们之间的分配由驱动决定^[2-4]。一些研究者在 NVIDIA 的基础上更进一步设计了可重配置的统一寄存器、便笺式存储和缓存,虽然可重配置性更高,可以实现更高的资源利用率,但是需要更高的面积开销^[68,69]。

此外, warp 节流技术的使用也使图形处理器上更多资源闲置。Yunho 发现图形处理器在采用 warp 节流技术时,由于减小了运行时的 warp 数量,导致寄存器的存储空间没有被完全利用,因此提出将空闲的寄存器存储空间重利用为受害者缓存来减轻缓存争用问题^[29]。然而,由于共享同一物理存储,可重配缓存会面临访问端口冲突问题。

2.4.3 片上互连网络

图形处理器可以同时运行数万个线程。为了快速为这大量的线程提供所需数据,图形处理器配备了高带宽的 DRAM 内存(例如 GDDR 或 HBM)^[30,31]。除了高带宽的内存之外,还需要高带宽的片上互连网络来快速向流多处理器提供数据,否则昂贵的高带宽内存将无法发挥作用。构建高吞吐量的片上互连网络在面积和功耗方面可能非常昂贵。因此,许多人研究如何使片上互连网络在单位面积上拥有更高的吞吐量^[32,33,70,71]。

Bakhoda 等人提出了一种经济高效的网格式的片上互连网络,利用半路由器降低网络成本,并在性能最小损失的情况下实现高吞吐量。请求从数量较多的流多处理器到数量较少的二级缓存,最后回到流多处理器的过程,会在数量较少的二级缓存的路由器上发生阻塞,从而导致性能瓶颈。Bakhoda 通过多端口路由器扩展了网络,以解决从多到少再到多的瓶颈,改善了网络性能^[32]。

此外,互连网络的仲裁策略会影响内存控制器收到请求的序列。Yuan 等人^[72]观察到,位于流多处理器和内存控制器之间的片上互连网络可能破坏内存访问

的行缓冲局部性。因此，他们提出了一种互连网络仲裁方案，通过优先考虑访问相同内存行的内存请求，以保留行局部性并无需复杂的 FR-FCFS (First-ready, first-come-first-service)^[73] 内存控制器。他们只使用了简单的 FIFO (First In First Out) 内存控制器，就实现了与复杂的 FR-FCFS 相似的性能。

2.4.4 内存控制器

2.4.4.1 访存粒度的压缩

一方面，访存规则的通用计算应用从粗粒度内存访问中受益（即每次内存访问获取大块数据），通过利用时空局部性实现最大内存带宽和峰值性能。另一方面，访存不规则的通用计算应用从细粒度内存访问受益，通过避免在粗粒度策略下可能发生的不必要数据传输，实现片外内存带宽的高效利用和节能。为了兼顾这两种方案的优势，Rhu 等人提出了一种适应性地调整运行时内核的内存访问粒度的局部性感知内存层次结构 (Locality-Aware Memory hierARchy, LAMAR)^[35]。LAMAR 对具有访存规则的高时空局部性的通用计算应用使用粗粒度访问，同时对访存不规则的通用计算应用使用细粒度访问，以减少内存过度读取。LAMAR 采用基于 Bloom 过滤器的低成本硬件预测机制来确定最佳访问粒度。

2.4.4.2 访存数据的压缩

对于许多访存密集型的通用图形处理器应用来说，片外内存带宽是瓶颈。数据压缩技术可以缓解内存带宽瓶颈。NVIDIA 和 AMD 在内存控制器对读写的数据块进行压缩，以节省内存带宽^[3,6]。Vijaykumar 等人提出了核心辅助的瓶颈加速 (Core-Assisted Bottleneck Acceleration, CABA) 框架，该框架利用未充分利用的计算单元来执行压缩等工作，以缓解图形处理器执行中的瓶颈^[38]。例如，为了缓解内存带宽瓶颈，CABA 动态创建辅助 warp，与原始 warp 在同一流多处理器上并行执行。辅助 warp 利用空闲的计算单元，对传入的压缩缓存行进行数据解压缩，对传出的缓存行进行压缩，从而减少内存传输数据，缓解了内存带宽瓶颈问题。然而，CABA 框架会增加开发人员的负担，而且开发的应用很难拓展到不同架构的图形处理器上。

与压缩类似，研究者可以通过近似计算的方法来降低内存带宽压力。Samadi 等人^[37] 引入了一种适用于图形处理器的近似计算方法，允许根据所需的质量水平权衡性能和输出准确性。他们的其中一种优化是数据打包，即减少用于存储输入数组的位数，通过牺牲精度来减少内存带宽压力。此外，Yazdanbakhsh 等人^[74] 提出了一种近似技术，图形处理器通过近似值，放弃了一些在缓存中未命中的读

请求的数据，以减轻内存带宽压力。被选择进行近似的读请求对质量的影响最小，并通过设定一个放弃率来控制性能能耗比和质量之间的权衡。然而，对于一些通用计算，误差是难以容忍的错误，有损压缩和近似计算只适用于部分应用。而且，大多数应用缺少规律的数据，很难被高效地无损压缩，因此访存数据的压缩本身具有一定的局限性。

2.4.4.3 存内计算

三维堆叠式内存技术使用高带宽紧密连接逻辑层和 DRAM 层，来缓解内存带宽瓶颈。该架构通过将访存密集型计算部分转移到内存逻辑层执行，来实现存内计算。Hsieh 等人提出了一种基于编译器的技术，根据简单的成本效益分析，将访存密集型代码卸载到内存逻辑层执行^[40]。而且，他们引入了一种运行时预测机制，以定位将被存内计算卸载的代码访问的同一内存栈中的数据，来最小化片外内存带宽的消耗，而无需程序员显示地搬运数据。然而现有的存内计算只支持加法和乘法操作，无法支持基于比较操作的算法，能够高效支持的应用有限^[40,42]。并且现有的存内计算工艺最多只能保障 16 比特的整形乘法计算，无法为高精度计算的图形处理器应用提供保障。

表 2.2 不同图形处理器访存优化策略的对比

Optimization Strategy	Improving access parallelism	Mitigating resource contention	High hardware overhead	Low generality	Ignoring graphics rendering
Two-level schedule	✓	✗	✗	✓	✓
Warp throttle	✗	✓	✓	✗	✓
Cache bypass	✓	only cache	✓	✗	✓
Reconfigurable cache	✗	✓	✗	✗	✓
Access compression	✗	✓	✗	✓	✓
Process in memory	✗	✓	✗	✓	✓
Unified-Cache GPGPU	✗	✓	✗	✗	✗
Decoupled-Tag Cache	✓	✗	✗	✗	✗
Re-Hot Cache	✗	✓	✗	✗	✗

2.4.5 不同访存优化策略的比较

表2.2总结了当前部分通用图形处理器访存优化策略的优缺点。现如今的通用图形处理器的访存优化策略虽然可以从提高访问并行度和缓解资源争用两个方面来优化访存，但是会存在高硬件开销、低通用性以及都只关注通用计算应用却忽视了图形渲染应用的缺点。而本文所提出的 Unified-Cache GPGPU、Decoupled-Tag Cache 以及 Re-Hot Cache 不仅可以提高通用图形处理器的访存性

能，而且都有着低硬件开销和高通用性的特征，同时也实现了对图形渲染应用的优化。

2.4.6 实验平台

本文在现有的图形处理器 LGPU 基础上进行设计和实验，所有实验都在仿真环境下进行。图2.3展示了 LGPU 的架构。LGPU 实现了 OpenGL 2.0 标准，采用了统一着色器架构，顶点着色器和像素着色器在同一个流多处理器上执行。

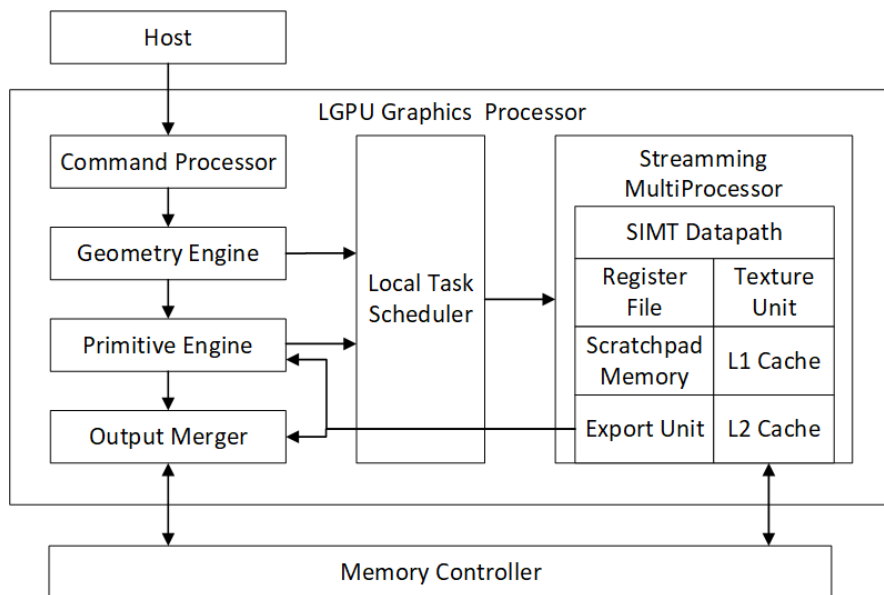


图 2.3 LGPU 图形处理器架构

Figure 2.3 LGPU graphics processor architecture

LGPU 的架构利用了图形渲染的三个层次的并行性：顶点和像素数据级的并行性、着色器指令级的并行性以及顶点和像素任务级的并行性。1) 顶点和像素数据级的并行性是指顶点之间或像素之间的计算并行。LGPU 将每 32 个顶点或像素打包为一个 warp，这些 warp 以单指令多线程的方式在流多处理器上执行。每个流多处理器最多可以同时执行 40 个 warp，从而实现顶点和像素数据级的并行计算。2) 着色器指令级的并行性是指在顶点或像素着色器内部的指令级别的并行执行。在顶点和像素着色器中，存在一些无依赖关系的指令组，例如几何变换和光照处理等操作。这些指令可以在流多处理器的功能单元中同时执行，从而提高着色器的执行效率。而且，一个顶点或像素是由许多属性组成的，如位置、颜色和纹理坐标等，着色器程序可以并行地处理这些属性。3) 顶点和像素任务级的并行性体现在顶点和像素着色的独立性。顶点和像素着色器之间相互独立，可以使用流多处理器来并行处理它们。这种并行性允许顶点和像素任务同时进

行, 进一步提高并行计算的效率。综合利用这三种并行性, LGPU 可以实现高效的图形渲染, 充分发挥图形处理器的计算能力。

第3章 统一缓存架构的通用图形处理器设计

本章节介绍了统一缓存架构的通用图形处理器（Unified-Cache GPGPU）的设计动机、设计理念、具体实现和性能评估。Unified-Cache GPGPU 的设计动机有两个：1) 缓存高效型的图形处理器应用由于缓存容量不足，会面临缓存争用问题。2) 图形处理器的固定功能流水线的 Shader IO 和通用计算的访存系统相互独立，通用计算的访存系统无法利用固定功能流水线的专用的 Shader IOB 的片上存储，导致片上存储利用率低。Unified-Cache GPGPU 的设计理念是通过统一图形渲染的 Shader IO 和访存系统，将固定功能流水线的 Shader IOB 的片上存储和图形处理器的便笺式存储、一级缓存和二级缓存相统一，以实现更高的片上存储利用率，增大图形处理器的一级和二级缓存容量。Unified-Cache GPGPU 的具体实现主要包含 Unified-Cache GPGPU 架构实现、统一便笺式存储和一级数据缓存、统一便笺式存储和 GPC 内的 Shader IOB，以及统一二级缓存和 GPC 间的 Shader IOB。为了评估和探索 Unified-Cache GPU 的架构设计，本文进行了 Unified-Cache GPGPU 的性能分析实验和设计空间探索实验。

3.1 相关背景：图形处理器的着色器输入输出缓冲区

图形渲染和通用计算应用所共享的访存系统包括寄存器、便笺式存储、一级和二级缓存、片上互连网络和内存。图形渲染应用独有的数据传输系统为着色器输入输出缓冲区（Shader IOB），该片上存储不会被流多处理器所感知，在图形处理器执行通用计算应用时被闲置。Shader IOB 的作用是暂存着色器的输出结果，并将着色器的结果按序交给下一阶段进行处理。因为流多处理器是乱序完成着色器的 warp，而图形标准要求顺序处理图元^[43,75]，所以为了保证流多处理器的输出不被阻塞以提高流多处理器的利用率，Shader IOB 的存储空间需要足够大以能够完全掩盖着色器的 warp 乱序完成所导致的开销。Shader IOB 的设计通常可以分为三类：

1) Shader IOB 完全通过内存实现：着色器将结果写入到内存中，固定功能流水线等待着着色器执行完成，然后按序从内存中读出结果，该方法会增加额外的访存开销，使原本就受限于内存带宽的性能进一步降低。

2) Shader IOB 完全通过片上存储实现：着色器将结果写入到固定功能流水线的专用的片上存储中，为了保证流多处理器的乱序执行不被阻塞，片上存储空

间的容量需要足够大以容纳所有 warp 的输出结果，该方法不会导致额外的访存开销，但是需要大量的片上存储的开销。

3) Shader IOB 通过内存和片上存储联合实现：该方法的片上存储无法容纳所有 warp 的输出结果，片上存储用来存储优先级较高的 warp 的输出结果，剩余的输出结果存储在内存中。当 Shader IOB 的片上存储部分存在空闲时，会从内存中读出优先级最高的 warp 的输出结果。理想情况下，如果流多处理器将着色器的 warp 顺序完成，并且后续的流水线没有阻塞，那么就无需向内存中写入和读取输出结果。该方法与前两个方法相比可以以较低的专用的片上存储开销减少大部分的访存开销。然而，每个流多处理器可以同时执行数十个 warp，即使通过内存和片上存储联合实现 Shader IOB，也需要约 20% 的图形处理器总缓存容量开销的专用的片上存储。而该固定功能流水线的 Shader IOB 的片上存储只为图形渲染应用所专用，不能被通用计算应用所利用，造成了片上存储利用率低的问题。

3.2 设计动机：通用图形处理器的缓存低效问题

尽管一些图形渲染和通用计算应用具有一定的访存局部性，但是由于缓存的容量不足，频繁的高吞吐量的内存访问经常导致有用的缓存行被提前换出，使得局部性无法被有效利用。从 NVIDIA 的图形处理器的架构发展可以发现，图形处理器的一级缓存和二级缓存的容量在不断增加，目前流多处理器的一级缓存容量最多可以达到 128KB^[4]。然而，相比流多处理器同时执行的数千个线程，平均每个线程依然只有数十字节的一级缓存容量，缓存容量的不足导致目前的通用图形处理器面临严重的缓存争用问题。

为了分析图形处理器由于缓存容量不足所导致的一级缓存争用问题的严重程度，本文测量了不同应用在一级缓存上的冷缺失率和容量缺失率。当访存请求在一级缓存缺失，却访问了先前访问过的缓存行时，本文将该缺失视为容量缺失。本文在实验中使用了 LGPU 作为实验平台。表3.1展示了用于实验的通用计算应用和图形渲染应用，由于驱动和编译器尚不完善，本文使用了来自 Polybench^[76] 和 Parboid^[77] 的部分通用计算应用和来自 glmark2^[78] 的部分图形渲染应用作为基准测试程序。基准测试程序的规模保证图形处理器的执行车道可以处于满负载的状态。为了使评估更加全面，对于通用计算，本文分别使用了缓存高效型和低效型应用以及计算密集型应用，对于图形渲染，本文使用了不同顶点数量、不同纹理大小和不同分辨率的应用。

图3.1展示了不同应用的一级数据缓存的冷缺失率和容量缺失率，AVG 表示所有应用的平均值。实验结果显示，平均一级缓存缺失率和平均容量缺失率分别为 54.5% 和 38.8%。容量缺失占总一级缓存缺失的 71.2%。在 15 个应用程序中，有 11 个应用程序中超过 70% 的一级缓存缺失为容量缺失。该结果表明图形处理器经常访问二级缓存和内存以再次获取先前从一级缓存中被替换的数据。这种大量的长延迟访存操作会导致访存系统的阻塞。

表 3.1 评估图形处理器的基准测试应用

Table 3.1 GPU applications for benchmark

General-Purpose Cache-Efficient Applications			
App	Description	App	Description
bicg ^[76]	BiCGStab Linear Solver	atax ^[76]	Matrix Transpose-Vector Mul.
gemv ^[76]	Vector Mul. and Matrix Add.	mvt ^[76]	Matrix Vector Product-Transpose
syrk ^[76]	Symm. rank-k operations	gesu ^[76]	Scalar, Vector and Matrix Mul.
syr2k ^[76]	Symm. rank2k operations		
General-Purpose Cache-Inefficient Applications			
App	Description	App	Description
spmv ^[77]	Sparse Matrix-Dense Vector Mul.	2mm ^[76]	2 Matrix Mul. and Matrix Add.
General-Purpose Compute-Intensive Applications			
App	Description	App	Description
conv2d ^[76]	2D Convolution		
Graphics Rendering Applications			
App	#Vertex	Texture size	Resolution
jellyfish ^[78]	13200	2*256*256	800*600
cat-tex ^[78]	43044	512*512	800*600
cube-tex ^[78]	36	512*512	800*600
2D-tex ^[78]	4	128*128	256*256
horse ^[78]	21516	0	800*600

如果图形处理器可以增大缓存容量，使部分缓存行在被换出之前重用，就可以显著提升性能。缓存容量的增加虽然不能减少冷缺失，但是可以有效减少容量缺失，从而增加缓存命中率和降低访存延迟，并减轻片上互连网络、二级缓存和片外内存的争用。然而，直接增大缓存容量会增加面积开销，问题的关键在于如何高效地增加缓存容量。

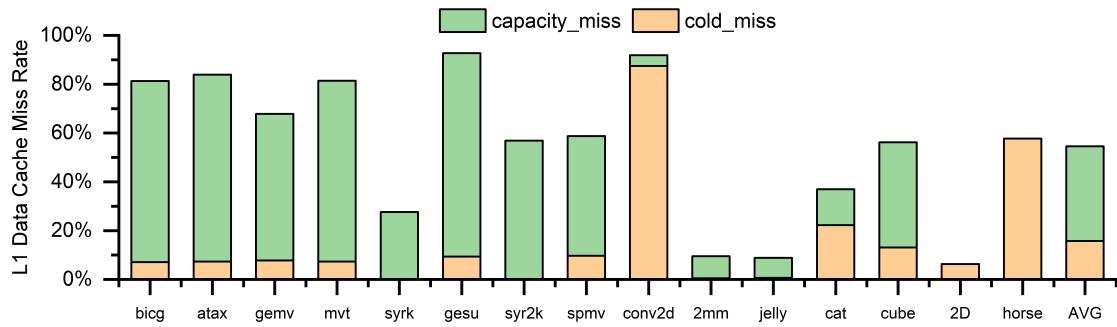


图 3.1 一级数据缓存的冷缺失率和容量缺失率

Figure 3.1 Cold miss and capacity miss rate of L1 data cache

3.3 统一缓存架构的通用图形处理器架构设计

图3.2展示了统一缓存架构的通用图形处理器架构。该通用图形处理器包含命令处理器 (Command Processor, CP)、输入装配器 (Input Assembler, IA)、几何引擎 (Geometry Engine, GE)、图元引擎 (Primitive Engine, PE)、输出混合器 (Output Merger, OM)、两个流多处理器 (Streaming Multiprocessor, SM)、二级缓存 (L2 Cache) 和片上互连网络 (Network On-Chip, NOC)。

对于图形渲染应用来说,需要固定功能流水线和流多处理器协同执行。固定功能流水线主要由命令处理器、输入装配器、几何引擎、图元引擎和输出混合器组成。流多处理器负责执行着色器程序 (包括顶点、外壳、域、几何和像素着色器),具体着色器程序由开发人员通过编写 GLSL 或者 HLSL 代码实现。固定功能流水线基于应用专用硬件的方式设计,用来加速图形渲染中固定的功能和算法,包括输入装配、裁剪、视口映射、光栅化、深度测试和颜色混合等操作。对于通用计算应用来说,协作线程阵列的调度由驱动和计算引擎 (Compute Engine, CE) 协同完成,协作线程阵列的执行由流多处理器完成。

3.3.1 设计理念: 统一缓存、便笺式存储和着色器输入输出缓冲区

图形处理器的图形固定功能流水线的专用的 Shader IOB,无法被通用计算的访存系统所利用,为了使通用计算的访存系统能利用 Shader IOB,本文设计了 Unified-Cache GPGPU,将便笺式存储、Shader IOB 和缓存相统一,来增加缓存的容量。Unified-Cache GPGPU 的实现包括三部分:

1) **一级缓存和便笺式存储的统一:** 在 Unified-Cache GPGPU 中,一级缓存和便笺式存储共享同一块物理上的片上存储,而逻辑上的空间划分由驱动负责管理。

2) **便笺式存储和 GPC 内 Shader IOB 的统一:** 对于顶点着色器和几何着色

器之间的所有 Shader IO，由于都是在同一个 GPC 中执行，称为 GPC 内的 Shader IO。Unified-Cache GPGPU 通过软硬件的协同，利用便笺式存储来实现 GPC 内的 Shader IO。

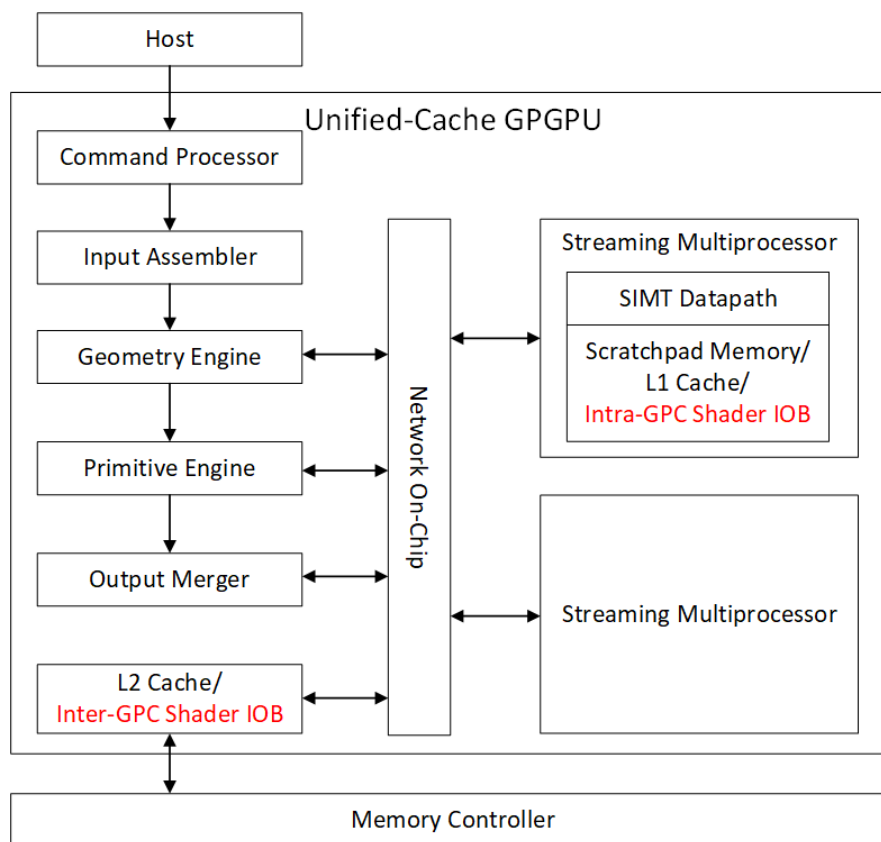


图 3.2 统一缓存架构的通用图形处理器架构

Figure 3.2 Unified-Cache GPGPU architecture

3) 二级缓存和 GPC 间 Shader IO 的统一: 经过了几何处理（包括裁剪和视口变换）的图元，需要根据图元所覆盖的屏幕区域发送到不同的 GPC 进行像素处理（包括光栅化，像素着色和输出混合）。此时图元的数据的输入输出发生在不同 GPC 之间，称为 GPC 间的 Shader IO。Unified-Cache GPGPU 通过软硬件的协同，利用二级缓存来实现 GPC 间的 Shader IO。

经过以上三个部分的实现，Unified-Cache GPGPU 统一了缓存、便笺式存储和 Shader IOB，这样在执行图形渲染时，通过软硬件的协同，可以动态地将便笺式存储和二级缓存作为 Shader IOB 使用，而未被作为 Shader IOB 使用的部分，依旧作为便笺式存储和缓存使用。在执行通用计算时，由于不会使用 Shader IOB，全部的片上存储都可以作为便笺式存储和缓存使用。

3.3.2 固定功能流水线

命令处理器 命令处理器是主机和通用图形处理器之间的接口，用于实现两者间的交互。用户程序通过命令处理器发起绘制任务，并将相关的建模数据、流水线配置和着色器代码存储在内存中，这些数据一起构成了一次完整的绘制任务。当用户程序发送绘制命令后，驱动将相应的数据准备好并存储在内存中。然后，主机将绘制命令发送到图形处理器的命令队列中，并唤醒命令处理器。命令处理器接收到命令后，开始控制图形处理器来完成该绘制命令。

输入装配器 输入装配器对应于图形渲染流水线的输入装配阶段。输入装配器从索引缓冲区中读取索引或者按顶点顺序生成索引，并将索引装配为几何图元（三角形，线和点等），最后将几何图元发送到几何引擎。

几何引擎 图形渲染流水线根据处理对象的不同可以划分为几何处理阶段和像素处理阶段，几何处理阶段的处理对象为几何图元，而像素处理阶段的处理对象为屏幕像素。几何引擎的功能对应图形渲染流水线的几何处理阶段，主要负责图元的几何处理。几何引擎的功能可以分为两部分：1) 管理顶点、外壳、域和几何着色器的调度，2) 对图元进行曲面细分、裁剪、剔除和视口映射。

图元引擎 从图元引擎开始，图形渲染流水线的处理对象转变为屏幕像素。图元引擎的功能包括两部分：1) 光栅化；2) 像素着色器的调度。光栅化会遍历图元所覆盖的屏幕像素，然后以 Quad（2*2 像素块）为基本单元，将每 8 个 Quad 打包为一个像素着色器的 warp。等待流多处理器执行完成后，将经过像素着色的屏幕像素发送给输出混合器。

输出混合器 输出混合器是图形渲染流水线的最后阶段，将结果最终更新到帧缓冲区中。输出混合器可以执行深度测试、模板测试和颜色混合操作。深度测试、模板测试和颜色混合都是从内存的帧缓冲区中读出目标像素的数据，然后和当前正在处理的像素的数据进行计算，最后将结果写回帧缓冲区的目标像素。其操作属于典型的读-改-写型原子操作。为了提高输出混合器的性能，本文使用了缓存来减少对内存的频繁的读写操作。

3.3.3 流多处理器

3.3.3.1 流多处理器架构设计

图3.3展示了 Unified-Cache GPGPU 的流多处理器（Streaming Multiprocessor, SM）的架构，流多处理器用于执行图形渲染的着色器和通用计算的协作线程阵列。流多处理器主要由以下部分组成：纹理单元（Texture Unit）、输出单元（SM/EXP Unit）、便笺式存储（Scratchpad Memory）、缓存（包括指令、常量和数据缓存）和4个流处理器簇（Streaming Processor Cluster, SPC）。每个流处理器簇具有独立的寄存器堆（Vector/Scalar General-Purpose Register, VGPR/SGPR）、向量算术单元（Vector ALU, VALU）、标量单元（SALU/SMEM）和分支单元（Branch Decode/EXEC）。流处理器簇以单指令多线程（Single Instruction Multiple Thread, SIMT）的方式执行 warp，每个流处理器簇可以同时执行多个 warp，这些 warp 会共享流处理器簇的各个执行单元。流处理器簇之间的 warp 执行相互独立，4个流处理器簇共享便笺式存储、纹理单元、输出单元和缓存。为了实现 warp 的并行执行，流处理器簇为每个 warp 分配了独立的寄存器空间和便笺式存储空间。在 Unified-Cache GPGPU 中，每个流处理器簇最多可以同时执行 16 个 warp，故每个流多处理器最多可以同时执行 64 个 warp，2048 个线程。

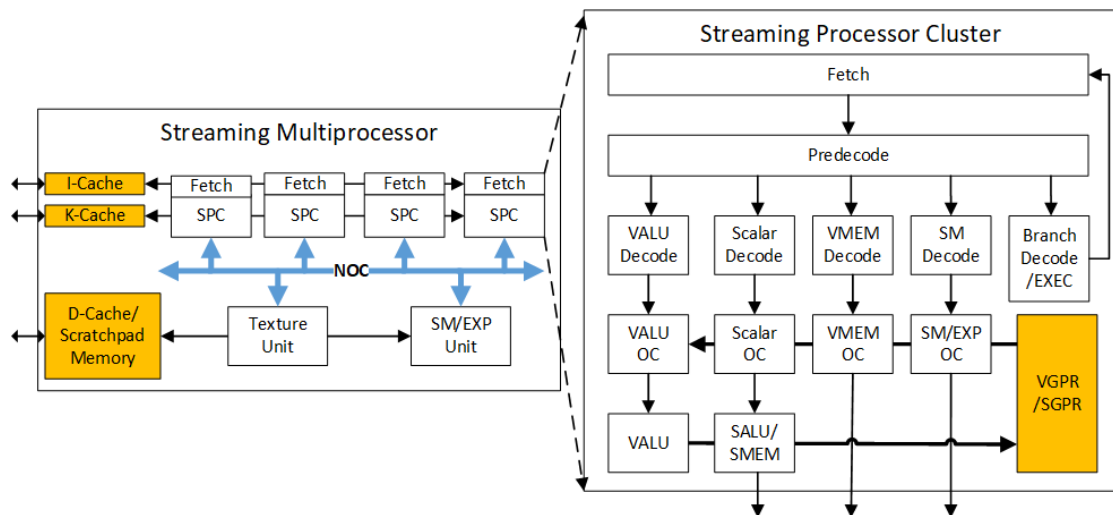


图 3.3 统一缓存架构的通用图形处理器的流多处理器架构

Figure 3.3 Streaming multiprocessor architecture of Unified-Cache GPGPU

3.3.3.2 流多处理器流水级设计

流多处理器总体上有 6 个流水级：取指令、预译码、译码、读寄存器、执行和写回。**1) 取指令阶段：**流多处理器采用轮询调度的方式选择一个 warp，并从

指令缓存中取出该 warp 的指令。**2) 预译码阶段**：根据指令的类型将指令发送到相应的译码器上。**3) 译码阶段**：对指令进一步译码，并通过记分板的方式解决指令之间的相关性。无相关的指令被发射到对应的执行单元的操作数收集器 (Operand Collector, OC) 上。如果某个 warp 的指令由于相关性或者栅障指令而导致无法发射，warp 调度器可以发射其他 warp 的可发射的指令来掩盖延迟。每个流处理器簇最多可以调度 16 个 warp 的指令，并且这些 warp 通常相互独立，没有相关性，所以大多数的延迟都可以被掩盖。**4) 读寄存器阶段**：多个操作数收集器相互交错地从多分区的寄存器堆中读取操作数，以避免寄存器端口冲突，然后将指令发射到对应的执行单元上。**5) 执行阶段**：向量算术单元执行向量算术指令；标量单元执行标量算术和访存指令，标量访存从常量缓存中读数据；分支单元执行分支指令；纹理单元除了执行纹理采样指令以外，还会执行向量访存指令，向量访存指令经过数据缓存执行读、写和原子操作；输出单元执行便笺式存储的读、写和原子指令，对于图形渲染应用，还可以将便笺式存储中的数据输出到图形固定功能流水线。**6) 写回阶段**：执行单元将计算的结果直接写回寄存器堆。**图形处理器不需要精确的异常处理，所以允许指令乱序提交。**

3.3.4 片上互连网络

如图3.2所示，Unified-Cache GPGPU 采用了链式的片上互连网络。网络上共有六个节点：两个流多处理器，几何引擎，图元引擎，输出混合器和二级缓存。片上互连网络内部由 4 个部分组成：上行窄总线，上行宽总线，下行窄总线和下行宽总线。各个节点会通过统一的总线进行通信和交互。例如，对于固定功能流水线的 warp 调度，固定功能流水线会通过上行窄总线和下行窄总线向流多处理器申请 warp 执行资源，然后通过上行宽总线初始化 warp 执行上下文，最后通过下行宽总线接收 warp 执行结果。对于流多处理器的一级缓存的缺失访存请求的处理，缺失访存请求通过下行窄总线发送到二级缓存，然后访存响应通过上行宽总线返回流多处理器。统一的链式的片上互连网络可以在性能接近的情况下有效节省面积。

3.4 统一缓存架构的软硬件协同实现

实现统一缓存架构需要软硬件协同实现，软件上需要驱动和编译器的支持，以通过便笺式存储和二级缓存来实现 Shader IO。统一缓存架构的实现包括三部分，分别是一级数据缓存和便笺式存储的统一、便笺式存储和 GPC 内 Shader IOB

的统一以及二级缓存和 GPC 间 Shader IOB 的统一。

3.4.1 一级数据缓存和便笺式存储的统一

不同的图形处理器应用会使用不同大小的便笺式存储。对于便笺式存储使用量少的应用，即使整个流多处理器处于满负载的情况下，依旧会有未使用的便笺式存储空间。如果使一级数据缓存和便笺式存储使用同一块片上存储，然后通过驱动动态地配置便笺式存储和一级数据缓存的大小，就可以将未利用的便笺式存储的存储空间重配置为一级数据缓存使用，从而增大了一级缓存的容量，缓解了缓存争用的问题。驱动可以为每个流多处理器分别配置一级缓存和便笺式存储的大小。计算引擎、几何引擎和图元引擎按需调度协作线程阵列和 warp，所需便笺式存储空间较大的协作线程阵列和 warp 会被优先调度至配置便笺式存储空间更大的流多处理器上执行。通用计算和图形渲染应用的 warp 可以在相同流多处理器上执行，但为了避免缓存相互污染，优先调度两者的 warp 在不同流多处理器上执行。

图3.4展示了统一的一级数据缓存和便笺式存储的架构。缓存和便笺式存储共享片上存储，该片上存储由 32 个双端口 SRAM 组成，每块 SRAM 每周期可以读和写 8 字节的数据，峰值带宽为每周期 256 字节。驱动会配置便笺式存储的大小，剩余的为一级数据缓存，便笺式存储和一级数据缓存会共享这每周期 256 字节的带宽。该片上存储会收到对缓存的读写请求和对便笺式存储的读写请求，两者通过交错的方式访问不同分区的 SRAM，来最大化带宽利用率。

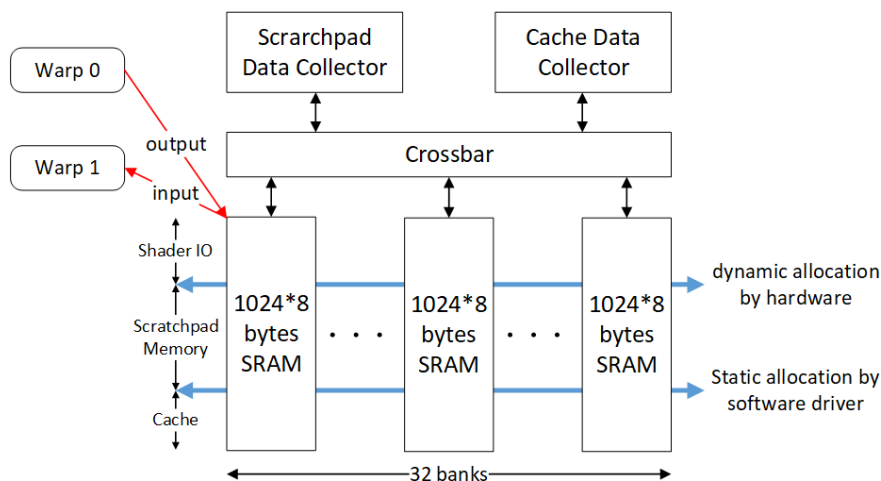


图 3.4 统一的一级数据缓存和便笺式存储的架构

Figure 3.4 Architecture of unified L1 data cache and scratchpad memory

3.4.2 便笺式存储和 GPC 内 Shader IOB 的统一

图形处理器会根据几何图元的数量将几何处理任务分配到不同的 GPC 上，从顶点着色器到几何着色器的图形渲染流水线都属于几何处理阶段。在几何处理阶段，Shader IO 都是在同一个 GPC 中执行，称为 GPC 内的 Shader IO。GPC 内所私有的片上存储包括便笺式存储和一级缓存，其中便笺式存储对于驱动可见且更容易操作，因此 Unified-Cache GPGPU 将便笺式存储和 GPC 内的 Shader IOB 相统一。

便笺式存储和 GPC 内 Shader IOB 的统一本质上是便将笺式存储作为 Shader IOB 来使用，需要编译器和固定功能流水线协同实现。利用便笺式存储实现 Shader IO 的关键点在于：1) 编译器生成着色器使用便笺式存储进行数据传输的代码。2) 硬件使有数据相关的着色器 warp 共享同一块便笺式存储空间。

3.4.2.1 基于便笺式存储的 Shader IO 的编译器实现

GPC 内需要 Shader IO 的着色器间数据传输包括顶点和外壳着色器之间、外壳和域着色器之间、域和几何着色器之间，以及顶点和几何着色器之间。编译器需要为各个着色器生成向便笺式存储写入输出数据以及从便笺式存储读取输入数据的代码，来实现 Shader IO。图3.5展示了基于便笺式存储的 Shader IO 的编译器实现，为了简化说明，每个 warp 只包含两个线程。

1) 顶点和外壳着色器间的 Shader IO：顶点和外壳着色器的各个线程的处理对象分别是相同的顶点，即顶点着色器的 warp 可以和外壳着色器的 warp 一一对应，因此不需要改变数据格式，编译器生成代码可以通过图3.5a的方式直接实现 Shader IO。

2) 外壳和域着色器间的 Shader IO：虽然外壳和域着色器的线程处理对象都是顶点，但是由于曲面细分使一个几何图元被切分为多个几何图元，顶点的数量会增多，因此一个外壳着色器的 warp 输出的顶点数据会被多个域着色器的 warp 所共享。通过图3.5b的方式，多个域着色器的 warp 可以访问相同地址的外壳着色器的输出顶点数据。

3) 顶点、域和几何着色器间的 Shader IO：在顶点和域着色器中，每个线程的处理对象为顶点，而在几何着色器中，每个线程的处理对象为几何图元。几何图元会包含多个顶点，因此多个顶点、域着色器的 warp 的输出数据会被一个几何着色器的 warp 所读取。图3.5c展示了顶点、域和几何着色器间 Shader IO 的实现，在顶点、域着色器的 warp 中，顶点是以数组结构体 (Structure of Array, SoA)

的格式处理的，编译器生成的代码需要将当前 SoA 格式的顶点数据根据几何图元类型转变为结构体数组（Array of Struture，AoS）的格式，然后存储在便笺式存储中。之后，几何着色器的 warp 从便笺式存储中以 AoS 的格式读出各个图元的顶点数据。

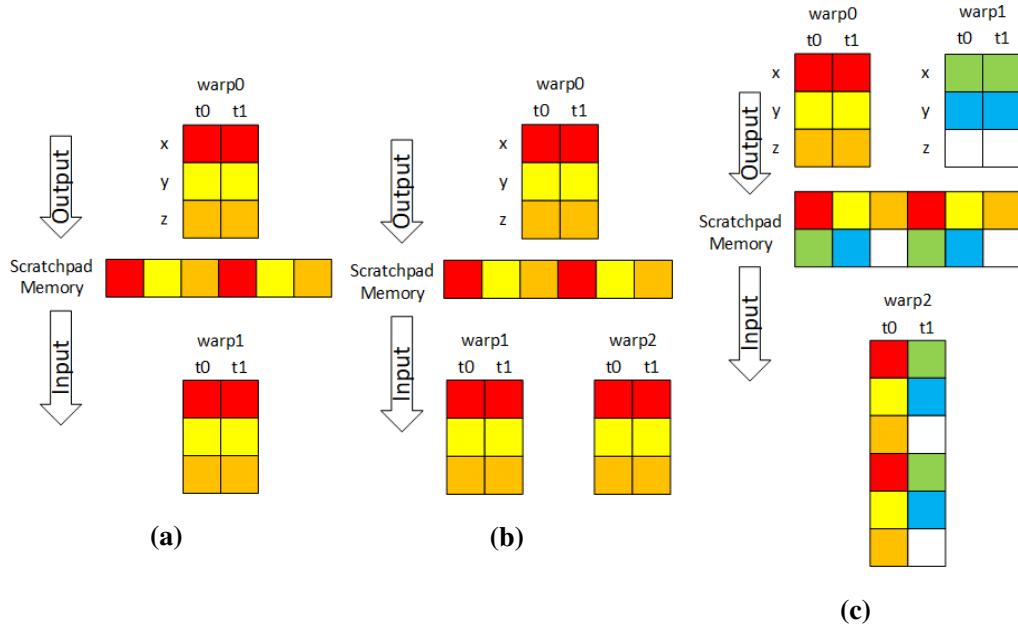


图 3.5 基于便笺式存储的 Shader IO 的编译器实现 (a) 顶点和外壳着色器间的 Shader IO，(b) 外壳和域着色器间的 Shader IO，(c) 顶点、域和几何着色器间的 Shader IO

Figure 3.5 Compiler implementation of Shader IO based on scratchpad memory (a) Shader IO between vertex and hull shader, (b) Shader IO between hull and domain shader, (c) Shader IO between vertex, domain and geometry shader

3.4.2.2 基于便笺式存储的 Shader IO 的硬件实现

GPC 内的 Shader IO 所涉及的着色器都属于几何处理阶段，几何引擎需要在不同着色器阶段通过对有数据相关的 warp 统一调度来实现便笺式存储的共享。为了使不同着色器的 warp 可以共享相同的便笺式存储空间，几何引擎会记录每个 warp 所分配的流多处理器标识和便笺式存储基地址。图3.4展示了基于便笺式存储的 Shader IO 的硬件实现，几何引擎动态地分配用于 Shader IO 的便笺式存储空间，warp0 向其中输出数据，然后 warp1 输入数据。图3.6展示了基于便笺式存储的 Shader IO 的执行流程，整个实现流程包括 4 部分，分别是 Shader IOB 的申请、共享、释放以及写回。

1) Shader IOB 的申请：几何引擎向流多处理器发起 warp0 执行资源（包括执行车道，寄存器和便笺式存储）申请，便笺式存储的申请大小取决于 Shader

IOB 所需的大小。对于顶点、域和几何着色器间的 Shader IOB，如果多个 warp 的输出数据共享连续的一块便笺式存储空间，那么只有第一个 warp 会进行申请，其余的 warp 直接使用第一个 warp 所申请的空间。几何引擎会记录每个 warp 申请到的流多处理器标识和便笺式存储基地址。

2) Shader IOB 的共享：当前着色器的 warp0 完成后，可以唤起后续着色器的已无数据相关的 warp3。warp3 可以从几何引擎中得到它所依赖的 warp0 的流多处理器标识和便笺式存储基地址。然后，warp3 向相同的流多处理器申请执行资源，以访问相同的便笺式存储。最后，初始化执行上下文，将 warp3 输入的便笺式存储基地址初始化为 warp0 输出的便笺式存储基地址。

3) Shader IOB 的释放：由于图形标准的按序处理图元的规定，图形渲染流水线的 warp 在固定功能流水线需要顺序处理。因此，直到依赖该便笺式存储内的数据的最后一个 warp，即 warp3，执行完成后，几何引擎会通知流多处理器释放图中蓝色的便笺式存储空间。

4) Shader IOB 的写回：当到达几何处理阶段的最后一个着色器后，其 warp 的输出数据需要返回给几何引擎。几何引擎会按 warp 的序向流多处理器依次发送便笺式存储读请求。便笺式存储收到几何引擎对 warp3 的请求后，将输出数据返回到几何引擎，并释放输出数据占用的图中黄色的便笺式存储空间。至此，几何处理阶段的基于便笺式存储的 Shader IO 完成。

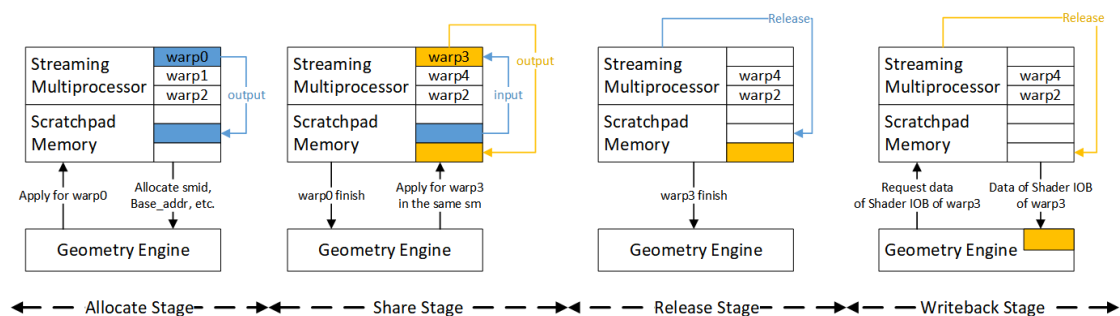


图 3.6 基于便笺式存储的 GPC 内的 Shader IO 的执行流程

Figure 3.6 Workflow of intra-GPC Shader IO based on scratchpad memory

3.4.3 二级缓存和 GPC 间 Shader IOB 的统一

为了负载均衡，经过了几何处理（包括裁剪和视口变换）的图元，需要根据图元所覆盖的屏幕区域发送到不同的 GPC 进行像素处理（包括光栅化，像素着色和输出混合）。此时图元数据的输入输出发生在不同 GPC 之间，称为 GPC 间的 Shader IO。因为便笺式存储和一级缓存是流多处理器私有的片上存储，自然也是

GPC 所私有的片上存储, 所以无法被利用为 GPC 间的 Shader IOB。而二级缓存是所有 GPC 所共享的存储, 适合用于 GPC 间的 Shader IO。因此, Unified-Cache GPGPU 将二级缓存和 GPC 间的 Shader IOB 相统一。

二级缓存和 GPC 间 Shader IOB 的统一本质上是将二级缓存作为 Shader IOB 来使用, 需要驱动和固定功能流水线的协同实现。利用二级缓存实现 GPC 间 Shader IO 的关键点在于: 1) 驱动在物理内存空间上为 Shader IOB 分配页面; 2) 硬件将 Shader IOB 的读写抽象为对内存的读写。

3.4.3.1 基于二级缓存的 Shader IO 的驱动实现

基于二级缓存的 Shader IO 与基于内存的相比在驱动的实现上基本一致, 两者都需要驱动在内存中为 Shader IO 分配物理页面。然而, 与传统的页面分配不同, Shader IO 的页面在应用执行期间不会被释放, 也不会被再次分配。而且与基于内存的 Shader IO 不同, 基于二级缓存的 Shader IO 实际上不会向分配的内存页面中读写数据。基于二级缓存的 Shader IO 分配内存页面的目的是使 Shader IO 的请求和普通访存请求的地址空间互斥, 避免两者发生冲突。

3.4.3.2 基于二级缓存的 Shader IO 的硬件实现

GPC 间的 Shader IO 并非通过着色器完成, 而是通过固定功能流水线完成。几何引擎在完成几何处理阶段后, 需要根据屏幕像素空间对任务重分配, 交由图元引擎开始像素处理阶段。由于一个几何图元可能覆盖多个屏幕像素空间, 一个几何图元的数据可能被多个 GPC 所共享, 所以需要在多个 GPC 间维持一致性以避免数据被过早释放。图3.7展示了基于二级缓存的 GPC 间 Shader IO 的架构, 其执行流程包括三部分, 分别是 Shader IOB 的分配、共享和释放。

1) Shader IOB 的分配: 几何引擎对几何图元进行裁剪、剔除和视口映射后, 需要将几何图元的数据写入二级缓存。构建一个几何图元需要图元的顶点坐标和图元引用的属性 (颜色, 纹理坐标等)。由于裁剪和多边形模式, 图元的顶点坐标和属性并非是一一对应的, 导致顶点坐标和属性分别有着各自的 Shader IOB。但是顶点坐标和属性的 Shader IO 的原理是一致的, 所以本文以顶点坐标为例解释如何实现 Shader IO。如图3.7所示, 完成几何处理的图元的顶点数据会被暂存在顶点缓冲区 (Vertex Buffer) 中, 当顶点缓冲区中的数据大小超过一个缓存行后, 将数据写入二级缓存中。二级缓存收到写请求后, 会按照正常的内存写请求操作, 为其分配缓存行并将数据写入。但是与普通的内存写请求不同, 被分配的缓存行会进入锁定状态, 此时该缓存行永远不会被替换直到收到明确的释放请

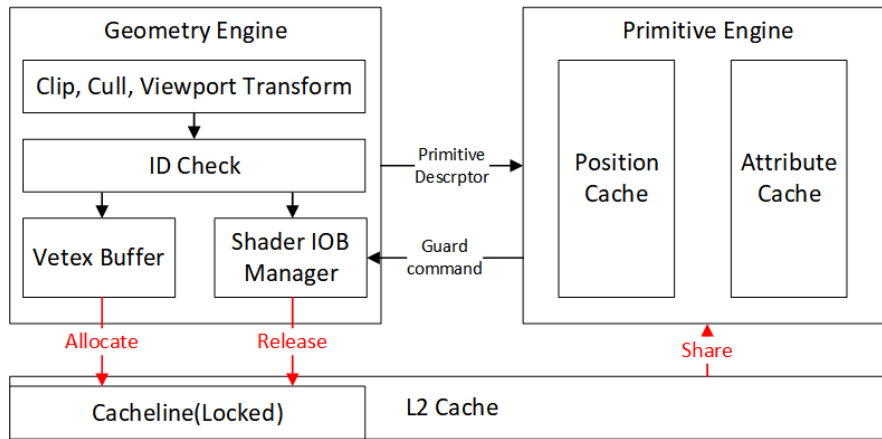


图 3.7 基于二级缓存的 GPC 间的 Shader IO 的架构

Figure 3.7 Architecture of inter-GPC Shader IO based on L2 cache

求。此外，为了减少缓存行的分配数量，本文利用了几何图元间的数据的局部性。在几何处理阶段中，几何图元的每个顶点坐标会被分配一个标识，如果两个顶点坐标的标识相同，说明它们的坐标完全相同，但是反之，顶点坐标相同，标识未必相同。如图3.7所示，标识检查（ID Check）会记录少量最近写入二级缓存的顶点坐标的标识和地址。如果新的几何图元存在相同标识的顶点坐标，那么就不需要将该顶点坐标写入二级缓存，而是使用相同标识的顶点坐标的地址，从而减少了缓存行的分配数量。

2) Shader IOB 的共享：几何引擎将图元数据写入二级缓存后，描述一个图元的信息需要图元的顶点坐标的地址和图元引用的属性的地址，称为图元描述符。图元描述符的数据宽度远比图元原本的表示形式小，可以以较低的开销在 GPC 之间传递。图元引擎收到图元描述符后，通过内存访问操作读取顶点坐标和属性。由于数据都以缓存行的形式存储在二级缓存中，所以一定会在二级缓存中命中。为了进一步优化性能，图元引擎分别使用了坐标和属性缓存来过滤对 Shader IOB 的访问，降低坐标和属性的访问延迟。

3) Shader IOB 的释放：由于一个几何图元可能被多个 GPC 所共享，所以直到这些 GPC 都从 Shader IOB 中读完该几何图元后，才能将该几何图元所占用的 Shader IOB 释放。Shader IOB 的存储是以缓存行为粒度，而一个几何图元的顶点坐标和属性通常不会超过一个缓存行。如果以几何图元为粒度释放 Shader IOB，那么会对一个缓存行发送多次释放请求。这不仅会导致设计更加复杂，而且过多的释放请求会增大片上互连网络的流量。因此，Unified-Cache GPGPU 以一个 warp 内的几何图元为粒度释放 Shader IOB。当一个 warp 内的几何图元都

写入 Shader IOB 后, 源 GPC 会向所有目的 GPC 广播一个作为“哨兵”的释放命令。该释放命令不参与图形渲染, 但是会随着图形渲染流水线前进。由于几何图元是按序处理的, 当释放命令前进到从 Shader IOB 读顶点坐标和属性的流水级时, 表明这个 warp 内的几何图元都已经从 Shader IOB 中读出, 此时目的 GPC 会向源 GPC 的 Shader IOB 管理器返回释放命令的响应。当源 GPC 收到所有目的 GPC 的响应后, 说明该 warp 内的几何图元都被所有 GPC 从 Shader IOB 中取出, 此时源 GPC 的 Shader IOB 管理器向二级缓存发送缓存行无效请求, 将 warp 所占用的所有缓存行转变为无效状态, 从而将对应的 Shader IOB 空间释放。

3.4.4 可拓展性分析

随着工艺制程的发展, 图形处理器的规模也在不断增长, 因此架构设计的可拓展性至关重要。基于内存的 Shader IO 由于访存系统和内存本身是随着图形处理器规模的增长而增大, 虽然会造成性能损失, 但是架构本身具有可拓展性。基于片上存储的 Shader IO 为了适应不断增多的流多处理器数量, 片上存储的容量也随之增大, 其所需的开销会逐渐增多以至于会限制图形处理架构的拓展, 因此不具备可拓展性。本文的 Unified-Cache GPGPU 的 Shader IO 基于便笺式存储和缓存, 便笺式存储和缓存的大小会随着流多处理器数量的增加而增大, 因此 Unified-Cache GPGPU 架构不仅可以提高性能, 而且具备可拓展性。

3.5 实验结果分析

3.5.1 实验评估方法

表3.1展示了用于实验评估的通用计算应用和图形渲染应用测试集。本文使用 Polybench^[76] 和 Parboid^[77] 的通用计算应用, glmark2^[78] 的图形渲染应用作为基准测试程序。本文通过仿真实验进行了性能分析和架构设计空间探索。

3.5.2 二级缓存大小与性能的关系

Unified-Cache GPGPU 因为将二级缓存和 GPC 间的 Shader IOB 相统一, 所以在保持总片上存储开销相同的情况下, 与传统的图形处理器相比二级缓存有着额外的约 16KB 的容量。如果传统图形处理器使用了 112KB 或者 128KB 大小的二级缓存, 那么 Unified-Cache GPGPU 就可以拥有 128KB 或者 144KB 的二级缓存。图3.8展示了不同应用在 112KB、128KB 和 144KB 的二级缓存上的性能比较, GM-ALL 表示所有应用的几何平均值, GM-CE 表示缓存高效型应用的几何平均值。总体上二级缓存越大图形处理器的性能越高。128KB 的二级缓存相对于

112KB 在所有应用上有着 5.54% 的性能提升，在缓存高效型应用上有着 10.9% 的性能提升。而 144KB 的二级缓存相对于 128KB 在所有应用上有着 6.73% 的性能提升，在缓存高效型应用上有着 11.5% 的性能提升。从图3.8中可以发现，对于缓存高效型应用，虽然 bicg、atax、mvt 和 syr2k 有着显著的性能提升，但是对于 gemv、syrk 和 gesu，其性能提升并不明显。这是因为二级缓存已经可以容纳 gemv、syrk 和 gesu 的工作集，即使增大二级缓存容量也不会显著降低其在二级缓存上的访问缺失率。即使部分有用的缓存行被替换出去，由于原本的缺失请求数量较少，内存访问也不会被阻塞，所以二级缓存的增大对它们的性能提升有限。对于缓存低效型应用和图形渲染应用，它们的性能瓶颈主要为吞吐量，二级缓存增大不会显著提升它们的性能。计算密集型应用的性能瓶颈为计算性能，故性能不变。与使用专用片上存储的 GPC 间 Shader IOB 相比，将二级缓存和 GPC 间 Shader IOB 相统一会增加图元引擎访问顶点数据的延迟，但是图元引擎使用了坐标和属性缓存来减小对性能的影响。

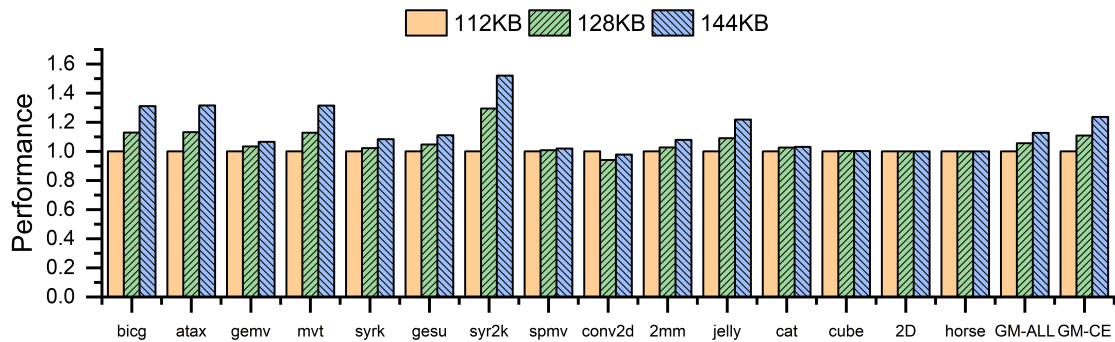


图 3.8 不同二级缓存大小的性能比较

Figure 3.8 Performance comparison of varying L2 cache sizes

3.5.3 一级缓存大小与性能的关系

由于 Unified-Cache GPGPU 将一级缓存、便笺式存储和 GPC 内的 Shader IOB 相统一，Unified-Cache GPGPU 不但可以根据程序的特性来配置一级缓存的容量，而且在保持总片上存储开销相同的情况下，Unified-Cache GPGPU 的一级缓存和便笺式存储的总容量相比传统的图形处理器大 20% 左右。图3.9展示了不同应用在 16KB、32KB 和 64KB 的一级缓存上的性能比较，GM-ALL 表示所有应用的几何平均值，GM-CE 表示缓存高效型应用的几何平均值。32KB 的一级缓存相对于 16KB 在所有应用上有着 5.46% 的性能提升，在缓存高效型应用上有着 10.1% 的性能提升。而 64KB 的一级缓存相对于 32KB 在所有应用上有着 12.4% 的性能提升，在缓存高效型应用上有着 23.0% 的性能提升。从图3.9中可以发现，对于

缓存高效型应用，虽然 bicg、atax、mvt 和 syr2k 有着显著的性能提升，但是对于 gemv、syrk 和 gesu，其性能提升并不明显，这与应用的工作集特点相关。syrk 在 32KB 和 64KB 的一级缓存上，性能几乎没有提升，是因为 64KB 所能容纳的工作集大小与 32KB 相同。同理，gesu 在 16KB 与 32KB 的一级缓存上，性能也几乎没有提升。但是在 64KB 的一级缓存上，gesu 的性能又有着显著提升，是因为 64KB 超过了将有用的缓存行替换出去的容量的界限，使得之前由于容量不足而缺失的请求在 64KB 的一级缓存上命中，因而性能有了显著提升。对于缓存低效型应用和图形渲染应用，它们的性能瓶颈主要为吞吐量，一级缓存增大不会显著提升它们的性能。计算密集型应用的性能瓶颈为计算性能，故性能不变。与使用专用片上存储的 GPC 内 Shader IOB 相比，将一级缓存、便笺式存储和 GPC 内 Shader IOB 相统一增加了几何引擎访问 GPC 内 Shader IOB 的延迟，但是减少了流多处理器向 GPC 内 Shader IOB 写数据的延迟，总体上数据传输的延迟基本保持不变。

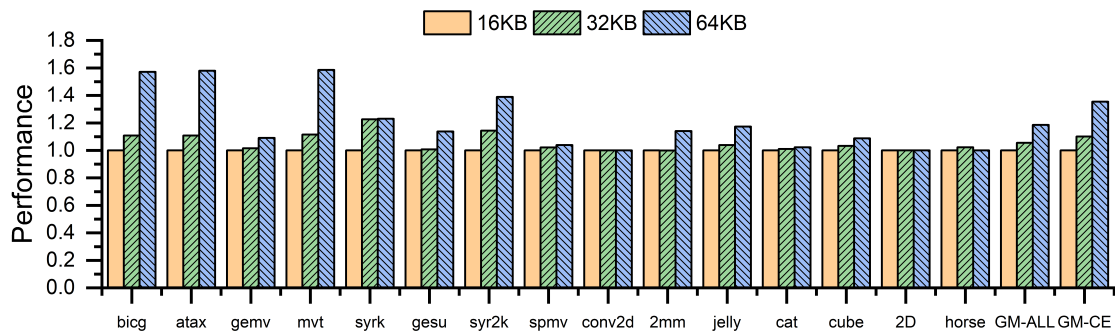


图 3.9 不同一级缓存大小的性能比较

Figure 3.9 Performance comparison of varying L1 cache sizes

3.5.4 片外数据访问量

为了进一步分析 Unified-Cache GPGPU 的性能提升，本文对片外数据访问量进行了分析。图3.10展示了 Unified-Cache GPGPU 和传统通用图形处理器在运行时从片外存储器访问的总数据量的比较，GM-ALL 表示所有应用的几何平均值，GM-CE 表示缓存高效型应用的几何平均值。Unified-Cache GPGPU 在所用应用上平均减少了 29% 的数据访问量，在缓存高效型应用上平均减少了 43% 的数据访问量。Unified-Cache GPGPU 增大了片上缓存的总容量，降低了总的缓存缺失率，所以减少了片外数据的访问量。图形渲染应用的片外数据访问量的降幅较低，有两点原因：一方面是因为 Unified-Cache GPGPU 会将部分缓存空间用于 Shader IO，所以实际使用的缓存容量小于通用计算应用；一方面由于图形渲染

应用的内存访问具有较高的空间局部性，在二级缓存上发生的缺失大多数为冷缺失，而缓存增大不能减少冷缺失的数量。

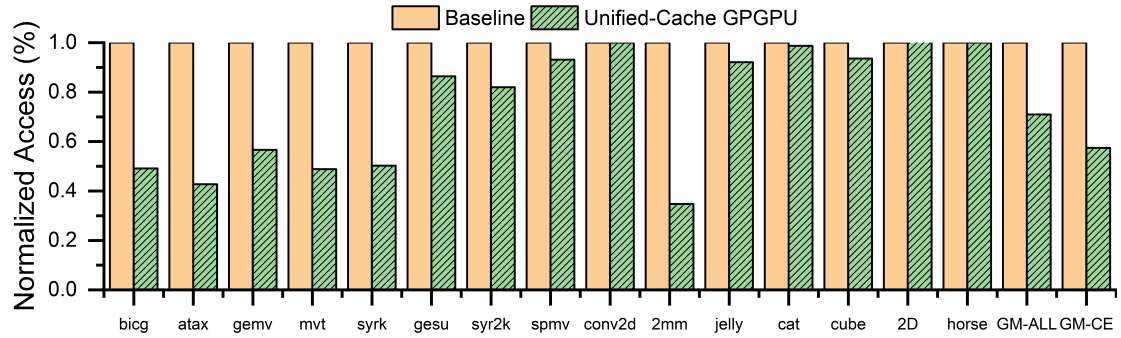


图 3.10 Unified-Cache GPGPU 和传统 GPGPU 的片外数据访问量的比较

Figure 3.10 Off-Chip memory traffic comparison between Unified-Cache GPGPU and traditional GPGPU

3.5.5 流多处理器组数量与性能的关系

本文设计的 Unified-Cache GPGPU 共包含两个流多处理器，让 Unified-Cache GPGPU 分别在两个流多处理器和一个流多处理器的配置下执行所有应用，图3.11展示了实验结果，GM-ALL 表示所有应用的几何平均值，GM-CE 表示缓存高效型应用的几何平均值。一个流多处理器与两个相比，性能在所有应用上平均下降了32.2%。两个流多处理器共享片上互连网络、二级缓存和内存带宽，流多处理器数量的减半虽然使峰值运算性能减半，但是内存带宽保持不变，在片上互连网络、二级缓存和内存带宽上的资源竞争用得到缓解，所以总的性能的降幅没有达到50%。

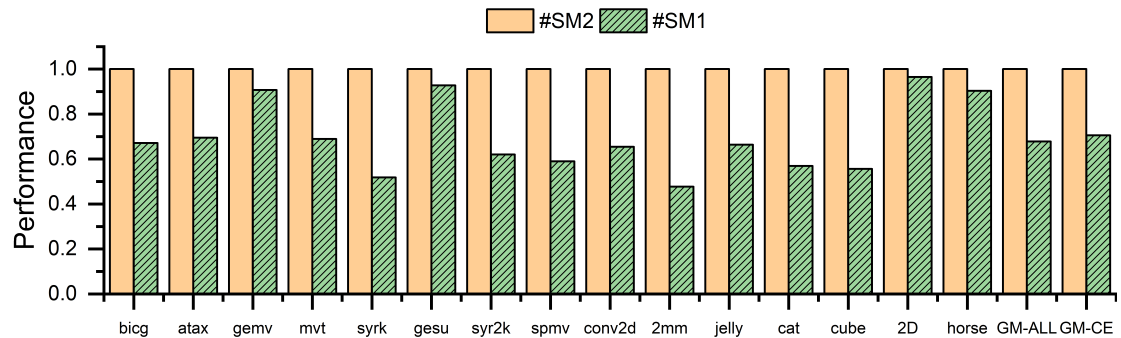


图 3.11 不同流多处理器数目的性能比较

Figure 3.11 Performance comparison of varying streaming multiprocessor numbers

从图3.11可以发现，除了 gemv、gesu、2D 和 horse 以外，其他应用都会由于流处理器数量的减少而导致性能明显下降。gemv、gesu 的性能之所以没有明显

下降,是因为它们的性能主要受限于访存系统,而不是运算单元的数量。2D 和 horse 的性能主要受限于图形固定功能流水线的光栅化的性能,光栅化生成像素的速率达不到流多处理器处理像素的速率。因此,即使流多处理器数量减少,性能也不会明显下降。

3.6 本章总结

在本章节中,本文设计了新的通用图形处理器架构,称为 Unified-Cache GPGPU,并最终进行了流片。Unified-Cache GPGPU 首先统一了一级缓存和便笺式存储,使一级缓存的容量可以根据应用特性来配置。然后统一了便笺式存储和 GPC 内的 Shader IOB,使 GPC 内的 Shader IOB 在未利用的情况下可以作为一级缓存使用,增大了一级缓存的容量。最后又统一了二级缓存和 GPC 间的 Shader IOB,使 GPC 间的 Shader IOB 在未利用的情况下可以作为二级缓存使用,增大了二级缓存的容量。Unified-Cache GPGPU 与传统的 GPGPU 相比,在相同片上存储开销的情况下,拥有更高的片上存储利用率,从而拥有更大的缓存容量,进而显著提高了缓存高效型应用的性能。

第4章 基于重命名的标记解耦合的一级缓存设计

本章介绍基于重命名的标记解耦合的一级缓存 (Decoupled-Tag Cache) 的设计动机、设计理念、具体实现和性能评估。Decoupled-Tag Cache 的设计动机是传统一级缓存架构不适用于大规模多线程应用的高吞吐量访存, 传统一级缓存架构的等待指令缓冲区 (Pending Instruction Buffer, PIB)、缺失状态保持寄存器 (Miss Status Holding Register, MSHR) 和紧耦合的组相联映射会限制并发访存请求数量, 使流多处理器不能有效掩盖内存访问的长延迟, 导致访存带宽利用率低。Decoupled-Tag Cache 的设计理念是基于重命名的方法重新设计一级缓存架构, 分别消除了等待指令缓冲区、MSHR 和紧耦合的组相联映射对并发请求数的限制。本文分别实现了顺序提交访存指令的 Decoupled-Tag Cache (In-Order Decoupled-Tag Cache, IO-Decoupled-Tag Cache) 和乱序提交访存指令的 Decoupled-Tag Cache (Out-of-Order Decoupled-Tag Cache, OO-Decoupled-Tag Cache)。图形渲染应用需要按程序顺序处理几何图元, 在 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 上性能相近。通用计算应用允许线程乱序完成, 在 OO-Decoupled-Tag Cache 上表现出更高的性能。为了评估和探索 Decoupled-Tag Cache 设计, 本文进行了 Decoupled-Tag Cache 和传统一级缓存在 Unified-Cache GPGPU 上的性能对比实验, 以及 Decoupled-Tag Cache 的设计空间探索实验。

4.1 设计动机: 传统一级缓存架构限制了并发访存请求数

4.1.1 传统一级缓存架构

图4.1展示了传统一级缓存架构。发射到访存单元的指令首先会执行访存合并, 然后将请求发射到标记阵列中进行比较, 并在等待指令缓冲区为指令分配空间。如果请求在缓存中命中, 那么直接从缓存中读数据, 将数据返回到等待指令缓冲区中并更新指令结果; 如果请求在缓存中缺失, 那么: 1) 更新标记的状态为保留 (Reserved) 状态, 并在缓存中为该缺失请求分配缓存行; 2) 在 MSHR 中分配项, 用来记录缺失请求; 3) 将缺失请求发送至内存系统的缺失请求队列中。如果由于缺失请求队列已满导致访存请求被阻塞, 说明一级缓存以外的内存系统 (片上互联总线、二级缓存、内存等) 已经处于满负载的状态, 此时的瓶颈不在一级缓存。当缺失请求的响应返回后, 除了更新标记状态和缓存行, 还要更新等待指令缓冲区中对应指令的结果。如果某条指令的结果更新完成, 那么等待

指令缓冲区会将该指令提交至下一流水级。从一级缓存处理访存缺失的行为可以发现，如果流多处理器无法发射足够多的缺失访存请求以掩盖长延迟的内存访问操作，原因可能有三点：1) 等待指令缓存区容量不足；2) 无法分配标记和缓存行；3) MSHR 的容量不足。

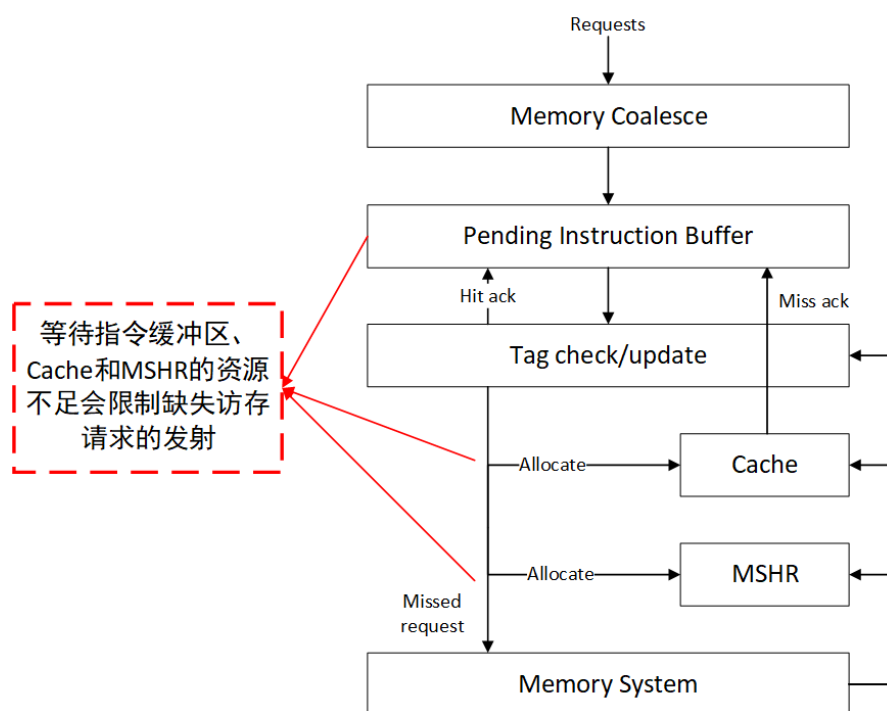


图 4.1 传统一级缓存架构

Figure 4.1 Traditional L1 cache architecture

等待指令缓冲区容量不足 传统一级缓存架构为了降低延迟，通常使用广播的方式更新等待指令缓冲区中所有指令的结果，使得等待指令缓冲区需要采用寄存器设计。而且为了减小面积开销，等待指令缓冲区的项数有限，最多为十数项。而在图形处理器上，流多处理器最多可以同时执行数十个 warp。以典型的矩阵乘应用为例，每个 warp 可以并发执行两条访存读指令，十数项的等待指令缓冲区不足以保证访存不阻塞。

无法分配标记和缓存行 为了避免片上互联网络的阻塞，一级缓存会为缺失请求预分配缓存行，并更新标记状态为保留（Reserved）状态。直到缺失请求的响应返回，将对应标记的状态更新为有效（Valid）状态后，标记和缓存行才能被再次分配。如果一级缓存在较短时间内收到多个相同组索引的缺失请求，使该组索引上所有相联的标记都处于保留状态，那么当再次收到相同组索引的缺失请求

时,就会因为无法分配标记和缓存行而导致访存阻塞。虽然采用合适的组索引哈希函数可以降低多个缺失请求同时访问相同组索引的概率,但是无法完全避免这种情况。

MSHR 的容量不足 MSHR 用来记录缺失请求的状态,并将相同地址的缺失请求合并。MSHR 的项数会限制最大并发缺失请求数。MSHR 通常采用全相联的结构,传统的一级缓存的 MSHR 通常有数十项。然而,流多处理器和二级缓存之间通过片上互连网络连接,现有的大规模图形处理器的一级缓存的缺失请求的延迟可以达到数百周期。数十项的 MSHR 无法完全掩盖访存延迟,导致访存带宽利用率低。而且,由于 MSHR 采用全相联的结构,数百项的 MSHR 会导致芯片的面积开销不可接受。

4.1.2 传统一级缓存架构访存瓶颈的量化分析

为了量化传统一级缓存架构的访存瓶颈,本文在使用传统一级缓存的 Unified-Cache GPGPU 上进行了量化分析,所使用的一级缓存的大小为 16KB,使用了 8 项的等待指令缓冲区和 32 项的 MSHR。表4.1展示了用于实验的通用计算应用和图形渲染应用,由于驱动和编译器尚不完善,本文使用来自 Polybench^[76] 和 Parboild^[77] 的部分通用计算应用和来自 glmark2^[78] 的部分图形渲染应用作为基准测试程序。基准测试程序的规模保证图形处理器的执行车道可以处于满负载的状态。为了使评估更加全面,对于通用计算,本文分别使用了缓存高效型和低效型应用以及计算密集型应用,对于图形渲染,本文使用了不同顶点数量、不同纹理大小和不同分辨率的应用。

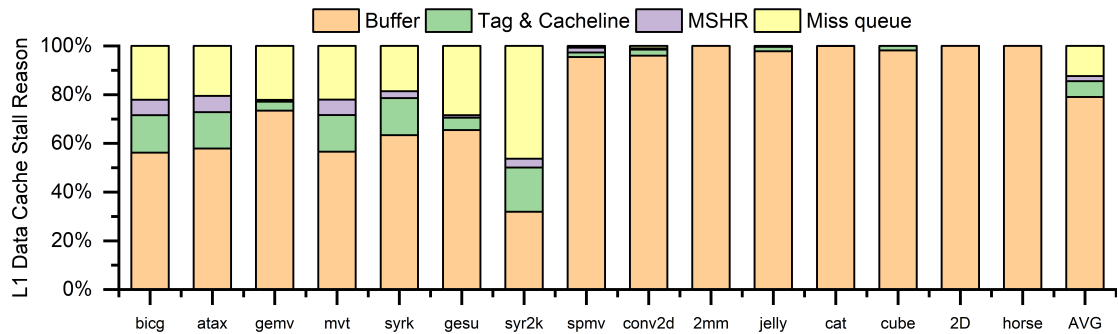


图 4.2 一级缓存访存瓶颈的量化分析

Figure 4.2 Quantitative analysis of L1 cache access bottlenecks

图4.2展示了等待指令缓存区、标记和缓存行、MSHR 和缺失请求队列在不同应用上导致访存阻塞的概率,AVG 表示所有应用的平均值。实验结果表明,平

均 79% 的访存阻塞的原因是等待指令缓冲区容量不足，平均 6.5% 的访存阻塞的原因是无法分配标记和缓存行，平均 2.1% 的访存阻塞的原因是 MSHR 容量不足。剩余的在缺失请求队列上发生的访存阻塞，其本质是访存子系统的瓶颈，与一级缓存无关。

表 4.1 评估图形处理器的基准测试应用

Table 4.1 GPU applications for benchmark

General-Purpose Cache-Efficient Applications			
App	Description	App	Description
bicg ^[76]	BiCGStab Linear Solver	atax ^[76]	Matrix Transpose-Vector Mul.
gemv ^[76]	Vector Mul. and Matrix Add.	mvt ^[76]	Matrix Vector Product-Transpose
syrk ^[76]	Symm. rank-k operations	gesu ^[76]	Scalar, Vector and Matrix Mul.
syr2k ^[76]	Symm. rank2k operations		
General-Purpose Cache-Inefficient Applications			
App	Description	App	Description
spmv ^[77]	Sparse Matrix-Dense Vector Mul.	2mm ^[76]	2 Matrix Mul. and Matrix Add.
General-Purpose Compute-Intensive Applications			
App	Description	App	Description
conv2d ^[76]	2D Convolution		
Graphics Rendering Applications			
App	#Vertex	Texture size	Resolution
jellyfish ^[78]	13200	2*256*256	800*600
cat-tex ^[78]	43044	512*512	800*600
cube-tex ^[78]	36	512*512	800*600
2D-tex ^[78]	4	128*128	256*256
horse ^[78]	21516	0	800*600

从图4.2可以发现，对于图形渲染应用，等待指令缓冲区不足是访存瓶颈的主导因素，这是因为高空间局部性导致单条访存指令所能发射的请求数较少，在等待指令缓冲区能够并发执行的指令数有限的情况下，较少的请求数不会触发标记和缓存行与 MSHR 的瓶颈。虽然实验结果显示等待指令缓冲区不足是主要瓶颈，但是随着等待指令缓冲区的增大，标记和缓存行与 MSHR 成为访存瓶颈的比例也会逐渐增大。只消除单一瓶颈所带来的性能收益会再次受限于其他瓶颈。只有消除了这三个瓶颈，一级缓存才可以并发处理更多的缺失请求，从而增

大掩盖访存延迟的能力和访存带宽利用率。

4.2 设计理念：消除传统一级缓存架构对并发请求数的限制

Decoupled-Tag Cache 的设计理念有三个：1) 采用重命名的方法，消除标记和缓存行紧耦合的映射关系；2) 不使用 MSHR，而是通过访命中行为来合并缺失请求；3) 使用大容量的 SRAM 设计等待指令缓冲区，允许同时执行足够多的指令。

4.2.1 消除紧耦合组相联映射导致的访存阻塞

组相联结构之所以会导致访存阻塞，是因为标记和缓存行以紧耦合的方式映射。当位于同一组上的缓存行都已经被分配之后，则无法再分配缓存行。而且由于标记和缓存行以紧耦合的方式映射，标记也无法被再次分配。此时，同一组上的缺失访存请求会被阻塞。然而，其他组的缓存行可能处于可以被分配的状态。如果采用重命名的方法将标记映射到可以被分配的缓存行上，就可以避免访存阻塞。Decoupled-Tag Cache 将标记视作逻辑缓存空间，将缓存行视作物理缓存空间，采用重命名的方法将逻辑缓存空间映射到物理缓存空间，从而将标记和缓存行的映射解耦合。由于消除了标记和缓存行的紧耦合映射，标记不再需要 Reserved 状态，可以在每次访问后立刻更新，即使缺失请求的响应还没有返回，标记的状态也可以立刻更新为 Valid 状态。每次发生缺失请求时，Decoupled-Tag Cache 首先在缓存行阵列里分配任意空闲的缓存行，然后选择被替换的标记项，最后更新标记和重命名表，将标记项映射至被分配的缓存行。

4.2.2 消除 MSHR 导致的访存阻塞

MSHR 的作用是合并相同地址的缺失访存请求，避免发送重复的访存请求。Decoupled-Tag Cache 没有采用 MSHR，而是通过缓存命中行为来合并相同地址的请求。当缓存命中时，表明对应的缓存行要么已经在缓存中，要么已经发出了缺失访存请求，此时都不需要再次发出相同的缺失请求。当缓存缺失时，虽然可能存在正在处理的相同地址的缺失请求，但是该缺失请求不会被合并，会发出重复的缺失请求。然而，无论是图形渲染应用还是通用计算应用，对于可利用的局部性都有着重用距离近的特征，发出重复请求的情况很少，几乎不会造成访存资源的浪费。

4.2.3 消除等待指令缓冲区导致的访存阻塞

Decoupled-Tag Cache 使用双端口 SRAM 设计了可以容纳数百条指令的等待指令缓冲区。该等待指令缓冲区没有为指令结果预留空间，而是为每条指令增加了等待计数器。缺失请求的响应不能直接更新指令的结果，而是更新指令的等待计数器。指令的等待计数器的值等于其依赖的缺失请求数，每次缺失请求返回后，就将对应指令的等待计数器递减，直到等待计数器的值为 0 时，表明指令的所有依赖的缓存行都已在缓存中。IO-Decoupled-Tag Cache 的等待指令缓冲区是先入先出的队列，如果指令 A 的请求命中了缓存，那么之前的某条指令 B 一定发出过该命中缓存行的请求。当指令 A 到达队列的队头时，指令 B 一定已经完成，该命中缓存行一定已经在缓存中准备就绪。虽然等待指令缓冲区的深度较大，但是因为不用为指令结果预留空间，所以宽度相对较小。经过综合评估，其面积开销在一级缓存中的比例为 12.6%，可以接受。虽然等待指令缓冲区的容量依旧有限，不可能支持无限的指令并发执行，但是数百条指令的并发缺失请求几乎可以完全利用一级缓存以下的内存子系统，足以完全掩盖访存延迟。因此，可以认为消除了等待指令缓冲区导致的访存阻塞。

4.3 基于重命名的标记解耦合的一级缓存实现

图形渲染应用的执行有两个特点：1) 着色器的 warp 在固定功能流水线是按序提交的。2) 绝大多数情况下，着色器程序没有内存写指令和原子指令。对于图形渲染应用，流多处理器通常采用最老优先的方式调度 warp，以基本按序的方式执行 warp。这样不同 warp 的同一访存命令也会以基本按序的方式发射到访存单元。即使乱序完成访存指令，也会由于按序提交 warp 的特征，对性能几乎没有提升。本文针对图形渲染应用设计了按序提交的基于重命名的标记解耦合的一级缓存（In-Order Decoupled-Tag Cache，IO-Decoupled-Tag Cache）。

对于通用计算应用，内核内的线程的完成顺序几乎没有要求。而且，通用计算应用的内核程序经常使用内存写指令和原子指令。因为图形处理器的缓存一致性是在二级缓存上实现的，所以内存写指令和原子指令都必须经过二级缓存，导致内存写指令和原子指令的平均访存延迟远大于内存读指令。假设采用按序提交指令的方式，那么短延迟的已完成的内存读指令会被长延迟的内存写和原子指令所阻塞，导致性能降低。本文在 IO-Decoupled-Tag Cache 的基础上设计了乱序提交的基于重命名的标记解耦合的一级缓存（Out-of-Order Decoupled-Tag Cache，OO-Decoupled-Tag Cache）。

4.3.1 顺序的基于重命名的标记解耦合的一级缓存实现

本文从架构、执行流程和正确性三个方面介绍顺序的基于重命名的标记解耦合的一级缓存的实现。

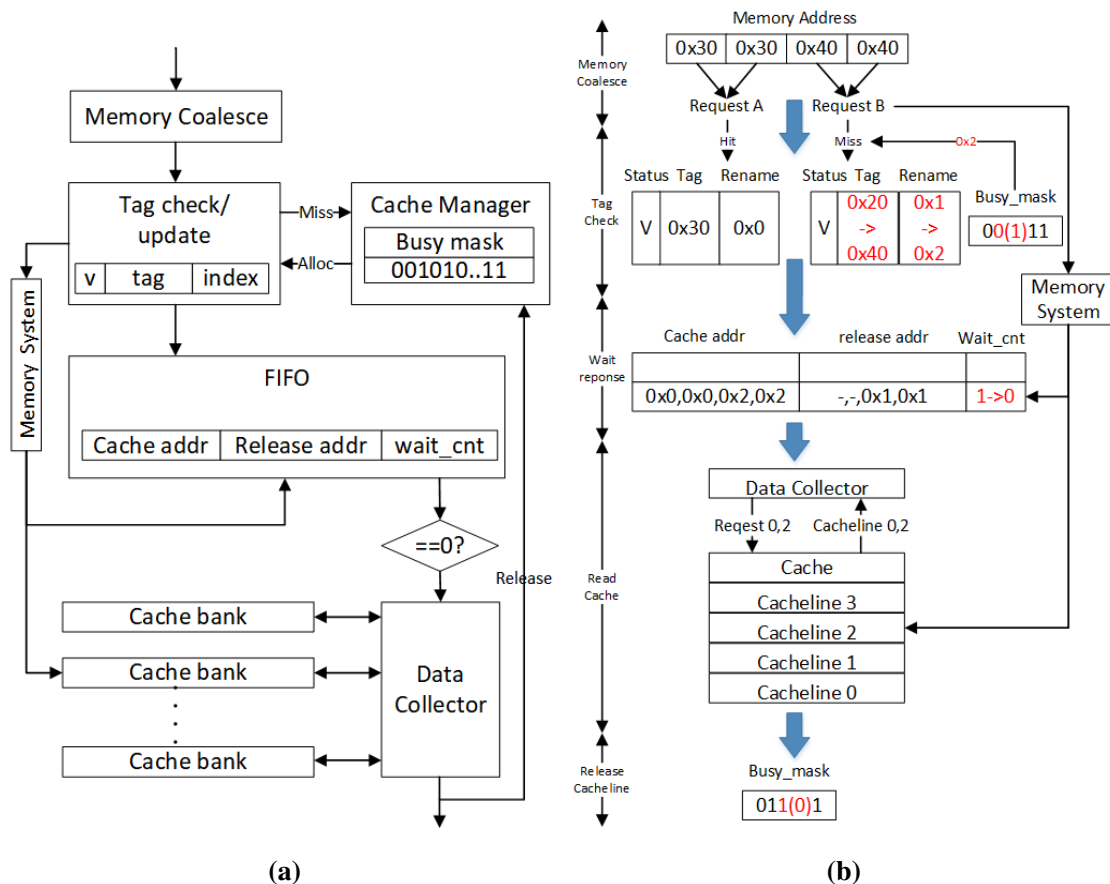


图 4.3 IO-Decoupled-Tag Cache 的实现 (a) IO-Decoupled-Tag Cache 的架构, (b) IO-Decoupled-Tag Cache 的执行流程

Figure 4.3 IO-Decoupled-Tag Cache implementation (a) IO-Decoupled-Tag Cache architecture, (b) IO-Decoupled-Tag Cache workflow

4.3.1.1 顺序的基于重命名的标记解耦合的一级缓存的架构

图4.3a展示了 IO-Decoupled-Tag Cache 的架构, 主要包含 5 个部分: 访存合并单元, 标记阵列, 等待指令队列, 缓存行阵列和缺失请求队列。

访存合并单元 (Memory Coalesce) 为了降低面积开销, 包含 2 个合并单元, 每个合并单元分别负责 16 个线程的地址合并。访存合并单元可以将访问相同缓存行的线程的请求合并为一个请求。

标记阵列由 4 个分区组成, 每周期最多可以处理 4 个请求, 以提高访存带宽的利用率。标记阵列需要除了需要记录每个标记的状态 (v) 和值 (tag) 以外,

还需要记录每个标记被重命名表所映射的缓存行索引 (index)。标记阵列通过缓存管理器 (Cache Manager) 分配和释放缓存行, 忙掩码 (Busy_mask) 的每一位对应一项缓存行。如果该位置位, 说明该缓存行已被分配, 否则说明该缓存行空闲。

等待指令队列采用 256 项的双端口 SRAM 设计, 每一项对应于一条指令。在绝大多数的应用中, 并发的 256 条访存指令足够完全掩盖访存延迟。每项会记录指令的每个线程的请求所命中的缓存行的索引和缺失后分配的缓存行索引 (Cache addr)、每个请求所替换的标记的缓存行的索引 (Release addr) 以及等待计数器 (wait_cnt)。等待计数器表示该指令正在等待完成的缺失请求数, 当等待计数器为 0 时, 说明该指令所有缺失请求的缓存行都已经在缓存中就绪。Release_addr 所记录的索引的缓存行在指令完成后会被释放, 并将 Busy_mask 对应的位复位。

缓存行阵列由多个分区 (Cache bank) 组成, 指令读缓存时在多个分区上交错读取数据以避免端口冲突问题。当等待指令队列首项的等待计数器为 0 时, 指令从队列中弹出并进入到数据收集器 (Data Collector) 中。数据收集器根据 Cache addr 以及请求在缓存行内的偏移地址, 计算请求访问的数据在分区上的索引, 并合并访问相同数据项的请求。不同分区的请求可以在同一时钟周期内完成。**缓存行阵列的物理缓存空间大小必须大于标记阵列的逻辑缓存空间**。否则, 当缓存行全部被分配 (Busy_mask 全部置位) 且恰好一直没有发生标记替换时, 新来的指令的缺失请求会由于无法分配缓存行而导致死锁。理论上, 缓存行阵列的容量越大, 可支持的并发缺失请求数越多, 掩盖访存延迟的能力越大。

缺失请求队列通过片上互连网络将缺失请求发射到二级缓存。在标记阵列上缺失的请求会被暂时存储在缺失请求队列中, 等待被发送至内存系统 (Memory System, 包括片上互连网络、二级缓存和内存) 中。从内存系统返回的响应除了会被写入到缓存行阵列中, 还会将对应指令的等待计数器递减。

4.3.1.2 顺序的基于重命名的标记解耦合的一级缓存的执行流程

图4.3b展示了内存读指令在 IO-Decoupled-Tag Cache 上的执行流程, 为了简化, 图4.3b中的内存读指令的线程宽度为 4, 标记阵列有两个分区, 每个分区各有一项标记, 等待指令缓冲区有两项, 缓存行阵列有 4 个缓存行。IO-Decoupled-Tag Cache 的指令执行流程包括 5 个阶段: 访存合并、标记检查、等待响应、读缓存和缓存行释放。

1) 访存合并：访存合并单元将指令访问相同缓存行的请求合并，0 和 1 号线程的请求被合并为请求 A，访问地址为 0x30，2 和 3 号线程的请求被合并为请求 B，访问地址为 0x40。

2) 标记检查：在标记检查阶段，请求 A 在分区 0 上命中，查询重命名表，得到缓存行的索引 0x0；请求 B 在分区 1 上未命中，替换了 0x20 标记，更新标记为 0x40，然后查询缓存行的忙掩码 (Busy_mask)，发现第 2 位为 0，说明索引为 0x2 的缓存行空闲，于是被分配了索引为 0x2 的缓存行，并将 Busy_mask 的第 2 位置位。根据重命名表，被替换的 0x20 标记所映射的缓存行索引为 0x1。最后缺失请求被发射到缺失请求队列中，等待发往二级缓存。

3) 等待响应：标记检查结束后，指令被发射到等待指令队列中，由于指令只有一个请求在缓存上未命中，所以等待计数器被初始化为 1。等待指令队列记录了每条指令的所有请求所依赖的缓存行的索引 (Cache_addr)，以及每个请求所替换的标记的缓存行的索引 (release_addr)，release_addr 所对应的缓存行在指令执行完成后会被释放。缺失请求的响应返回后，指令的等待计数器递减，并将缓存行写入到被分配的 0x2 索引的缓存行上。

4) 读缓存：当指令到达等待指令队列的首项并且等待计数器为 0 时，从队列中取出指令。指令根据 Cache_addr 从缓存中读出索引为 0x0 和 0x2 的缓存行，并更新指令的结果。

5) 释放缓存行：指令得到结果后，会被提交至下一流水级，同时根据 release_addr 释放索引为 0x1 的缓存行，将 Busy_mask 的第 1 位复位。

4.3.1.3 顺序的基于重命名的标记解耦合的一级缓存的正确性

为了证明 IO-Decoupled-Tag Cache 的正确性，需要保证指令所依赖的缓存行在读缓存时一定在缓存中。IO-Decoupled-Tag Cache 顺序提交指令，标记检查时的指令顺序一定与释放缓存行时的指令顺序相同。指令在标记检查时会得到其所依赖的缓存行的索引，而这些缓存行只会在后续指令替换了标记后才能被释放。而且，由于释放缓存行的指令顺序与标记检查一致，当前指令读缓存时后续指令不可能释放缓存行，使得指令所依赖的缓存行在读缓存时一定在缓存中，由此证明了 IO-Decoupled-Tag Cache 的正确性。

4.3.2 乱序的基于重命名的标记解耦合的一级缓存的实现

与 IO-Decoupled-Tag Cache 不同，OO-Decoupled-Tag Cache 乱序提交指令，导致了两者在架构和执行流程上的区别。

4.3.2.1 乱序的基于重命名的标记解耦合的一级缓存的架构

图4.4a展示了 OO-Decoupled-Tag Cache 的架构，OO-Decoupled-Tag Cache 与 IO-Decoupled-Tag Cache 的架构总体上相同，但是为了支持乱序提交指令，OO-Decoupled-Tag Cache 有四点改进。

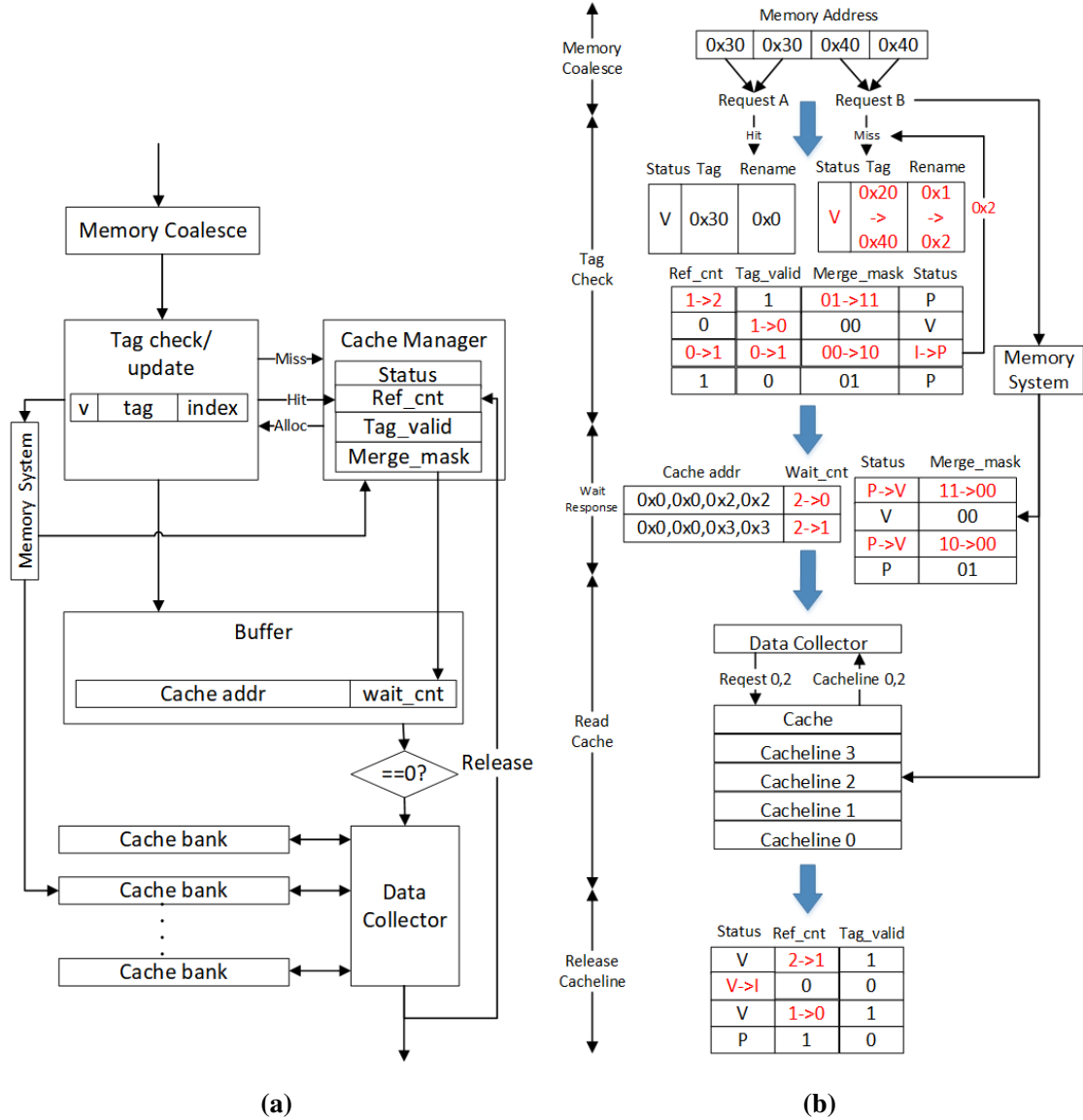


图 4.4 OO-Decoupled-Tag Cache 的实现 (a) OO-Decoupled-Tag Cache 的架构, (b) OO-Decoupled-Tag Cache 的执行流程

Figure 4.4 OO-Decoupled-Tag Cache implementation (a) OO-Decoupled-Tag Cache architecture, (b) OO-Decoupled-Tag Cache workflow

1) OO-Decoupled-Tag Cache 的等待指令缓冲区的管理方式由队列变为了随机存取。如果指令的等待计数器的值为 0，表明其所依赖的缓存行都已经在缓存中就绪，那么指令可以立刻从等待指令缓冲区中取出，从而实现指令的乱序提

交。OO-Decoupled-Tag Cache 会通过引用计数器 (Ref_cnt) 和 Tag_valid 状态位来释放缓存行, 无需记录指令所替换的标记的缓存行索引 (release_addr)。

2) 为了能同时更新依赖某缓存行的所有指令的等待计数器, OO-Decoupled-Tag Cache 为每个缓存行增加了一个合并掩码 (Merge_mask), 用来记录依赖该缓存行的所有指令。**合并掩码的宽度等于等待指令缓冲区所能容纳的指令数**, 掩码的第 i 位对应着等待指令缓冲区的第 i 条指令, 如果缓存行索引为 N 的缓存行的合并掩码的第 i 和第 j 位为 1, 表示等待指令缓冲区的第 i 和第 j 条指令在等待缓存行索引为 N 的缓存行返回。

3) OO-Decoupled-Tag Cache 为每个缓存行增加了新的状态 (Status), 除了空闲 (Invalid) 和有效 (Valid) 状态外, 新增加了等待 (Pending) 状态。Pending 状态表示缓存行已分配, 但是响应还未返回。Valid 状态表示缓存行已分配并且响应已返回。

4) OO-Decoupled-Tag Cache 为每个缓存行增加了一个引用计数器 (Ref_cnt), 引用计数器的值表示当前依赖该缓存行的正在执行的指令数。OO-Decoupled-Tag Cache 还为每个缓存行增加了 Tag_valid 状态位, 该位用来表示缓存行的标记是否有效, 如果某个标记被替换出去, 那么将其缓存行的 Tag_valid 复位。引用计数器和 Tag_valid 状态位采用类似垃圾回收的方式来协同实现缓存行的释放。对于 IO-Decoupled-Tag Cache, 当缓存行的标记被替换后, 那么后续指令无法命中该缓存行, 即不存在依赖该缓存行的后续指令。因此, 当替换标记的指令完成后, 就可以将该缓存行释放。然而, 乱序提交会破坏顺序提交时的指令间缓存行的依赖关系, 所以使用引用计数器和 Tag_valid 位来实现缓存行的释放。当缓存行的引用计数器为 0 且 Tag_valid 位复位时, 表明当前既没有依赖该缓存行的正在执行的指令, 后续指令也无法命中该缓存行, 也就是说对正在执行和后续的指令来说该缓存行是无用的, 可以将该缓存行释放。

4.3.2.2 乱序的基于重命名的标记解耦合的一级缓存的执行流程

图4.4b展示了内存读指令在 OO-Decoupled-Tag Cache 上的执行流程。OO-Decoupled-Tag Cache 的指令执行流程包括 5 个阶段: 访存合并、标记检查、等待响应、读缓存和缓存行释放。

1) **访存合并**: 在访存合并阶段 OO-Decoupled-Tag Cache 和 IO-Decoupled-Tag Cache 基本相同, 访问相同缓存行地址的请求被合并, 合并生成了请求 A 和请求 B, 同时为指令在等待指令缓冲区中分配了空闲项, 索引为 0x1。

2) 标记检查: 在标记检查阶段, 请求 A 在分区 0 上命中, 查询重命名表得到缓存行的索引为 0x0 且状态为 Pending, 首先根据 0x0 的索引, 将表中第 0 项的缓存行的引用计数递增, 由 1 变为 2, 然后根据指令在等待指令缓冲区中的索引 0x1, 将缓存行的合并掩码的 0x1 位置位, 由 01 变为 11。请求 B 在分区 1 上未命中, 替换了标记 0x20, 更新标记为 0x40, 并被分配了索引为 0x2 的缓存行, 根据重命名表, 被替换的标记所映射的缓存行索引为 0x1, 故将索引 0x1 的缓存行的 Tag_valid 状态位复位。然后初始化被分配的索引为 0x2 的缓存行的引用计数为 1, 置位 Tag_valid, 状态由 Invalid 变为 Pending, 同时初始化该缓存行的合并掩码, 只将 0x1 位置位, 即初始化为 10。最后更新重命名表, 将该缺失请求的标记映射到索引为 0x2 的缓存行, 由 0x1 变为 0x2。

3) 等待响应: 标记检查结束后, 指令被发射到等待指令缓冲区的 0x1 项中。根据标记检查的结果, 0 和 1 号线程需要读 0x0 索引的缓存行, 2 和 3 号线程需要读 0x2 索引的缓存行, 所以 Cache_addr 为 0x0, 0x0, 0x2, 0x2。虽然请求 A 命中, 但是由于命中的缓存行的状态为 Pending, 请求 A 需要等待该缓存行就绪。再加上未命中的请求 B, 该指令的等待计数器被初始化为 2。然后, 索引为 0x0 的缓存行的响应返回后, 根据合并掩码 11, 将等待指令缓冲区中第 0 和第 1 条指令的等待计数器递减; 索引为 0x2 的缓存行的响应返回后, 根据合并掩码 10, 将等待指令缓冲区中第 1 条指令的等待计数器递减。而且, 缓存行的响应返回后, 将索引为 0x0 和 0x2 的缓存行的状态由 Pending 转变为 Valid。

4) 读缓存: 此时, 等待指令缓冲区中的索引为 0x1 的指令的等待计数器为 0, 将指令从等待指令缓冲区中取出, 然后指令根据 Cache_addr 从缓存中取出索引为 0x0 和 0x2 的缓存行, 并更新指令结果。虽然等待指令缓冲区的索引为 0x0 的项也存在指令, 但是由于其等待计数器的值不为 0, 表明其依赖的缓存行没有全部就绪, 还需要等待缓存行索引为 0x3 的缓存行的响应返回, 不能开始读缓存。

5) 释放缓存行: 读缓存结束后, 释放指令所访问的缓存行的引用, 索引 0x0 和 0x2 的缓存行的引用计数器递减, 分别由 2 变 1 和由 1 变 0。如果缓存行的引用计数器为 0, 而且 Tag_valid 状态位为 0, 那么该缓存行可以被释放, 将其状态转变为 Invalid。图 4.4b 可以看到, 缓存行索引 0x1 的缓存行的标记被替换后, 其引用计数变为 0 且 Tag_valid 复位, 状态由 Valid 转变为 Invalid 状态。

4.3.3 基于重命名的标记解耦合的一级缓存的开销

本文对 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 分别进行了综合, 结果显示 OO-Decoupled-Tag Cache 的面积比 IO-Decoupled-Tag Cache 大 4.7%, 相对于整个图形处理器来说开销较低。总体上, OO-Decoupled-Tag Cache 虽然有更小的等待指令缓冲区, 但是会有额外的 10KB 的 SRAM 和 1200 字节的寄存器开销, 以及更高的组合逻辑的开销。

4.3.3.1 等待缓冲区的开销

由于 IO-Decoupled-Tag Cache 采用顺序提交的方式, 即使等待指令缓冲区中后执行的指令延迟较短, 其等待计数器先为 0, 也会被先执行的延迟长的指令所阻塞, 导致指令的实际延迟常常取决于最大访存延迟。因此, IO-Decoupled-Tag Cache 如果想完全掩盖访存延迟, 必须使等待指令缓冲区中的指令发射的缺失请求数超过最大访存延迟。而 OO-Decoupled-Tag Cache 乱序提交指令, 延迟短的指令的等待计数器为 0 后, 可以立刻从等待缓冲区取出。因此, OO-Decoupled-Tag Cache 如果想完全掩盖访存延迟, 只需要使等待指令缓冲区的指令发射的缺失请求数超过平均访存延迟。由此, 本文设计的 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 的等待指令缓冲区分别最多可以容纳 256 和 128 条指令。图形处理器的一级缓存除了会执行向量访存指令以外, 还要执行纹理采样指令, 所以等待指令缓冲区除了需要存储缓存行索引、地址偏移等信息, 还要存储纹理过滤的配置信息。根据本文的图形处理器的设计, 等待指令缓冲区的数据宽度大约为 100 字节, 所以 OO-Decoupled-Tag Cache 的等待指令缓冲区的 SRAM 开销比 IO-Decoupled-Tag Cache 小 12.5KB 左右。

4.3.3.2 合并掩码的开销

为了能将一个 warp 的 32 个线程的请求在理想情况下合并为一个缓存行请求, 图形处理器的缓存行大小通常为 128 字节。假设缓存物理空间大小, 即缓存行阵列的大小, 为 $S\text{KB}$, 那么缓存行阵列就有 $8 * S$ 个缓存行, 总共有 $8 * S$ 个合并掩码。本文实现的 OO-Decoupled-Tag Cache 的等待指令缓冲区可以容纳 128 条指令, 所以合并掩码的宽度为 128 位, 那么总的合并掩码的开销为 $8 * S * 128\text{bits} = S/8\text{KB}$ 。此外, 由于标记阵列被分为 4 个分区以允许最多同时处理 4 个请求, 所以合并掩码也需要被划分为 4 个分区, 每个分区的大小为 $S/32\text{KB}$ 。本文实现的 OO-Decoupled-Tag Cache 使用了 80KB 的缓存行阵列, 所以合并掩码的存储开销为 4 个 2.5KB 的 SRAM。

4.3.3.3 寄存器的开销

OO-Decoupled-Tag Cache 增加的寄存器开销包括状态寄存器, Tag_valid 状态位和引用计数器。OO-Decoupled-Tag Cache 的缓存行状态除了 Invalid 和 Valid, 还增加了 Pending 状态。而且, 每个缓存行需要 1 位的 Tag_valid 寄存器。对于引用计数器, 在极限情况下, 等待指令缓冲区中的每条指令的每个线程都引用了同一缓存行, 那么引用计数最大为 $128 \times 32 = 4096$, 所以每个缓存行需要 13 位的引用计数器。综上, OO-Decoupled-Tag Cache 的每个缓存行需要额外的 15 位寄存器开销。本文实现的 OO-Decoupled-Tag Cache 使用了 80KB 的缓存行阵列, 所以额外的寄存器总开销为 1200 字节。

4.4 基于重命名的标记解耦合的一级缓存与缓存旁路的兼容性

Decoupled-Tag Cache 改进了图形处理器的一级缓存, 目前图形处理器和一级缓存相关的常用的访存优化技术是缓存旁路。在传统一级缓存上的缓存旁路技术的具体实现为: 将指令发出的访存请求绕过一级缓存, 返回的响应只会用来更新等待指令缓冲区中的指令结果, 而不会被写入到缓存中。然而, Decoupled-Tag Cache 的等待指令缓冲区不会为指令的结果预留空间, 所以传统一级缓存上的缓存旁路实现不完全适用于 Decoupled-Tag Cache。Decoupled-Tag Cache 以其他的方式来实现缓存旁路, 而且相比传统一级缓存的缓存旁路实现更简单, 具有更低的面积开销。

Decoupled-Tag Cache 上的缓存旁路的具体实现为: 如果某个请求想要绕过一级缓存, 那么需要为该请求只分配缓存行, 不分配标记。因此, 由于该请求没有更新标记, 其行为对于其他请求来说是透明的, 相当于绕过了一级缓存。由于不会将已有的标记替换, 就不会无效有用的缓存行的标记, 与缓存旁路技术一样缓解了缓存争用问题。

传统一级缓存的缓存旁路实现需要增加一条返回的缓存行到等待指令缓冲区的数据通路。Decoupled-Tag Cache 的缓存旁路实现不需要增加新的数据路径, 只需要不为绕过的请求分配标记即可实现。因此, Decoupled-Tag Cache 不仅兼容缓存旁路技术, 而且实现的面积开销更低。

4.5 实验结果分析

4.5.1 实验评估方法

本文的实验平台为 Unified-Cache GPGPU。表4.1展示了用于实验评估的通用计算应用和图形渲染应用测试集。本文使用 Polybench^[76] 和 Parboid^[77] 的通用计算应用, glmark2^[78] 的图形渲染应用作为基准测试程序。本文通过仿真实验进行了性能分析和架构设计空间探索。

4.5.2 性能分析

图4.5展示了在 16KB 的缓存大小配置下 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 在不同应用上相对于传统一级缓存的加速比, GM-ALL 表示所有应用的几何平均值, GM-CE 表示缓存高效型应用的几何平均值。IO-Decoupled-Tag Cache 相比传统一级缓存在所有应用上平均提高了 22% 的性能, 在缓存高效型应用上平均提高了 3% 的性能。OO-Decoupled-Tag Cache 相比传统一级缓存在所有应用上平均提高了 30% 的性能, 在缓存高效型应用上平均提高了 12% 的性能。可以发现, Decoupled-Tag Cache 在缓存高效型应用上性能提升不显著, 但是在缓存低效型应用和图形渲染应用上有着显著的性能提升, 在 spmv、2mm、jelly、cat 和 cube 上有着超过 50% 的性能提升, 在 spmv 上甚至有着超过一倍的性能提升。这是因为缓存低效型应用和图形渲染应用的主要性能瓶颈为访存带宽, 而 Decoupled-Tag Cache 通过消除结构所导致的访存阻塞, 增多了并发访存请求数, 提高了掩盖访存延迟的能力, 增大了访存带宽的利用率, 进而有效提高了缓存低效型应用和图形渲染应用的性能。

OO-Decoupled-Tag Cache 在通用计算应用上的性能提升相比图形渲染应用更显著。图形渲染应用的执行存在序的要求, 而通用计算应用的内核对线程的完成顺序几乎没有要求。因此, 允许乱序完成访存指令的 OO-Decoupled-Tag Cache 相比只能顺序完成访存指令的 IO-Decoupled-Tag Cache 更适合通用计算应用的执行。而且, 图形渲染应用通常只有内存读指令, 而通用计算应用经常存在内存写指令和原子指令。内存写指令和原子指令的平均延迟远大于内存读指令, 使得 IO-Decoupled-Tag Cache 上的低延迟的内存读指令经常被长延迟的内存写和原子指令所阻塞, 导致平均访存延迟增大。而在 OO-Decoupled-Tag Cache 上, 乱序完成的特点允许后来的低延迟的内存读指令先提交, 不会被阻塞。综合以上两点原因, OO-Decoupled-Tag Cache 在通用计算应用上比图形渲染应用有着更高的性能提升。

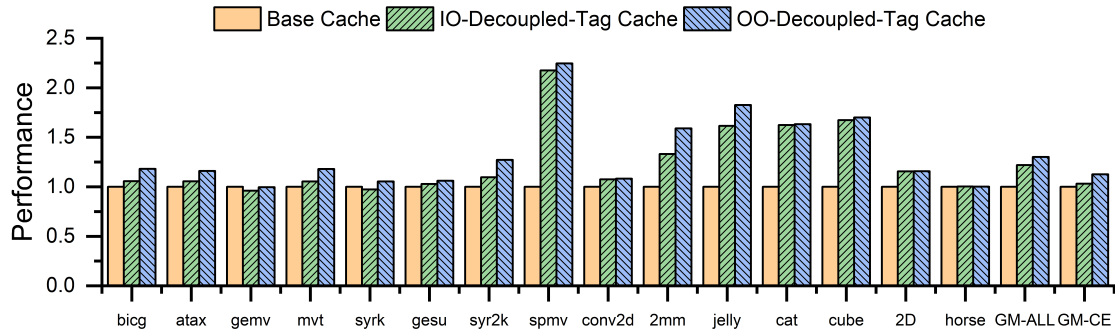


图 4.5 Decoupled-Tag Cache 和传统一级缓存的性能比较

Figure 4.5 Performance comparison of Decoupled-Tag Cache and traditional L1 Cache

4.5.3 面积分析

本文对 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 进行了综合, 结果显示 OO-Decoupled-Tag Cache 的面积比 IO-Decoupled-Tag Cache 大 4.7%。图4.6展示了 IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 内部各个模块的面积占比, 其中访存合并单元 (Memory Coalesce) 和缓存行阵列 (Cacheline Array) 占据了最多的面积。访存合并单元的开销主要来自于组合逻辑, 不仅需要各个线程的请求地址进行比较, 而且需要将请求根据地址发送到不同的标记阵列的分区上。缓存行阵列的开销主要来自于片上存储。由于新增的合并掩码和额外的寄存器的开销, OO-Decoupled-Tag Cache 的标记阵列的面积比例超过 IO-Decoupled-Tag Cache。而 OO-Decoupled-Tag Cache 的等待指令缓冲区的片上存储开销更小, 所占的面积比例较低。总体上, OO-Decoupled-Tag Cache 比 IO-Decoupled-Tag Cache 的面积略大。对于主要面向图形渲染应用的场景, 通用图形处理器使用 IO-Decoupled-Tag Cache 可以以较低的面积开销实现较高的性能。对于主要面向通用计算应用的场景, 通用图形处理器使用 OO-Decoupled-Tag Cache 可以以略高的面积开销实现更高的性能。对于兼具通用计算和图形渲染应用的场景, 通用图形处理器使用 OO-Decoupled-Tag Cache 可以实现更高的性能面积比。

4.5.4 平均并发缺失请求数

为了进一步分析 Decoupled-Tag Cache 性能提升的原因, 本文测量了 Decoupled-Tag Cache 和传统一级缓存在不同应用上的平均并发缺失请求数。图4.7展示了实验结果, AVG 表示所有应用的平均值, IO-Decoupled-Tag Cache 的平均并发缺失请求数相比传统一级缓存提高了 81.2%, OO-Decoupled-Tag Cache 的平均并发缺失请求数相比 IO-Decoupled-Tag Cache 提高了 7.82%。这与 Decoupled-Tag Cache

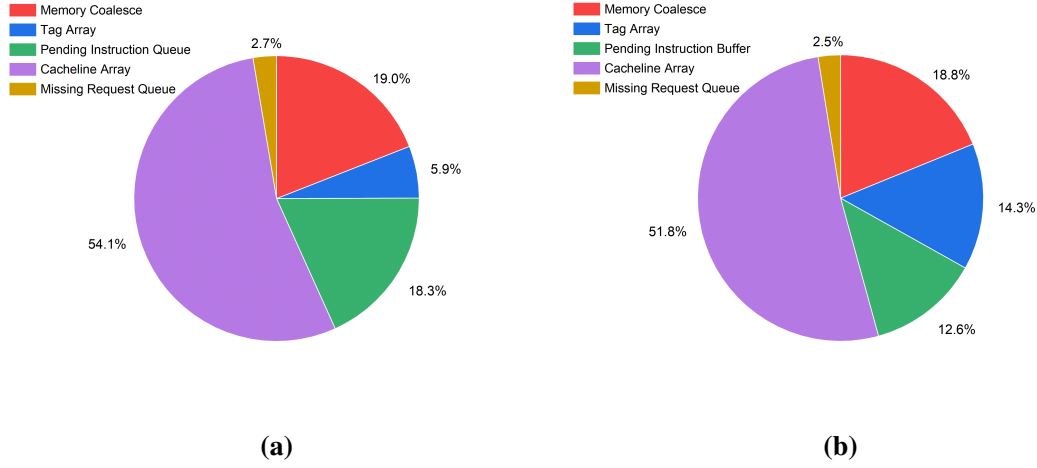


图 4.6 Decoupled-Tag Cache 的各模块的面积占比 (a) IO-Decoupled-Tag Cache 的各模块的面积占比, (b) OO-Decoupled-Tag Cache 的各模块的面积占比

Figure 4.6 Module area breakdown of Decoupled-Tag Cache (a) Module area breakdown of IO-Decoupled-Tag Cache, (b) Module area breakdown of OO-Decoupled-Tag Cache

的性能提升相对应, 由于平均并发请求数增大, 提高了掩盖访存延迟的能力, 提高了片上互连网络、二级缓存和内存的利用率, 即提高了总的访存带宽利用率, 进而提升了性能。此外, 从图4.7可以发现, spmv、2mm、jelly、cat、cube 和 2D 的平均并发缺失请求数在 Decoupled-Tag Cache 上有着明显的提升。这些应用的访存行为特征具有较高的空间局部性, 会将其一条指令的 32 个线程的请求合并为少量的请求, 传统一级缓存的较小的等待指令缓冲区会严重限制其并发的缺失请求数。而 Decoupled-Tag Cache 消除该瓶颈, 使其并发的缺失请求数得到显著提升, 性能也随之显著提升。

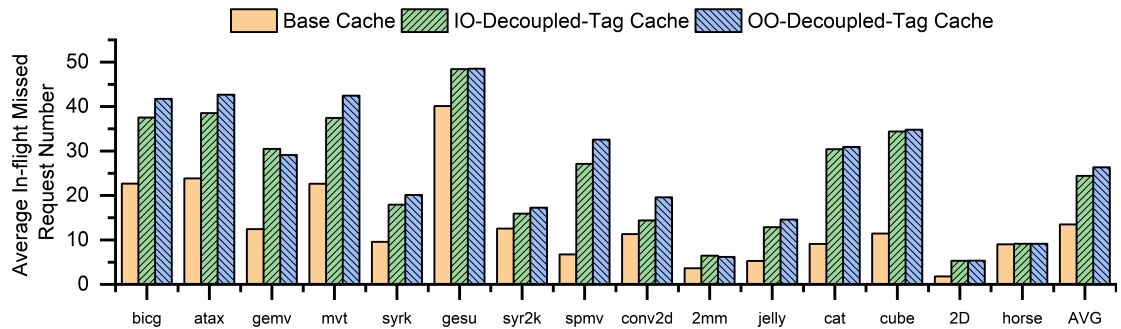


图 4.7 Decoupled-Tag Cache 和传统一级缓存的平均并发缺失请求数的比较

Figure 4.7 Comparison of average number of concurrent missing requests of Decoupled-Tag Cache and traditional L1 cache

4.5.5 逻辑缓存空间大小与性能的关系

Decoupled-Tag Cache 与传统一级缓存不同，由于将标记和缓存行的映射解耦合，存在由标记阵列组成的逻辑缓存空间和由缓存行组成的物理缓存空间。图4.8展示了在不同应用上，IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag 在 16KB、32KB 和 64KB 的逻辑缓存大小下的性能比较，GM-ALL 表示所有应用的几何平均值，GM-CE 表示缓存高效型应用的几何平均值。

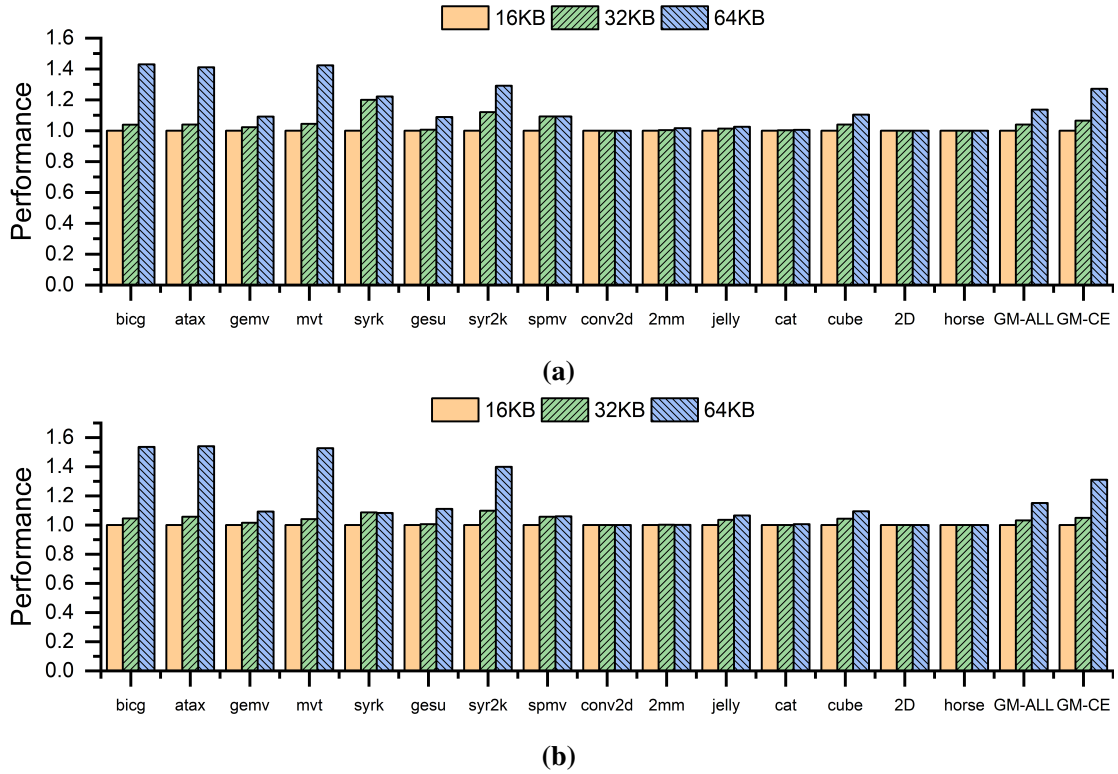


图 4.8 不同缓存大小的 Decoupled-Tag Cache 的性能比较 (a) 不同缓存大小的 IO-Decoupled-Tag Cache 的性能比较, (b) 不同缓存大小的 OO-Decoupled-Tag Cache 的性能比较

Figure 4.8 Performance comparison of Decoupled-Tag Cache with varying cache sizes (a) Performance comparison of IO-Decoupled-Tag Cache with varying cache sizes, (b) Performance comparison of OO-Decoupled-Tag Cache with varying cache sizes

在 IO-Decoupled-Tag Cache 上, 32KB 和 64KB 的缓存相比 16KB 在缓存高效型应用上分别有 7% 和 27% 的性能提升, 在 OO-Decoupled-Tag Cache 上, 32KB 和 64KB 的缓存相比 16KB 在缓存高效型应用上分别有 5% 和 31% 的性能提升。而如第三章所述, 在传统一级缓存上, 32KB 和 64KB 的缓存相比 16KB 在缓存高效型应用上分别有着 10.1% 和 35.4% 的性能提升。可以发现, Decoupled-Tag Cache 与传统一级缓存相比, 性能对缓存大小更不敏感。这是由于更大缓存带来的访存延迟降低的好处被 Decoupled-Tag Cache 掩盖访存延迟的能力抵消了一

部分。缓存增大可以降低缓存的缺失率，降低平均访存延迟。而 Decoupled-Tag Cache 相比传统一级缓存可以发射更多的缺失访存请求，能掩盖更多的访存延迟，使 Decoupled-Tag Cache 的缓存增大带来的效益没有传统一级缓存显著。

4.5.6 物理缓存空间大小与性能的关系

本文测试了在 16KB 的逻辑缓存大小的情况下，IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 在通用计算应用上使用不同物理缓存大小时的性能，并将结果归一化到 32KB 物理缓存大小的 IO-Decoupled-Tag Cache 的性能上。图4.9展示了实验结果。每次发生缺失请求，就需要在物理缓存空间中为其分配一个缓存行，图形渲染应用的访存行为通常具有较高的空间局部性，物理缓存空间通常不是其性能瓶颈，所以本文只对通用计算应用进行了测试。

首先从图4.9可以发现，IO-Decoupled-Tag Cache 比起 OO-Decoupled-Tag Cache 对物理缓存空间大小更敏感，IO-Decoupled-Tag Cache 在 24KB 上有着明显性能下降，而 OO-Decoupled-Tag Cache 则是在 16.5KB 上才有着明显的性能下降。之所以会出现这种区别，是因为两者释放缓存行的机制不同。IO-Decoupled-Tag Cache 是被动地释放缓存行，当标记被指令替换后且当该指令完成后，被替换标记的缓存行才会被释放。而 OO-Decoupled-Tag Cache 是主动地释放缓存行，指令完成后就会将缓存行的引用计数器递减，如果某个缓存行的引用计数器为 0 且没有标记映射，那么就会释放缓存行。主动释放相比被动释放速度更快，可以更早地将缓存行释放。因此，在使用相同物理缓存空间大小的情况下，OO-Decoupled-Tag Cache 比 IO-Decoupled-Tag Cache 总是有更多的可用缓存行，使得 OO-Decoupled-Tag Cache 的性能对物理缓存空间大小更不敏感。

其次，从图4.9可以发现，16.5KB 的 IO-Decoupled-Tag Cache 的性能为 0，这是因为 16.5KB 的物理缓存空间的 IO-Decoupled-Tag Cache 会发生死锁。由于 IO-Decoupled-Tag Cache 是被动地释放缓存行，当剩余的空间不足以容纳一条指令所需的所有缓存行时，该指令无法继续执行同时也无法释放被其替换的标记的缓存行，那么就会导致死锁。而 OO-Decoupled-Tag Cache 可以主动地释放缓存行，所以可以避免这种情况。

4.5.7 等待指令缓冲区大小与性能的关系

本文测试了在 16KB 的逻辑缓存大小和 32KB 的物理缓存大小的情况下，IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache 在所有应用上使用不同等待指令缓冲区大小时的性能，并将结果归一化到 128 项的等待指令缓冲区的 IO-

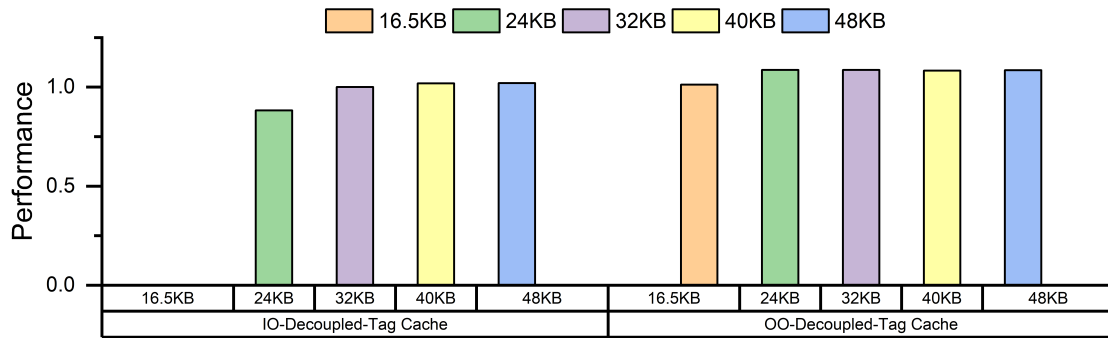


图 4.9 通用计算应用上不同物理缓存大小的 Decoupled-Tag Cache 的性能比较

Figure 4.9 Performance comparison of Decoupled-Tag Cache with varying physical cache sizes on general-purpose computing applications

Decoupled-Tag Cache 的性能上。图4.10展示了实验结果。实验结果显示，在所有应用上，相比 128 项的等待指令缓冲区，32 项的 IO-Decoupled-Tag Cache 的性能下降了 16.2%，32 项的 OO-Decoupled-Tag Cache 只下降了 2.06%，这说明 OO-Decoupled-Tag Cache 的性能比起 IO-Decoupled-Tag Cache 对等待指令缓冲区的大小更不敏感。这是由于 OO-Decoupled-Tag Cache 和 IO-Decoupled-Tag Cache 提交指令的次序不同所导致的。由于 IO-Decoupled-Tag Cache 顺序提交指令，即使后续的指令已完成，也无法从等待指令指令缓冲区中弹出，直到前面所有的指令都弹出后，该指令才会释放所占用的等待指令缓冲区项。而 OO-Decoupled-Tag Cache 允许乱序提交指令，当前指令完成后，可以立刻从等待指令缓冲区中弹出，并立即释放其占用的等待指令缓冲区项。因此，在相同情况下，OO-Decoupled-Tag Cache 相比 IO-Decoupled-Tag Cache 总是有更多的空闲的等待指令缓冲区项，使得 OO-Decoupled-Tag Cache 相比 IO-Decoupled-Tag Cache 对等待指令缓冲区的大小更不敏感。

此外，本文发现，缓存低效应用和图形渲染应用的性能对等待指令缓冲区大小的变化更加敏感。与 128 项的相比，在 32 项的 IO-Decoupled-Tag Cache 上，spmv 的性能下降了 32.7%，jelly 的性能下降了 35.6%。这是因为这些应用具有较高的空间局部性，使得它们的访存指令的 32 个线程的访存请求经常被访存单元合并为一至两个请求。当等待指令缓冲区较小时，即使将等待指令缓冲区占满，发射的缺失请求数也不足以完全掩盖访存延迟，导致访存带宽利用率低。因此，对于具有高空间局部性的应用，减小等待指令缓冲区的大小会造成严重的性能降级。

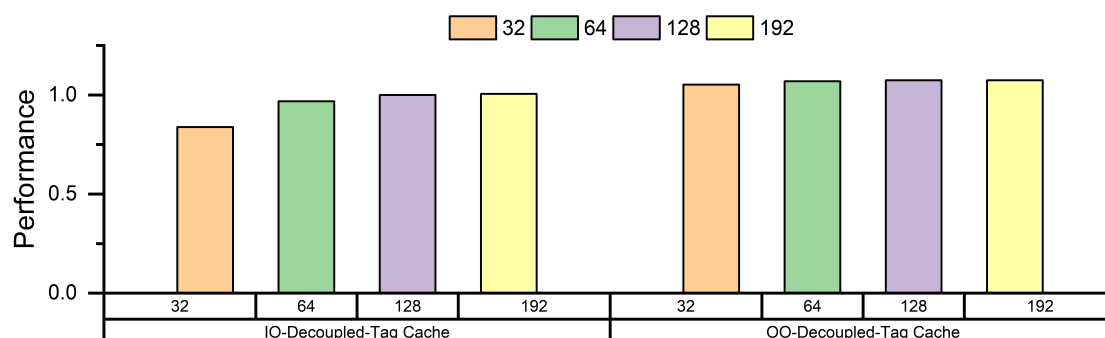


图 4.10 不同等指令缓冲区大小的基于重命名的标记解耦合的一级缓存的性能比较

Figure 4.10 Performance comparison of Decoupled-Tag Cache with varying pending instruction buffer sizes

4.6 本章总结

图形处理器应用通常具有高访存吞吐量的特点。然而，传统一级缓存架构不适应高吞吐量的访存需求。传统一级缓存为了最大程度降低访存延迟，采用了紧耦合映射的组相联、缺失状态保持寄存器和寄存器设计的等待指令缓冲区。这些结构会限制最大并发缺失请求数，降低掩盖访存延迟的能力，导致内存带宽利用率低。因此，本文提出了基于重命名的标记解耦合的一级缓存（Decoupled-Tag Cache）。首先，Decoupled-Tag Cache 采用了重命名的方法，将标记组成的逻辑地址空间映射到缓存行组成的物理地址空间，消除了标记和缓存行的组相联紧耦合映射所导致的访存阻塞。其次，Decoupled-Tag Cache 没有使用缺失状态保持寄存器，而是通过访存命中的行为来合并缺失请求，消除了缺失状态保持寄存器所导致的访存阻塞。最后，Decoupled-Tag Cache 使用 SRAM 来设计等待指令缓冲区，支持数百条访存指令同时执行，消除了等待指令寄存器堆所导致的访存阻塞。在此基础上，本文分别设计了顺序提交和乱序提交的基于重命名的标记解耦合的一级缓存，顺序的 Decoupled-Tag Cache 适应于图形渲染应用按序处理几何图元的特征，乱序的 Decoupled-Tag Cache 虽然有着更高的面积开销，但是在通用计算应用上有着更显著的性能提升。而且，本文发现在 Decoupled-Tag Cache 上可以几乎没有开销地实现缓存旁路技术，与目前的先进访存优化技术相兼容。经过实验评估，与传统一级缓存相比，在图形渲染应用上，顺序的 Decoupled-Tag Cache 可以提高超过 50% 的性能，在所有应用上，顺序的 Decoupled-Tag Cache 平均提高了 22.0% 的性能，乱序的 Decoupled-Tag Cache 平均提高了 30.0% 的性能。

第 5 章 流量感知的可重配置热点缓存设计

本章介绍了流量感知的可重配置热点缓存 (Re-Hot Cache) 的设计动机、设计理念、具体实现和实验评估。Re-Hot Cache 的设计动机有两点：1) 高吞吐量的缺失请求会造成片上互连网络、二级缓存和内存带宽的争用，导致缺失请求的平均访存延迟增加。2) 图形固定功能流水线的像素着色重排序缓冲区大小随着流多处理器数量增加而增大，而该重排序缓冲区的片上存储在通用计算时会被闲置。Re-Hot Cache 的设计理念是将该闲置的图形固定功能流水线的片上存储在执行通用计算应用时重配置为二级缓存的热点缓存，一级缓存的缺失请求会先访问该热点缓存，如果缺失才会再访问二级缓存，从而缓解二级缓存争用问题，并且降低平均访存延迟。Re-Hot Cache 的具体实现包含流量感知可重配置热点缓存的架构和执行流程。为了评估和探索 Re-Hot Cache 架构，本文在 Unified-Cache GPGPU 上进行了 Re-Hot Cache 和基准架构的性能对比实验，以及 Re-Hot Cache 架构的设计空间探索实验。

5.1 设计动机

5.1.1 闲置的固定功能流水线的片上存储

图5.1展示了图形处理器的图元引擎的架构。图元引擎由光栅化器和重排序缓冲区 (Reorder Buffer, ROB) 组成。光栅化器将几何图元 (三角形, 点和线) 转换为覆盖屏幕的 2×2 的像素块, 然后将每 8 个像素块打包为一个像素着色器的 warp, 并将 warp 发射到流多处理器上执行。根据 OpenGL 和 DirectX 标准, 在相同像素上的输出混合操作的执行顺序必须与程序指定的几何图元顺序相同^[43,75]。然而, 流多处理器会乱序完成 warp。因此, 需要使用重排序缓冲区来保证几何图元的顺序。重排序缓冲区需要跟踪所有正在执行的 warp 的状态, 该状态包括像素块的屏幕坐标 (32 字节), 面方向 (1 字节), 深度 (128 字节), 亚采样点覆盖 (16 字节) 和层 (12 字节) 等信息。每个流多处理器最多可以同时执行 64 个 warp, 所以每个流处理器组在重排序缓冲区至少需要能存储 64 个 warp 状态的空间, 即至少 64×189 字节的片上存储空间。虽然重排序缓冲区拥有可观的片上存储开销, 开销会随着流多处理器数量的增加而线性增大, 但是在图形处理器执行通用计算时无法利用该片上存储, 导致片上存储的浪费。

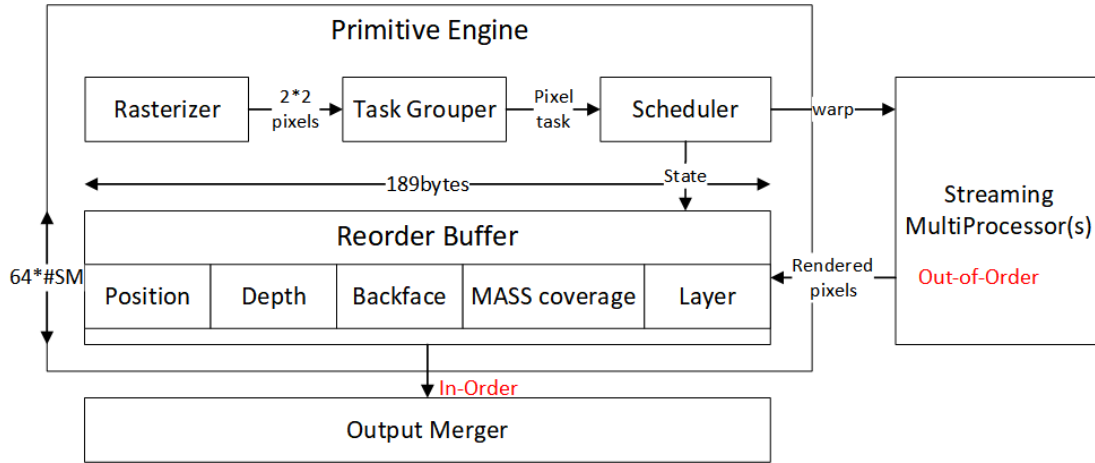


图 5.1 图元引擎架构

Figure 5.1 Primitive engine architecture

5.1.2 二级缓存的缓存争用

为了分析二级缓存争用问题，本文在使用 16KB 的 OO-Decoupled-Tag Cache 的 Unified-Cache GPGPU 上进行了实验，测试了不同二级缓存大小下应用的加速比。表5.1展示了用于实验的通用计算应用，由于驱动和编译器尚不完善，本文使用来自 Polybench^[76] 和 Parboid^[77] 的部分通用计算应用作为基准测试程序。基准测试程序的规模保证图形处理器的执行车道可以处于满负载的状态。为了使评估更加全面，本文使用了二级缓存高效型和低效型应用，以及计算密集型应用作为基准测试程序。

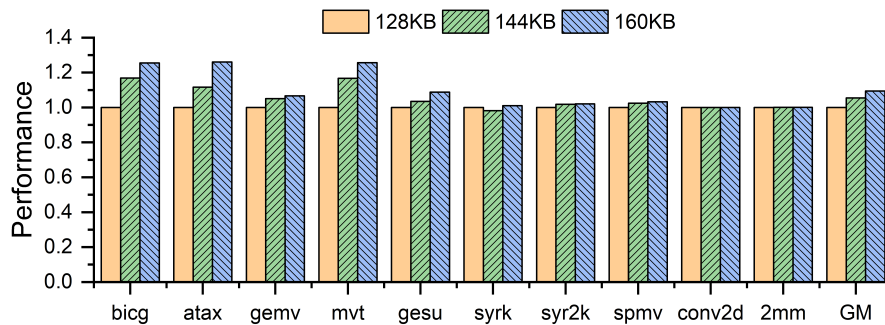


图 5.2 不同二级缓存大小的性能比较

Figure 5.2 Performance comparison of varying L2 cache sizes

本文测量了应用在 128KB、144KB 和 160KB 的二级缓存上的性能，图5.2展示了实验结果，GM 表示所有应用的几何平均值。实验结果显示，随着二级缓存增大，二级缓存高效型应用的性能明显提高，而二级缓存低效型应用和计算密集型应用的性能略微提高或者保持不变。二级缓存高效型应用的工作集无法被二

表 5.1 评估图形处理器的基准测试应用

Table 5.1 GPU applications for benchmark

L2Cache-Efficient Applications			
App	Description	App	Description
bicg ^[76]	BiCGStab Linear Solver	atax ^[76]	Matrix Transpose-Vector Mul.
gemv ^[76]	Vector Mul. and Matrix Add.	mvt ^[76]	Matrix Vector Product-Transpose
gesu ^[76]	Scalar, Vector and Matrix Mul.		
L2Cache-Inefficient Applications			
App	Description	App	Description
syrk ^[76]	Symm. rank-k operations	syr2k ^[76]	Symm. rank2k operations
spmv ^[77]	Sparse Matrix-Dense Vector Mul.	2mm ^[76]	2 Matrix Mul. and Matrix Add.
Compute-Intensive Applications			
App	Description	App	Description
conv2d ^[76]	2D Convolution		

级缓存全部容纳,经常发生的二级缓存缺失使内存带宽处于过载的状态,导致平均访存延迟增加。当二级缓存增大时,缺失率的下降会减轻内存带宽的负载,从而降低平均访存延迟并提高性能。如果能将应用的一部分工作集交给其他缓存存储,就可以减轻二级缓存高效型应用在二级缓存上的负载,使二级缓存能够保存原本会被替换出的工作集,减少总的缺失率来降低平均访存延迟并提高性能。

为了进一步分析二级缓存争用问题,本文在 128KB 的二级缓存上进行实验,测量了二级缓存行在生命周期内的最高访问频率的分布,图5.3展示了实验结果,AVG 表示所有应用的平均值。其中,平均 64.3% 的二级缓存行只会被访问一次,平均 17.9% 会被访问 2 至 4 次,平均 3.7% 会被访问 5 至 7 次,而剩余的平均 14.1% 的二级缓存行会被访问 8 次以上。大多数缓存行之所以只会被访问一次,一方面是因为存在流式的数据,另一方面是因为由于二级缓存争用问题,导致有用的缓存行在被再次利用之前替换出二级缓存,可以看到 syrk 和 syr2k 的二级缓存行平均访问频率较高,这是因为它们的工作集较小,使二级缓存争用问题较弱。二级缓存中存在部分高访问频率的缓存行,说明这些缓存行经常被一级缓存的缺失请求所访问。如果存在一个离流多处理器更近的热点缓存来存储这些高访问频率的缓存行,不仅可以降低访问这些高访问频率请求的平均访存延迟,而且可以缓解二级缓存争用问题。

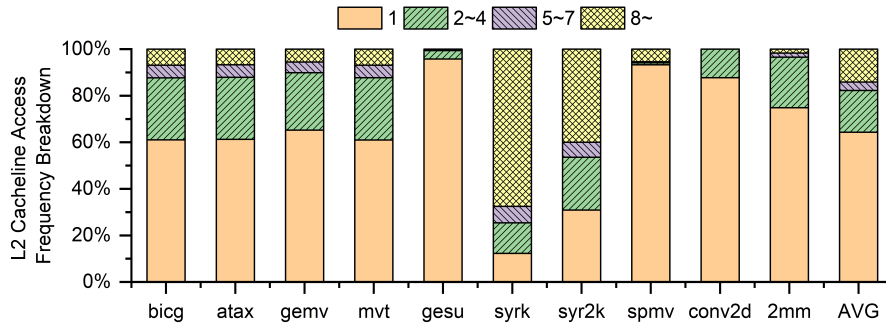


图 5.3 二级缓存行访问频率的分布

Figure 5.3 Distribution of L2 cache lines access frequency

5.2 流量感知的可重配置热点缓存的实现

为了缓解二级缓存争用问题，并且利用通用计算时闲置的固定功能流水线的片上存储，本文将像素着色重排序缓冲区在执行通用计算时重配置为热点缓存。该热点缓存只存储二级缓存中访问频率高的缓存行来过滤缺失请求对二级缓存的访问。本节主要介绍了 Re-Hot Cache 的架构和执行流程。

5.2.1 流量感知的可重配置热点缓存架构

图5.4展示了 Re-Hot Cache 的架构，>? 表示 *Ref_cnt* 是否大于 *threshold*。本文从内存层次、数据类型和管理策略三个方面介绍流量感知的可重配置热点缓存的架构。

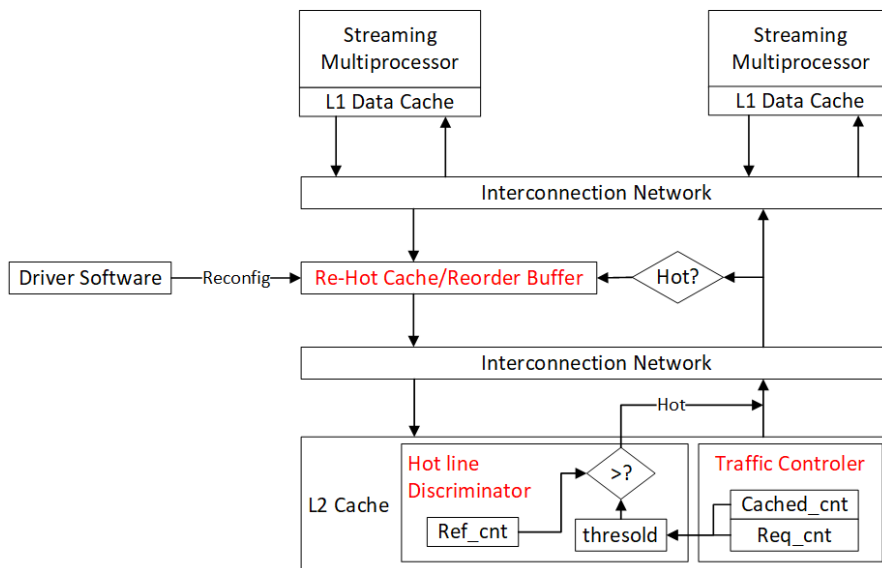


图 5.4 流量感知的可重配置热点缓存架构

Figure 5.4 Re-Hot Cache architecture

5.2.1.1 流量感知的可重配置热点缓存的内存层次

如图3.2所示,图形处理器通过片上互连网络将各个模块连接,片上互连网络上主要包含六个节点,分别是两个流多处理器组,几何引擎,图元引擎,输出混合器和二级缓存。各个节点的内存请求会通过片上互连网络发送到二级缓存,而二级缓存会将缓存行再通过片上互连网络发送回各个节点。由于固定功能流水线的访存并行度小于流多处理器,固定功能流水线的性能对于访存延迟更加敏感。因此,本文所设计的图形处理器的固定功能流水线(包括几何引擎、图元引擎和输出混合器)在片上互连网络上的位置相比流多处理器更接近二级缓存,位于流多处理器和二级缓存之间。流多处理器发出的访存请求会路过固定功能流水线,同样二级缓存返回流多处理器的缓存行也会路过固定功能流水线。由于 Re-Hot Cache 来自于图元引擎内部的重排序缓冲区片上存储的重配置, Re-Hot Cache 在访存层次中会处于流多处理器的一级缓存和二级缓存之间。一级缓存的缺失请求会首先访问 Re-Hot Cache,如果在 Re-Hot Cache 中缺失才会再访问二级缓存。虽然 Re-Hot Cache 的标记检查会导致缺失请求的访存延迟多两个周期,但相比于原本的数百周期延迟几乎可以忽略不计。

5.2.1.2 流量感知的可重配置热点缓存的存储数据类型

流多处理器内的一级缓存有指令缓存、常量缓存和数据缓存,其中指令缓存和常量缓存的容量通常足以存储应用的所有指令和常量数据,指令缓存和常量缓存的大多数请求缺失属于冷缺失。流多处理器高吞吐量的内存访问大多数来自于数据缓存。热点缓存策略对指令和常量访存的性能提升有限。因此, Re-Hot Cache 只会存储数据缓存的缺失请求所访问的缓存行,以避免指令和常量数据对热点缓存造成污染。

5.2.1.3 流量感知的可重配置热点缓存的管理策略

由于 Re-Hot Cache 来自于图元引擎内部的重排序缓冲区片上存储的重配置,且图元引擎只能与其所属 GPC 内的流多处理器所共享, Re-Hot Cache 也只会为所属 GPC 内的流多处理器所共享。根据前文所述,每存在一个流多处理器,重排序缓冲区就至少需要 64×189 字节的片上存储开销。本文所设计的图形处理器存在两个流多处理器,所以至少需要 128×189 字节的片上存储开销。

本文使用了两个深度为 128 的宽度为 128 字节的 SRAM 来作为可重配置的片上存储。在执行图形渲染应用时,该片上存储配置为一个深度为 128 的宽度为 256 字节的重排序缓冲区,最多可以跟踪 128 个 warp 的状态;在执行通用计算

应用时，该片上存储配置为一个深度为 256 的宽度为 128 字节的缓存行阵列使用，最多可以存储 256 个 128 字节的缓存行，共 32KB 大小的缓存。虽然会引入少量的额外片上存储开销，但是可以完全利用原本闲置的片上存储。

现代图形处理器通常采用了非包含非互斥的缓存层次架构^[2,3,5]。假设对该片上存储也使用非包含非互斥的策略，即仅仅作为一个中间缓存层级 (L1.5 Cache)，不仅无法提升性能，而且会导致性能降低。这是因为对于非包含非互斥的缓存包容性策略，返回一级缓存的缓存行，同样也会返回 L1.5 Cache。然而，L1.5 Cache 的大小只略大于或者小于一级缓存，一级缓存因容量不足导致的缺失大概率也会在 L1.5 Cache 中缺失，使 L1.5 Cache 失去缓存的作用。因此，本文将该片上存储作为一个热点缓存 (Re-Hot Cache) 来使用。Re-Hot Cache 只存储二级缓存中最常被访问的缓存行。既可以降低一级缓存常访问的缓存行的访问延迟，也可以降低二级缓存的负载。

5.2.2 流量感知的可重配置热点缓存的执行流程

图5.5展示了 Re-Hot Cache 的执行流程。当驱动发现图形处理器的某个 GPC 的流多处理器只执行通用计算应用时，就可以将该 GPC 的图元引擎的重排序缓冲区重配置为 Re-Hot Cache。一级缓存的缺失请求通过片上互连网络发送到 Re-Hot Cache，如果请求在 Re-Hot Cache 中命中，那么直接返回命中的缓存行，否则再将请求发送到二级缓存。Re-Hot Cache 不会记录任何缺失请求的状态，没有使用缺失状态保持寄存器。

在一级缓存和 Re-Hot Cache 中都缺失的请求在二级缓存中处理，如果该请求所访问的缓存行被评估为热点缓存行，那么将返回的缓存行的状态设为 Hot。当该缓存行沿着片上互连网络返回一级缓存时，会经过 Re-Hot Cache。Re-Hot Cache 通过监听总线上的访存请求的响应，如果响应的缓存行的状态为 Hot，就将该缓存行存入 Re-Hot Cache 中。

为了评估缓存行是否为热点，二级缓存需要监控每个缓存行的访问频率并设置一个访问频率的阈值。如何设置评估热点缓存的访问频率的阈值是设计的一个难点。如果阈值过低，过多的缓存行会被写入 Re-Hot Cache，导致 Re-Hot Cache 发生缓存争用。如果阈值过高，被写入 Re-Hot Cache 的热点缓存行数量过少，导致 Re-Hot Cache 效率低。而且，合适的访问频率阈值不仅取决于不同应用的特征，而且取决于同一应用执行周期内的不同时间段。因此，本文设计了流量感知的动态热点缓存行评估方法。

5.2.2.1 流量感知的动态热点缓存行评估方法

动态评估热点缓存行，是为了在不同应用以及同一应用的不同执行阶段都可以向 Re-Hot Cache 写入适量的热点缓存行，以达到避免缓存争用和提高缓存效率的最佳权衡点。Re-Hot Cache 会监测二级缓存总的请求数量和写入 Re-Hot Cache 的缓存行数量。Re-Hot Cache 的作用是作为二级缓存的补充容量来承担部分二级缓存的负载，因此二级缓存总的请求数量和写入 Re-Hot Cache 的缓存行数量的比例应该接近二级缓存与 Re-Hot Cache 大小的比例。假设二级缓存的大小为 S ，Re-Hot Cache 的大小为 $\alpha S (0 < \alpha < 1)$ ，二级缓存总的请求数量为 req_cnt ，被写回 Re-Hot Cache 的缓存行数量为 $cached_cnt$ ，那么为了维持 Re-Hot Cache 的效率并避免缓存争用问题，需要保证： $\alpha - \delta < cached_cnt/req_cnt < \alpha + \delta$ 。其中 δ 是一个容忍度，只要偏差不超过容忍度都可以认为写入 Re-Hot Cache 的缓存行数量合适。

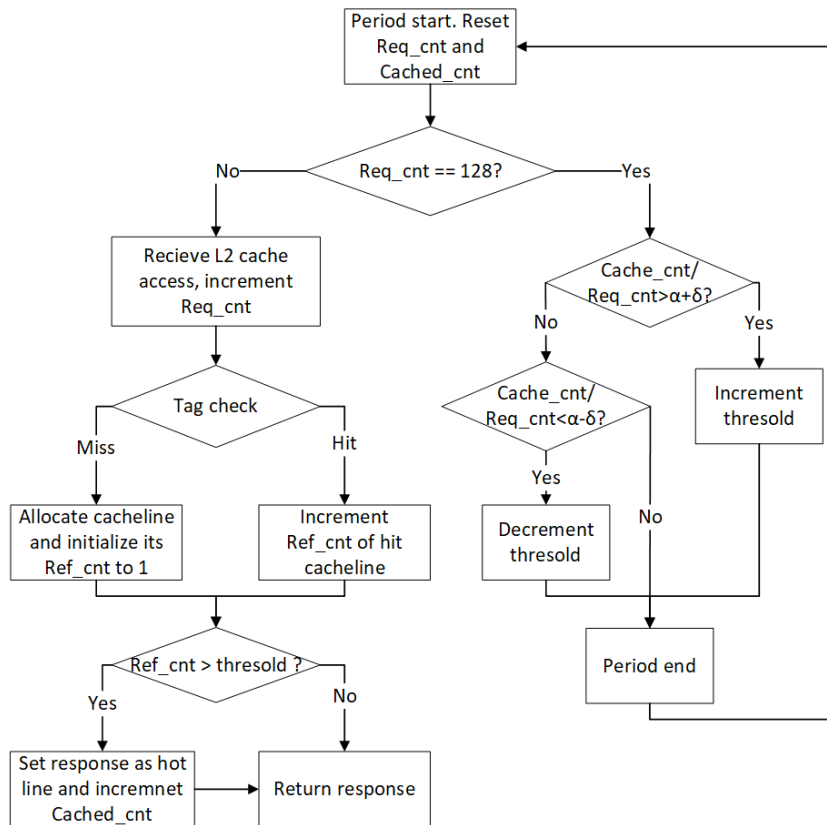


图 5.5 流量感知的可重配置热点缓存的执行流程

Figure 5.5 Re-Hot Cache workflow

为了统计二级缓存行的访问频率，Re-Hot Cache 为每个二级缓存行增加一个请求计数器。当访存请求在二级缓存中缺失并为其分配缓存行时，将被分配缓存

行的请求计数器初始化为 1。当访存请求在二级缓存命中时, 将命中缓存行的请求计数器的值递增。如果某个访存请求所命中的缓存行的请求计数器的值超过了阈值, 那么该缓存行就会被评估为热点缓存行。当该缓存行沿着片上互连网络返回一级缓存时会被顺带写入 Re-Hot Cache 中。阈值的大小决定了写回 Re-Hot Cache 的缓存行的流量。为了在通用计算应用的不同执行阶段动态地调整阈值大小, 二级缓存每收到 128 个请求后就会调整一次阈值。在收到 128 个请求之前, 二级缓存会累计总请求数 (req_cnt) 和写回 Re-Hot Cache 的缓存行数 ($cached_cnt$)。在收到 128 个请求后, 如果 $cached_cnt/req_cnt \geq \alpha + \delta$, 表明写回 Re-Hot Cache 的缓存行数量过多, 就将阈值递增; 如果 $cached_cnt/req_cnt \leq \alpha - \delta$, 表明写回 Re-Hot Cache 的缓存行数量过少, 就将阈值递减; 如果 $\alpha - \delta < cached_cnt/req_cnt < \alpha + \delta$, 表明写回 Re-Hot Cache 的缓存行数量适中, 阈值保持不变。

5.2.3 流量感知的可重配置热点缓存的硬件开销

流量感知的可重配置热点缓存的硬件开销包含 SRAM 的开销、寄存器开销和组合逻辑的开销。SRAM 的开销来自于增大的重排序缓冲区的片上存储, 将原本 128*189 字节的 SRAM 变为了两个 128*128 字节的 SRAM, 总的 SRAM 存储开销增大了约 8.4KB。寄存器的开销主要来自于为每个二级缓存行新增加的请求计数器。根据图 5.3, 4 位的计数器足以涵盖绝大多数应用的情况。本文的图形处理器使用了 128KB 的二级缓存, 共 1024 个缓存行, 请求计数器的寄存器的开销为 512 字节。加上两个 8 位的 req_cnt 和 $cache_cnt$ 计数器, 共 514 字节的寄存器开销。组合逻辑的开销主要来自于 Re-Hot Cache 的标记检查逻辑。总体上, Re-Hot Cache 以较低的硬件开销为通用计算应用增加了 32KB 的片上缓存。

5.3 实验结果分析

5.3.1 实验评估方法

本文在采用了 OO-Decoupled-Tag Cache 的 Unified-Cache GPGPU 上实现了 Re-Hot Cache。表 5.1 展示了用于实验评估的通用计算应用。本文使用 Polybench^[76] 和 Parboid^[77] 的部分通用计算应用作为基准测试程序。本文通过仿真实验进行了性能分析实验和架构设计空间探索实验。本文所使用 Unified-Cache GPGPU 使用了大小为 32KB 的 Re-Hot Cache 和 128KB 的二级缓存, 所以流量控制参数 α 的大小为 0.25。本文在实验中所使用的流量控制参数 δ 的大小为 0.125。

5.3.2 加速比

图5.6展示了 Re-Hot Cache 和 L1.5 Cache 相对于基准架构 (Baseline) 的加速比, GM 表示所有应用的几何平均值。L1.5 Cache 表示将重排序缓冲区的片上存储重配置为一级和二级缓存之间的中间缓存层级的架构。与 Baseline 相比, Re-Hot Cache 在所有应用和二级缓存高效型应用上分别实现了 4.04% 和 8.61% 的平均性能提升。在 gesu 上, 最高有着 14.4% 的性能提升。这种改进一方面得益于 Re-Hot Cache 过滤了对高访问频率的缓存行的访问, 缓解二级缓存争用问题, 另一方面得益于减少了高访问频率的缓存行的访问延迟。对于 Baseline, 在执行通用计算应用时图形固定功能流水线的重排序缓冲区的片上存储被闲置。L1.5 Cache 虽然将闲置的片上存储作为片上缓存被利用, 但是由于缓存过小, 有用的缓存行大多数已经被一级缓存所利用, 导致 L1.5 Cache 对性能几乎没有提升, 甚至部分应用会由于 L1.5 Cache 所增加的访存延迟而导致性能下降。

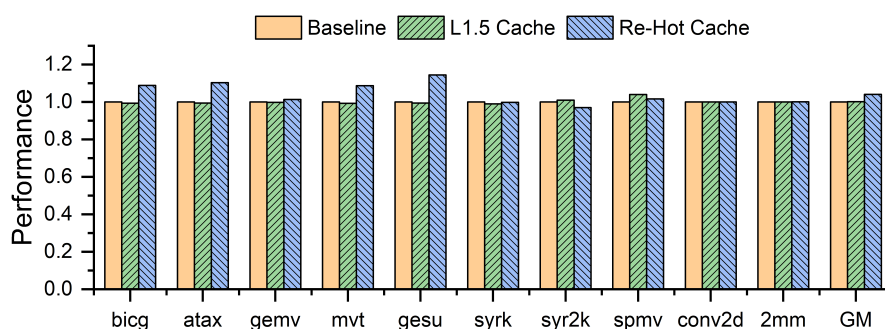


图 5.6 L1.5 Cache 和流量感知的可重配置热点缓存相对于传统 GPGPU 架构的加速比

Figure 5.6 Speedup of L1.5 Cache and Re-Hot Cache over traditional GPGPU architecture

5.3.3 缓存命中率

为了进一步分析 Re-Hot Cache 如何提高性能, 本文对缓存命中率进行了分析。图5.7显示了缓存命中和缺失的情况, 每个应用的左列是在 Baseline 上的结果, 右列是在 Re-Hot Cache 上的结果, GM 表示所有应用的几何平均值。访存请求可能在一级缓存上命中 (L1 Hit)、在 Re-Hot Cache 上命中 (Re-Hot Hit)、在二级缓存上命中 (L2 Hit)、或者全部缺失 (Miss)。虽然标记检查会使 Re-Hot Cache 的一级缓存缺失请求的平均访存延迟略微增加, 但是与二级缓存的平均访存延迟 (200 周期左右) 相比几乎可以忽略不计。平均有 20.4% 的访存请求在 Re-Hot Cache 上命中, 与 Baseline 相比, Re-Hot Cache 平均降低了 4.63% 的总的缓存缺失率, 减小了 4.63% 的片外内存访问。可以发现, 虽然 conv2d 的缓存命中率显著提升, 但是性能没有升高, 这是因为计算密集型的 conv2d 的性能瓶颈为计算

性能，缓存命中率的提高不能提高性能。Re-Hot Cache 过滤了二级缓存中访问频率高的缓存行的请求，使二级缓存中原本用来存储这些缓存行的空间可以用来存储其他缓存行，所以 Re-Hot Cache 隐式地增大了二级缓存的容量。总缓存缺失率的降低减少了对片外内存的访问，缓解了片外内存带宽所导致的瓶颈。

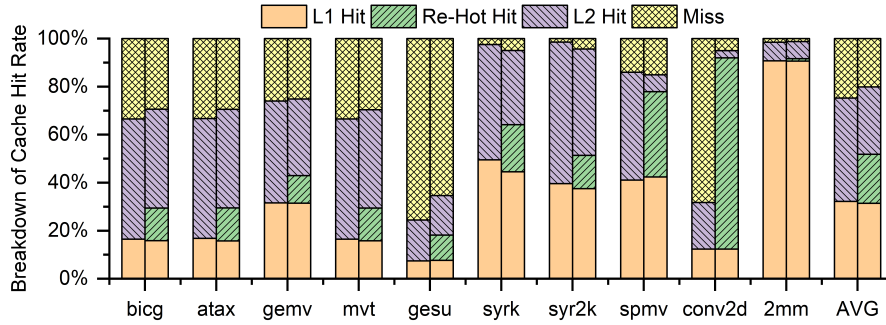


图 5.7 流量感知的可重配置热点缓存架构和传统 GPGPU 架构的缓存命中率的比较

Figure 5.7 Cache hit rate comparison of Re-Hot Cache and traditional GPGPU architecture

5.3.4 平均访存延迟

本文比较了 Re-Hot Cache 与 Baseline 的平均访存延迟，图5.8展示了实验结果，AVG 表示所有应用的平均值。与 Baseline 相比，Re-Hot Cache 平均降低了 10.7% 的平均访存延迟。Re-Hot Cache 在二级缓存高效型应用上，平均访存延迟有着明显的下降，使访存性能得到提升。Re-Hot Cache 的标记检查所增加的两周期的访存延迟相对于平均一至两百周期的访存延迟几乎可以忽略不计。在 syrk 和 syr2k 上，虽然平均访存延迟增加，但是性能只是略微降低。相比起在二级缓存高效型应用上带来的收益，损耗可以忽略不计。

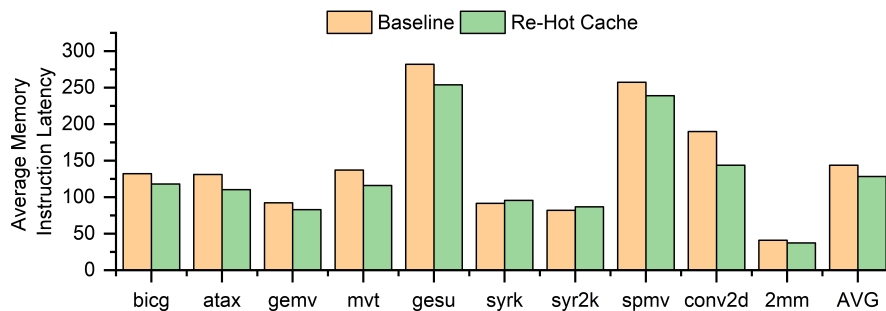


图 5.8 流量感知的可重配置热点缓存架构和传统 GPGPU 架构的平均访存延迟的比较

Figure 5.8 Comparison of average memory access latency of Re-Hot Cache and traditional GPGPU architecture

5.3.5 一级缓存大小对流量感知的可重配置热点缓存性能的影响

一级缓存大小在 Unified-Cache GPGPU 架构中可以根据应用特性进行配置。为了评估不同一级缓存大小对 Re-Hot Cache 的影响, 本文对二级缓存缓存高效型应用使用了不同大小的一级缓存进行了实验。图5.9显示, Re-Hot Cache 对 16KB 的一级缓存提升了 8.61% 的性能, 对 32KB 的一级缓存提升了 9.62% 的性能, 对 64KB 的一级缓存提升了 4.89% 的性能。即使是在更大的一级缓存下, 图形处理器仍然面临二级缓存争用问题, Re-Hot Cache 在 64KB 的一级缓存下对 gesu 仍有着 13.6% 的性能提升。

图5.9显示 Re-Hot Cache 在 16KB 的缓存上的性能提升反而比 32KB 的略低。这是因为 16KB 的一级缓存的缺失请求数远比 32KB 的多, 二级缓存收到的总请求数过多导致写入 Re-Hot Cache 的热点缓存行数量过多, 使 Re-Hot Cache 在 16KB 的一级缓存上反而会面临比 32KB 更严重的缓存争用问题。即使在具有较大一级缓存的情况下, Re-Hot Cache 也可以缓解二级缓存争用问题, 适用于二级缓存高负载的应用, 如 gesu。总体来说, Re-Hot Cache 是在大多数图形处理器架构中提高访存性能的有效方法。

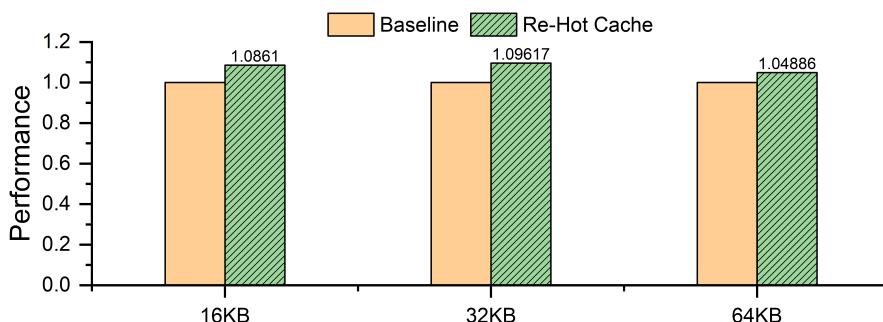


图 5.9 不同一级缓存大小的流量感知的可重配置热点缓存架构的性能比较

Figure 5.9 Performance comparison of Re-Hot Cache with varying L1 cache sizes

5.3.6 访存友好的通用图形处理器的性能分析

图5.10展示了访存友好的通用图形处理器与传统 GPGPU 相比在表4.1所示应用上的性能提升, GM 表示所有应用的几何平均值。传统 GPGPU 使用了分离的 64KB 的传统一级缓存和 64KB 的便笺式存储。访存友好的通用图形处理器使用了统一的 64KB 的一级缓存和 64KB 的便笺式存储, 在通用计算应用上, 分别应用了 Unified-Cache GPGPU、Decoupled-Tag Cache 和 Re-Hot Cache。在图形渲染应用上, 分别应用了 Unified-Cache GPGPU 和 Decoupled-Tag Cache。访存友好的通用图形处理器架构在所有应用上平均提高了 52.8% 的性能。在 spmv 上, 性

能提升幅度最大，提高至原本的 2.38 倍。在总共 15 个应用中，有 7 个应用的性能提升超过了 80%。总体上，访问友好的通用图形处理器设计可以显著且高效地提高通用图形处理器的性能。

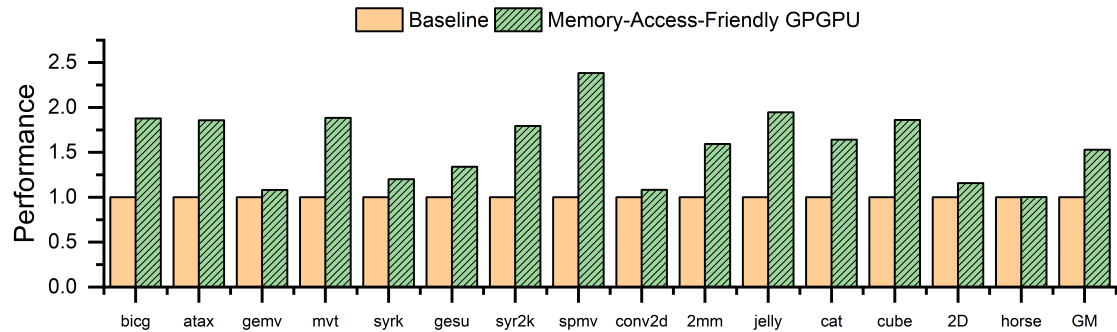


图 5.10 访存友好的通用图形处理器架构和传统 GPGPU 架构的性能比较

Figure 5.10 Performance comparison of memory-access-friendly general-purpose graphics processor architecture and traditional GPGPU architecture

5.4 本章总结

本文提出了流量感知的可重配置热点缓存架构（Re-Hot Cache），来降低通用计算应用的平均访存延迟。该架构利用闲置的图形固定功能流水线的像素着色重排序缓冲区的片上存储，在执行通用计算时重配置为热点缓存，用来存储二级缓存中高访问频率的缓存行，以降低高访问频率缓存行的访问延迟。并且提出了流量感知的动态热点缓存行评估方法，通过动态控制热点缓存行的数量，来提高 Re-Hot Cache 的效率。通过将同一片上存储在执行图形渲染和通用计算应用时分别配置为不同功能的单元，实现图形处理器片上存储资源的高利用率。实验结果显示，与基准架构相比，Re-Hot Cache 以较低的硬件开销在二级缓存高效型应用上平均有着 8.61% 的性能提升。

第6章 总结与展望

6.1 本文总结

随着数据量和计算需求的增加,传统中央处理单元已难以胜任复杂任务。为满足高性能计算需求,GPGPU 因其强大的并行处理能力成为首选。然而,GPGPU 在缓存和带宽上面临严重的争用问题。为加速 GPGPU 应用,本文从缓存命中率、平均并发访存请求数和平均访存延迟三个方面优化 GPGPU 访存架构。首先,通过统一图形渲染的 Shader IO 和通用计算的访存系统,提升片上存储利用率,增大缓存容量。其次,消除传统一级缓存架构导致的访存阻塞,增加平均并发访存请求数。最后,利用图形固定功能流水线的片上存储,设计可重配置的热点缓存,降低平均访存延迟。

为了增大缓存命中率,本文设计了统一缓存架构的通用图形处理器(Unified-Cache GPGPU)。Unified-Cache GPGPU 的设计理念是将缓存、便笺式存储和 Shader IOB 相互统一,并根据应用执行状态动态地分配和释放 Shader IOB,从而提高片上存储的利用率,在相同片上存储开销的情况下增大缓存容量。本文首先介绍 Unified-Cache GPGPU 的整体架构,然后详细说明了统一一级缓存和便笺式存储的实现,统一便笺式存储和 GPC 内 Shader IOB 的实现,以及统一二级缓存和 GPC 间 Shader IOB 的实现。与传统 GPGPU 相比,Unified-Cache GPGPU 在缓存高效型应用上平均提高了 35.4% 的性能。

为了增大平均并发访存请求数,本文设计了基于重命名的标记解耦合一级缓存(Decoupled-Tag Cache)。其设计理念包括三点:使用重命名方式将标记映射到缓存行,消除紧耦合组相联映射导致的访存阻塞;通过访存命中行为合并缺失请求,消除 MSHR 导致的访存阻塞;通过等待计数器判断请求是否完成,而不是在等待指令缓冲区为结果预留空间,从而可以使用大容量 SRAM 设计等待指令缓冲区,消除等待指令缓冲区导致的访存阻塞。为了进一步提升效率,本文分别为图形渲染和通用计算应用设计了顺序和乱序的基于重命名的标记解耦合一级缓存(IO-Decoupled-Tag Cache 和 OO-Decoupled-Tag Cache)。IO-Decoupled-Tag Cache 利用图形渲染应用按序处理几何图元的特点,顺序提交访存指令而几乎没有性能损失,以较低的面积开销提高图形渲染应用的性能。OO-Decoupled-Tag Cache 适应通用计算应用允许乱序完成线程及长延迟内存写和原子指令的特点,可以乱序提交访存指令,在通用计算应用上实现更高性能。实验表明,Decoupled-

Tag Cache 与传统一级缓存相比在图形渲染应用上可以提高超过 50% 的性能，在所有应用上平均提高了 30.0% 的性能。

为了降低平均访存延迟，本文设计了流量感知的可重配置热点缓存 (Re-Hot Cache)。通过将图形固定功能流水线重排序缓冲区的片上存储在执行通用计算时重配置为一个热点缓存，专门用来存储二级缓存的高访问频率缓存行，以缓解二级缓存争用问题并降低平均访存延迟。为提高热点缓存的效率，二级缓存不仅监测所有缓存行的访问频率，还监测写入热点缓存的流量，通过动态调整写入热点缓存的数据流量来优化 Re-Hot Cache 的性能。实验结果表明，Re-Hot Cache 与传统 GPGPU 架构相比以较低的硬件开销，平均提高了二级缓存高效型应用 8.61% 的性能。

6.2 展望

在此小节，本文将对访存友好的通用图形处理器架构的未来研究方向进行展望。

具体应用 目前统一缓存架构的通用图形处理器和顺序的基于重命名的标记解耦合的一级缓存已在龙芯处理器中得到应用，并有效提高了龙芯通用图形处理器的性能。接下来打算将乱序的基于重命名的标记解耦合的一级缓存和流量感知的可配置缓存架构应用于下一代龙芯处理器中。

像素着色任务的调度策略 目前，图元引擎采用轮询调度的策略，将 warp 轮流发送至流多处理器执行。然而，像素着色的纹理采样操作具有较高的空间局部性，相邻的像素通常访问内存中相邻的纹素。而轮询调度的策略会将相邻的两个 warp 发送至不同的流多处理器执行，导致两个 warp 会利用不同的一级缓存，破坏了相邻两个 warp 的空间局部性，使二级缓存会收到重复的缓存行的请求。如果将像素根据屏幕空间进行分类，将相同屏幕空间的像素打包的 warp 发送至相同的流多处理器执行，就可以充分利用像素间的空间局部性，从而减少重复的缺失请求的数量。

输出混合器和二级缓存的统一 通用计算在通用图形处理器的二级缓存引入了原子操作单元，原子操作单元有着较高的开销。图形固定功能流水线的输出混合器通过读-改-写操作来更新帧缓冲区，其操作本质上属于原子操作。目前，输出

混合器内部有独立的类似原子操作的执行单元，但是在通用图形处理器执行通用计算任务时被闲置。如果将输出混合器和二级缓存的原子操作执行单元相统一，使得输出混合器通过二级缓存来执行原子操作，就可以降低开销。

内存一致性 随着通用计算应用的发展，通用图形处理器需要考虑内存一致性的实现，以保证程序执行的正确性。对于流多处理器内部的相同地址的先读后写操作，由于缺失读请求和写请求分别沿片上互连网络的不同数据通路发送至二级缓存，二级缓存可能以先写后读的次序收到请求，导致读请求取回错误的数。为了解决这个问题，目前对于同一 warp 内的先读后写操作，通过显式的控制指令堵塞写请求，直至读请求完成。然而，指令流的阻塞会降低性能。如果能够将缺失读请求和写请求沿片上互连网络的相同数据通路发送至二级缓存，并且利用高带宽的写请求端口，在一个时钟周期发送更多的缺失读请求，就不仅可以提高性能，而且可以实现顺序一致性。

大型矩阵访问加速 通用图形处理器目前被广泛应用于深度学习领域。在深度学习应用中，经常需要访问大型矩阵数据。在目前的实现中，首先通过访存指令经过一级缓存将数据从内存搬运至寄存器，然后再通过指令将数据从寄存器搬运至便笺式存储，最后矩阵运算指令才能从便笺式存储读取数据进行计算。然而，大矩阵属于流式数据，具有较低的时间局部性，经过一级缓存时会将有用的缓存行数据驱逐出缓存，导致缓存低效。而且，数据从一级缓存到寄存器，再从寄存器到便笺式存储，不仅耗时，而且耗能。如果可以设计专门的指令直接将数据从内存搬运至便笺式存储，并绕过一级缓存，就可以有效提高性能并减少能耗。

参考文献

- [1] NVIDIA. NVIDIA: NVIDIA's Next Generation CUDA Compute Architecture: Kepler GK110 (whitepaper) [J]. 2012.
- [2] NVIDIA. NVIDIA Tesla V100 GPU Architecture Whitepaper [J]. 2017.
- [3] NVIDIA. NVIDIA Turing GPU architecture [J]. 2018.
- [4] NVIDIA. NVIDIA Ampere GA102 GPU Architecture [J]. 2020.
- [5] AMD. AMD Graphics Cores Next (GCN) Architecture (whitepaper) [J]. 2012.
- [6] Pomianowski A. RDNA 2 Gaming Architecture [C]//2021 IEEE Hot Chips 33 Symposium (HCS). IEEE, 2021: 1-18.
- [7] Singh I, Shriraman A, Fung W W, et al. Cache coherence for GPU architectures [C]//2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2013: 578-590.
- [8] Hechtman B A, Che S, Hower D R, et al. QuickRelease: A throughput-oriented approach to release consistency on GPUs [C]//2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2014: 189-200.
- [9] Bagsorkhi S S, Gelado I, Delahaye M, et al. Efficient performance evaluation of memory hierarchy for highly multithreaded graphics processors [J]. ACM SIGPLAN Notices, 2012, 47 (8): 23-34.
- [10] Brian K, Rune S, Graham W. A Deep Dive Into Unreal Engine's 5 Nanite [J]. 2021.
- [11] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks [J]. Advances in neural information processing systems, 2012, 25.
- [12] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition [C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [13] Wu Z, Pan S, Chen F, et al. A comprehensive survey on graph neural networks [J]. IEEE transactions on neural networks and learning systems, 2020, 32(1): 4-24.
- [14] Che S, Beckmann B M, Reinhardt S K, et al. Pannotia: Understanding irregular GPGPU graph applications [C]//2013 IEEE International Symposium on Workload Characterization (IISWC). IEEE, 2013: 185-195.
- [15] Li A, Song S L, Liu W, et al. Locality-aware CTA clustering for modern GPUs [J]. ACM SIGARCH Computer Architecture News, 2017, 45(1): 297-311.
- [16] Koo G, Oh Y, Ro W W, et al. Access pattern-aware cache management for improving data utilization in GPU [C]//Proceedings of the 44th annual international symposium on computer architecture. 2017: 307-319.
- [17] Oh Y, Kim K, Yoon M K, et al. APRES: Improving cache efficiency by exploiting load

- characteristics on GPUs [J]. ACM SIGARCH computer architecture news, 2016, 44(3): 191-203.
- [18] Nai L, Xia Y, Tanase I G, et al. GraphBIG: understanding graph computing in the context of industrial solutions [C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2015: 1-12.
- [19] Che S, Beckmann B M, Reinhardt S K, et al. Pannotia: Understanding irregular GPGPU graph applications [C]//2013 IEEE International Symposium on Workload Characterization (IISWC). IEEE, 2013: 185-195.
- [20] Eyerman S, Heirman W, Du Bois K, et al. Many-core graph workload analysis [C]//SC18: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2018: 282-292.
- [21] Chatterjee N, O'Connor M, Loh G H, et al. Managing DRAM latency divergence in irregular GPGPU applications [C]//SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2014: 128-139.
- [22] Pekhimenko G, Bolotin E, Vijaykumar N, et al. A case for toggle-aware compression for GPU systems [C]//2016 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2016: 188-200.
- [23] Koo G, Jeon H, Annavaram M. Revealing critical loads and hidden data locality in GPGPU applications [C]//2015 IEEE International Symposium on Workload Characterization. IEEE, 2015: 120-129.
- [24] Jog A, Kayiran O, Chidambaram Nachiappan N, et al. OWL: Cooperative thread array aware scheduling techniques for improving GPGPU performance [J]. ACM SIGPLAN Notices, 2013, 48(4): 395-406.
- [25] Kayiran O, Jog A, Kandemir M T, et al. Neither more nor less: Optimizing thread-level parallelism for GPGPUs [C]//Proceedings of the 22nd international conference on Parallel architectures and compilation techniques. IEEE, 2013: 157-166.
- [26] Rogers T G, O'Connor M, Aamodt T M. Cache-conscious wavefront scheduling [C]//2012 45th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE, 2012: 72-83.
- [27] Li D, Rhu M, Johnson D R, et al. Priority-based cache allocation in throughput processors [C]//2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2015: 89-100.
- [28] Zheng Z, Wang Z, Lipasti M. Adaptive cache and concurrency allocation on GPGPUs [J]. IEEE Computer Architecture Letters, 2014, 14(2): 90-93.
- [29] Oh Y, Koo G, Annavaram M, et al. Linebacker: Preserving victim cache lines in idle register files of GPUs [C]//Proceedings of the 46th International Symposium on Computer Architecture. 2019: 183-196.
- [30] Agarwal N, Nellans D, O'Connor M, et al. Unlocking bandwidth for GPUs in CC-NUMA

- systems [C]//2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2015: 354-365.
- [31] Agarwal N, Nellans D, Stephenson M, et al. Page placement strategies for GPUs within heterogeneous memory systems [C]//Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems. 2015: 607-618.
- [32] Bakhoda A, Kim J, Aamodt T M. Throughput-effective on-chip networks for manycore accelerators [C]//2010 43rd Annual IEEE/ACM international symposium on microarchitecture. IEEE, 2010: 421-432.
- [33] Kim H, Kim J, Seo W, et al. Providing cost-effective on-chip network bandwidth in GPGPUs [C]//2012 IEEE 30th International Conference on Computer Design (ICCD). IEEE, 2012: 407-412.
- [34] Seznec A. Decoupled sectored caches: conciliating low tag implementation cost [C]//Proceedings of the 21st annual international symposium on Computer architecture. 1994: 384-393.
- [35] Rhu M, Sullivan M, Leng J, et al. A locality-aware memory hierarchy for energy-efficient GPU architectures [C]//Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture. 2013: 86-98.
- [36] Ström J, Wennersten P, Rasmusson J, et al. Floating-point buffer compression in a unified codec architecture [C]//Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on Graphics hardware. 2008: 75-84.
- [37] Samadi M, Lee J, Jamshidi D A, et al. Sage: Self-tuning approximation for graphics engines [C]//Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture. 2013: 13-24.
- [38] Vijaykumar N, Pekhimenko G, Jog A, et al. A case for core-assisted bottleneck acceleration in GPUs: enabling flexible data compression with assist warps [J]. ACM SIGARCH Computer Architecture News, 2015, 43(3S): 41-53.
- [39] Sathish V, Schulte M J, Kim N S. Lossless and lossy memory I/O link compression for improving performance of GPGPU workloads [C]//Proceedings of the 21st international conference on Parallel architectures and compilation techniques. 2012: 325-334.
- [40] Hsieh K, Ebrahimi E, Kim G, et al. Transparent offloading and mapping (TOM) enabling programmer-transparent near-data processing in GPU systems [J]. ACM SIGARCH Computer Architecture News, 2016, 44(3): 204-216.
- [41] Pattnaik A, Tang X, Jog A, et al. Scheduling techniques for GPU architectures with processing-in-memory capabilities [C]//Proceedings of the 2016 International Conference on Parallel Architectures and Compilation. 2016: 31-44.
- [42] Song L, Zhuo Y, Qian X, et al. GraphR: Accelerating graph processing using ReRAM [C]//

- 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2018: 531-543.
- [43] Microsoft. DirectX-Specs [EB/OL]. 2014. <https://microsoft.github.io/DirectX-Specs/>.
- [44] Williams L. Pyramidal parametrics [C]//Proceedings of the 10th annual conference on Computer graphics and interactive techniques. 1983: 1-11.
- [45] Beers A C, Agrawala M, Chaddha N. Rendering from compressed textures [C]//Proceedings of the 23rd annual conference on Computer graphics and interactive techniques. 1996: 373-378.
- [46] Manavski S A. CUDA compatible GPU as an efficient hardware accelerator for AES cryptography [C]//2007 IEEE International Conference on Signal Processing and Communications. IEEE, 2007: 65-68.
- [47] Michalakes J, Vachharajani M. GPU acceleration of numerical weather prediction [C]//2008 IEEE International Symposium on Parallel and Distributed Processing. IEEE, 2008: 1-7.
- [48] Wu E, Liu Y, Liu X. An improved study of real-time fluid simulation on GPU [J]. Computer Animation and Virtual Worlds, 2004, 15(3-4): 139-146.
- [49] Yang J, Wang Y, Chen Y. GPU accelerated molecular dynamics simulation of thermal conductivities [J]. Journal of Computational Physics, 2007, 221(2): 799-804.
- [50] NVIDIA. GeForce256 [EB/OL]. <https://www.techpowerup.com/gpu-specs/geforce-256-sdr.c731>.
- [51] Kirk D. GeForce3 architecture overview [Z]. 2001.
- [52] Igehy H, Eldridge M, Proudfoot K. Prefetching in a texture cache architecture [C]//Proceedings of the ACM SIGGRAPH/EUROGRAPHICS workshop on Graphics hardware. 1998: 133-ff.
- [53] Lindholm E, Nickolls J, Oberman S, et al. NVIDIA Tesla: A unified graphics and computing architecture [J]. IEEE micro, 2008, 28(2): 39-55.
- [54] Kirk D B, Wen-Mei W H. Programming massively parallel processors: a hands-on approach [M]. Morgan kaufmann, 2016.
- [55] Tsuchiyama R, Nakamura T, Iizuka T, et al. The opencl programming book [J]. Fixstars Corporation, 2010, 63: 11.
- [56] Wittenbrink C M, Kilgariff E, Prabhu A. Fermi GF100 GPU architecture [J]. IEEE Micro, 2011, 31(2): 50-59.
- [57] NVIDIA. NVIDIA GeForce GTX 980: Featuring Maxwell, The Most Advanced GPU Ever Made (whitepaper) [J]. 2013.
- [58] NVIDIA. NVIDIA Tesla P100: The Most Advanced Datacenter Accelerator Ever Built (whitepaper) [J]. 2016.
- [59] NVIDIA. NVIDIA DGX-1 System Architecture White Paper [J]. 2017.
- [60] Foley D, Danskin J. Ultra-performance Pascal GPU and NVLink interconnect [J]. IEEE Micro, 2017, 37(2): 7-17.
- [61] Gebhart M, Johnson D R, Tarjan D, et al. Energy-efficient mechanisms for managing thread

- context in throughput processors [C]//Proceedings of the 38th annual international symposium on Computer architecture. 2011: 235-246.
- [62] Narasiman V, Shebanow M, Lee C J, et al. Improving gpu performance via large warps and two-level warp scheduling [C]//Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture. 2011: 308-317.
- [63] Yoon M K, Oh Y, Lee S, et al. Draw: investigating benefits of adaptive fetch group size on gpu [C]//2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). IEEE, 2015: 183-192.
- [64] Lee M, Song S, Moon J, et al. Improving GPGPU resource utilization through alternative thread block scheduling [C]//2014 IEEE 20th international symposium on high performance computer architecture (HPCA). IEEE, 2014: 260-271.
- [65] Rogers T G, O'Connor M, Aamodt T M. Divergence-aware warp scheduling [C]//Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture. 2013: 99-110.
- [66] Chen X, Wu S, Chang L W, et al. Adaptive cache bypass and insertion for many-core accelerators [C]//Proceedings of International Workshop on Manycore Embedded Systems. 2014: 1-8.
- [67] Chen X, Chang L W, Rodrigues C I, et al. Adaptive cache management for energy-efficient GPU computing [C]//2014 47th Annual IEEE/ACM international symposium on microarchitecture. IEEE, 2014: 343-355.
- [68] Gebhart M, Keckler S W, Khailany B, et al. Unifying primary cache, scratch, and register file memories in a throughput processor [C]//2012 45th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE, 2012: 96-106.
- [69] Jing N, Wang J, Fan F, et al. Cache-emulated register file: An integrated on-chip memory architecture for high performance GPGPUs [C]//2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). IEEE, 2016: 1-12.
- [70] Jang H, Kim J, Gratz P, et al. Bandwidth-efficient on-chip interconnect designs for GPGPUs [C]//Proceedings of the 52nd Annual Design Automation Conference. 2015: 1-6.
- [71] Ziabari A K, Abellán J L, Ma Y, et al. Asymmetric NoC architectures for GPU systems [C]//Proceedings of the 9th International Symposium on Networks-on-Chip. 2015: 1-8.
- [72] Yuan G L, Bakhoda A, Aamodt T M. Complexity effective memory access scheduling for many-core accelerator architectures [C]//Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture. 2009: 34-44.
- [73] Memory access scheduling, author=Rixner, Scott and Dally, William J and Kapasi, Ujval J and Mattson, Peter and Owens, John D [J]. ACM SIGARCH Computer Architecture News, 2000, 28(2): 128-138.
- [74] Yazdanbakhsh A, Pekhimenko G, Thwaites B, et al. RFVP: Rollback-free value prediction with safe-to-approximate loads [J]. ACM Transactions on Architecture and Code Optimization (TACO), 2016, 12(4): 1-26.

- [75] Segal M, Akeley K. The OpenGL Graphics System: A Specification (Version 4.6 (Core Profile)-October 22, 2019) [J]. 2021.
- [76] Grauer-Gray S, Xu L, Searles R, et al. Auto-tuning a high-level language targeted to GPU codes [C]//2012 innovative parallel computing (InPar). Ieee, 2012: 1-10.
- [77] Stratton J A, Rodrigues C, Sung I J, et al. Parboil: A revised benchmark suite for scientific and commercial throughput computing [J]. Center for Reliable and High-Performance Computing, 2012, 127: 27.
- [78] Github. glmark2 [EB/OL]. <https://github.com/glmark2/glmark2>.

致 谢

读博士是我人生中一次至关重要的历练，这段旅程堪称精益求精的探索。前四年，我专注于工程实践，通过不懈的努力，掌握了大量工程技能，并获得了丰富的科研产业化经验。在这个阶段，我结交了众多经验丰富的工程师，他们的指导和经验对我产生了深远的影响。最后一年，我将重心转向学术研究，致力于掌握学术研究的方法。这段时间，我深刻认识到了学术研究的广阔视野，激励和启发我不断进步。在博士五年的历程中，我经历了无数次的挑战和磨炼，但也因此蜕变成长。我不断调整学习和工作方式，不断追求更高的目标。这段时间里，我的努力、奋斗和坚持见证了我的成长与进步。最后，我要衷心感谢我的家人，他们是我生命中最坚实的后盾。他们给予了我无尽的鼓励和支持，在我追求梦想的道路上一路陪伴着我，让我始终坚定前行，勇往直前。没有他们的支持，我无法完成这段充满挑战的旅程。

我要衷心感谢各位老师，在科研道路上不断给予我指引和支持。您们就像黑夜中的孔明灯，照亮了前行的道路，让我少走弯路，并不断精益求精。特别感谢我的导师胡伟武老师，能够在茫茫人海中遇到您是我的幸运。胡老师的细心指导，使我得以顺利完成博士课题。胡老师在学术上给予我悉心的指导，我将永远铭记在心，我从您身上领略顶尖学者的学术风范和从您身上学习珍贵的学术研究经验。在我朦胧中，感谢苏孟豪老师给我指明了博士期间的研究方向，让我投身于通用图形处理器架构的研究课题中，我从您身上领略顶尖 SoC 工程师的风范和从您身上学习宝贵的芯片设计经验。我要感谢王剑、张福新、章隆兵、肖俊华等老师，感谢他们在我生活上和科研上无私的奉献，为我营造良好的学习环境和为我的成长供给充足的养分。

感谢张立志、赵士彭、刘苏等博士在为人处世和学术上的指导。感谢张立志博士，作为你的师弟，在各方面都得到你的帮助，感谢你处于视野井底和学术谷底的我拉上去。感谢赵士彭博士，作为你的舍友，非常感谢你在生活和学术上的帮助。感谢刘苏博士，作为您的师弟，非常感谢您在工作 and 学术上的帮助。

感谢与我同年入学的王睿、侯邦、王重熙、郭振江、兰彦志、李昱辰、张嘉辉、赵天伟同学，感谢你们的陪伴，让我的博士生涯充满了乐趣。感谢邢世远、谢本壹、谭弘泽等师兄、感谢李文青、王雪、郑雅文等师姐，感谢李冬、刘洋、宋鹏皓等师弟，谢谢他/她们对我的帮助和指导。感谢蔡飞、陈为、姜文奇、宫一

夫、石瑞恺等同事在工作中的帮助。感谢教育处的冯钢、周世佳、李慧等老师，以及微处理器研究中心的高燕萍等秘书，感谢他/她们在我学业上的帮助。感谢各位博士论文审阅老师，感谢他们的悉心指导和宝贵建议。感谢中国科学院大学、中国科学院计算技术研究所和龙芯中科技术有限公司给与我们成长的良好环境和美好的学生生涯回忆。

最后，在这里我要感谢我自己，感谢你的不屈不挠和永不言败，请你继续努力前进，不断进取。

作者简历及攻读学位期间发表的学术论文与研究成果

【基本情况】

姓名: 赵皓宇 性别: 男 出生日期: 1997.02.09 籍贯: 吉林省通化市

【教育经历】

2019 年 9 月–2024 年 6 月: 中国科学院计算技术研究所, 攻读博士学位

2015 年 9 月–2019 年 7 月: 中国科学技术大学计算机学院, 获得学士学位

【攻读博士学位期间发表的论文】

1. Zhao, Haoyu, Longbing Zhang, and Fuxin Zhang. "RBGC: Repurpose the Buffer of Fixed Graphics Pipeline to Enhance GPU Cache." Proceedings of the Great Lakes Symposium on VLSI 2023. 2023. (CCF C 类会议)
2. 赵皓宇, 王重熙, 宋鹏皓, 章隆兵. “基于无裁剪图形流水线的三维图形处理器”, 高技术通讯, 2024. (CCF C 类国内期刊)

【攻读博士学位期间的获奖情况】

2024 年获得中国科学院计算技术研究所“所长奖学金优秀奖”

